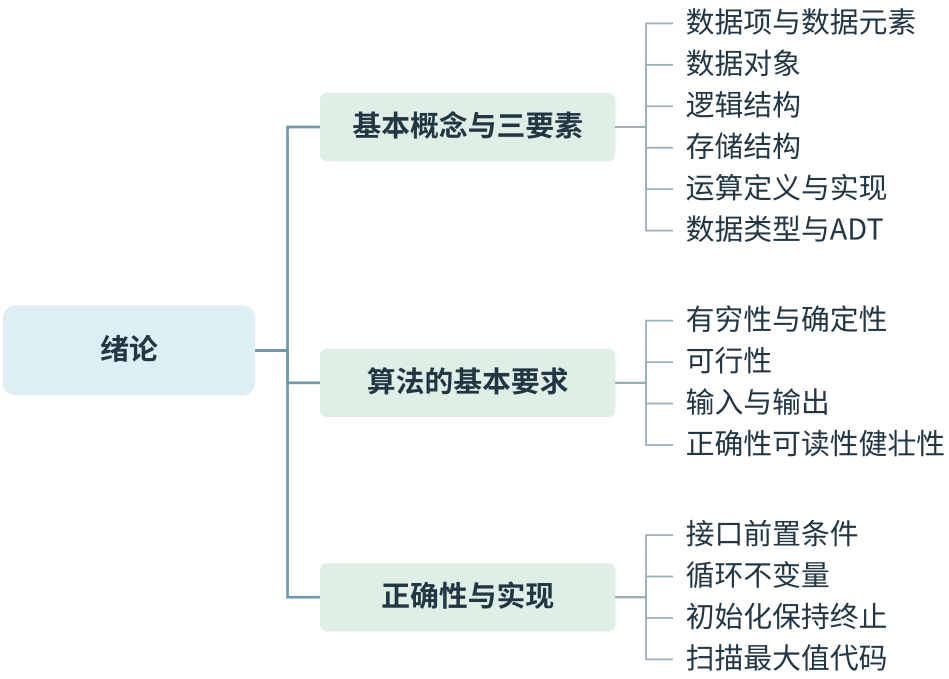
# 绪论 1

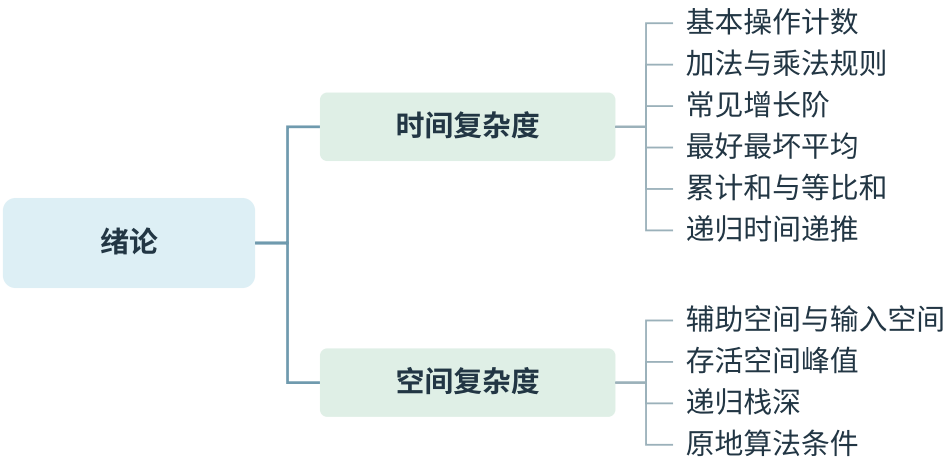
**知识导图**




**错项排除·有穷不等于很快**

执行 2 的 n 次方步，只要对每个合法有限 n 都能结束，仍可满足有穷性。可行性也不是“考试规模下跑得快”；先判基本特性，再评价实际效率，不要混用两个层次。

**知识导图（续）**




**错项排除·上界可以不紧**

线性时间当然满足 $O(n^2)$，但选择题通常要求紧数量级。若求和结果是 $1+2+4+\cdots$，用最大项乘项数会得到合法却不紧的上界；还要检查能否消去多余的对数因子。

本书中，$n$ 表示输入规模。数组默认从 0 开始编号；线性表的 “第 $i$ 个元素” 默认采用从 1 开始的位序，二者同时出现时会写明对应关系。指针变量保存结点地址，`p->next` 是 `p` 所指结点的后继指针域；`p=NULL` 表示 `p` 当前不指向任何有效结点。伪代码中的赋值号 `=` 不表示数学等式，循环边界和执行顺序同样是算法的一部分。

**不变量·累计值与轮数不同**

第 k 轮结束时 sum=k(k+1)/2，循环次数是满足停止不等式的最小 k。sum 约等于 n 不代表循环 n 次；每轮增量逐步变大，正是平方根阶出现的原因。

## 1.1 逻辑关系怎样落实到存储

**数据元素**是数据的基本单位，一个元素可由若干数据项组成。例如一条学生记录是一个元素，学号和姓名是它的数据项。**数据对象**是性质相同的数据元素的集合。讨论数据结构时，还必须说明元素之间的关系：是排成先后序列，形成父子层次，还是由任意边相连。

**逻辑结构**描述元素之间的逻辑关系，不涉及具体地址。集合结构没有规定元素间的先后关系；线性结构中，除首尾元素外，每个元素都有一个直接前驱和一个直接后继；树形结构表达层次；图结构表达更一般的关联。栈和队列都以线性关系为基础，但允许的操作位置不同，因此属于不同的抽象数据类型。

**存储结构**是逻辑结构在计算机中的表示。顺序存储用相邻位置体现关系，链式存储用指针联系结点，索引存储另建索引以定位元素，散列存储由关键字计算位置。四者都只是表示方法，不能把 “线性结构” 直接等同于 “连续地址”。线性表既能顺序存储，也能链式存储。

### 1.1.1 例 1-1　值以外还要存什么

**例 1.1.1** (配套例题)

〖1.1-005〗在存储数据时，通常不仅要存储各数据元素的值，还要存储（　）。

A. 数据的操作方法　B. 数据元素的类型　C. 数据元素之间的关系　D. 数据的存取方法

**解析**。若只保存若干值，就无法区分这些值原本属于一个序列、一棵树还是一张图。存储结构必须表达元素间的关系，答案为 **C**。关系未必另占一个专门字段：数组的相邻位置已隐含先后，链表则用指针显式记录后继。

运算的定义与实现也要分开。线性表的 “按位序取值” 是一种抽象运算；在数组上用下标访问、在链表上顺指针查找，则是两种实现。选择存储结构时，要同时考虑需要保存的关系和需要执行的运算，不能只看对象包含多少个元素。

**抽象数据类型（ADT）**把数据对象、关系和允许的操作合在一起描述，而不限定具体的机器实现。例如，栈规定 `push`、`pop` 及后进先出语义；顺序栈和链栈都能实现这一约定。

## 1.2 算法怎样保证结束并得到正确结果

算法是求解一类问题的有限操作序列，通常要求有穷性、确定性、可行性、输入和输出：对合法输入，执行有限步后结束；每一步含义明确，并能在有限时间内完成；可以有零个或多个输入，至少有一个输出。输出不限于屏幕打印，也可以是返回值或修改后的数据状态。

算法的有穷性针对**每一次合法输入**。运行时间很长，仍可能满足有

穷性；没有终止条件的服务程序可以是有用的程序，却不属于这里所说的有限求解算法。可行性要求操作能够执行，并不要求它在现实规模下足够快。正确、可读、健壮以及时空效率是进一步评价设计的维度；“必须具备的特性”与“希望达到的设计质量”属于不同层面的问题。

证明循环正确性，可以找一个每轮前后都保持的关系，称为**循环不变式**。例如，依次扫描数组求最大值：处理前 $i$ 个元素后，`max` 等于这 $i$ 个元素的最大值。证明分三步：初始化时说明该关系为何成立；每轮用下一个元素更新后说明它仍成立；终止时 $i=n$，由不变式可知结果覆盖了全体元素。此外还要说明循环变量不断接近边界，以排除关系成立却永不终止的程序。

空表、单元素、重复元素和查找失败都不是额外装饰。若把初始化写成 `max=A[0]`，输入规模就必须满足 $n \ge 1$；若允许空数组，应另行约定失败时的返回值，不能继续访问 `A[0]`。

**不变量·先列每轮工作量**

外层 i 取 1、2、4 等值，内层工作量恰与 i 相同。用等比和可知所有小轮之和仍受末轮的常数倍控制；“两层循环”没有提供平方阶的充分条件。

## 1.3 怎样由执行次数得到时间复杂度

实际运行时间受机器、编译器和输入分布影响。比较算法的增长规律时，先选择一种基本操作，统计它的执行次数 $T(n)$，再研究 $n$ 增大时的数量级。本书采用单位成本模型：定长整数的比较、赋值及一次数组访问都视为常数时间。若整数位数随输入增长，或者需要访问外存，这个假设就要重新检查。

$T(n)=O(f(n))$ 给出渐近上界；$T(n)=\Omega(f(n))$ 给出渐近下界；上下界同阶时记为 $\Theta(f(n))$。选择题通常用 $O$ 表示所要求的紧数量级。$3n^2+7n+9=\Theta(n^2)$：当规模足够大时，低次项和常数因子都不会改变增长阶。把 $O(n)$ 写成 $O(2n)$ 并没有更精确地给出渐近阶。

### 1.3.1 先看循环变量如何变化

`i++` 经过 $n$ 次增长到 $n$；`i*=2` 则约经 $\log_2 n$ 次到达循环边界。分析嵌套循环时，不能只数层数。若内层范围依赖外层变量，应先写出每轮次数再求和。例如：

```
for i = 1 to n:
    for j = 1 to i:
        执行一次基本操作
```

总次数为 $1+2+\cdots+n=n(n+1)/2$，所以是 $\Theta(n^2)$。外层和内层各有一个变量，并不意味着次数总是 $n\times n$；本例只是恰好达到平方阶。

### 1.3.2 例 1-2　循环由累计和决定何时停止

**例 1.3.1** (配套例题)

【2017】下列函数的时间复杂度是（ ）。

```
int func(int n) {
    int i=0, sum=0;
    while (sum<n) sum += ++i;
    return i;
}
```

A. $O(\log n)$　B. $O(n^{1/2})$　C. $O(n)$　D. $O(n\log n)$

**解析**。条件检查的是 `sum`，应先把它写成循环次数的函数。执行 $k$ 轮后，`i=k`，$\text{sum} = 1+2+\cdots+k = k(k+1)/2$。终止时，$k$ 是满足 $k(k+1)/2 \geq n$ 的最小非负整数，即

$$k = \left\lceil \frac{\sqrt{1+8n}-1}{2} \right\rceil .$$

每轮只做常数次操作，故 $k = \Theta(\sqrt{n})$，答案为 **B**。`sum` 最终约为 $n$，不能据此推出执行了 $n$ 轮——每轮的增量本身也在增长。分析按题目给定的整数模型进行；实际实现还应防止固定字长溢出。

### 1.3.3 例 1-3　两层循环为什么仍是线性时间

**例 1.3.2** (配套例题)

【2022】下列程序段的时间复杂度是（ ）。

```
int sum=0;
for (int i=1; i<n; i*=2)
    for (int j=0; j<i; j++)
        sum++;
```

A. $O(\log n)$　B. $O(n)$　C. $O(n\log n)$　D. $O(n^2)$

**解析**。外层的取值为 $1, 2, 4, \ldots, 2^k$，其中 $2^k < n \leq 2^{k+1}$。各轮内层次数正好为这些值，总计

$$1+2+4+\cdots+2^k = 2^{k+1}-1.$$

该和小于 $2n$，而当 $n>1$ 时又不小于 $n$ 的某个常数倍，故为 $\Theta(n)$，答案为 **B**。若把每一轮都估成最后一轮的 $O(n)$，再乘以外层轮数，得到的只是不紧的 $O(n\log n)$ 上界，忽略了前几轮内层规模很小这一事实。

### 1.3.4 最好、最坏与平均情况在比较什么

线性查找长度为 $n$ 的表：目标在首位时比较 1 次；在末位或不存在时，通常比较 $n$ 次。这是相同规模下不同输入造成的差异。**最坏时间**取规模为 $n$ 的合法输入中的最大执行次数；**最好时间**取最小；**平均时间**还须说明输入的概率分布。不能把“平均”直接理解为最好与最坏的算术平均。

若 $n$ 个元素被查找的概率相同，并且保证查找成功，则顺序查找的平均比较次数为 $(n+1)/2$。若还可能失败，就要把失败概率及其代价计入；第 7 章将按查找模型分别计算。

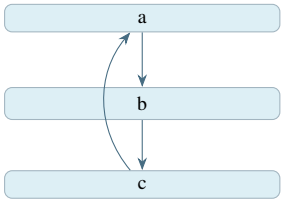
**课程连接·同一关系，多种表示**

操作系统的空闲块可用位图或链表记录，表达的都是“哪些块可用”。更换表示会改变定位和更新代价。复习时把逻辑关系与地址表示分开，才能解释同一功能为何有不同算法。

递归算法先写出递推关系。例如，若每次只递归求解规模为 $n-1$ 的一个子问题，并做常数工作，则 $T(n)=T(n-1)+\Theta(1)=\Theta(n)$。若产生两个规模为 $n/2$ 的子问题，并在本层做线性合并，则 $T(n)=2T(n/2)+\Theta(n)$：每层总工作为 $\Theta(n)$，约有 $\log_2 n$ 层，总时间为 $\Theta(n\log n)$。不能只看到规模减半就写成对数时间；每层有多少个子问题同样决定总量。

**最小反例·链式不等于树**

三个结点 a→b→c→a 可以构成循环单链表的物理链接，链接图含环，不能据此认定为树。采用指针联系只是存储表示；逻辑对象究竟是线性表、树还是图，还需查看关系与操作约定。




循环链接含环，链式存储不能确定树关系

## 1.4 辅助空间中为何要计入递归栈

**辅助空间复杂度**衡量算法运行时所需的额外空间随输入规模增长的情况，一般不计输入本身占用的空间；题目对输出区是否计入另有约定时，按题意处理。本书比较排序时把归并辅助数组计入额外空间，不重复计算原数组。

几个循环变量只占 $\Theta(1)$ 空间，循环执行一万次也不需要同时保存一万个变量。相反，递归调用尚未返回时，调用记录必须同时保留参数、返回位置和局部状态。递归求和每层只用常数空间，但同一时刻最多存在 $n$ 层调用，因此辅助空间为 $\Theta(n)$；时间与空间同阶只是这个例子的特点，并非所有递归都如此。

若一次调用先处理左半、返回后再处理右半，左右子树的递归调用就不会全程同时展开。二叉树遍历的调用总数为 $\Theta(n)$，最大栈深为树高 $h$，故递归辅助空间是 $O(h)$。完全二叉树中 $h=\Theta(\log n)$；单支链中 $h=\Theta(n)$。这里不能用“访问了 $n$ 个结点”代替“同时保留了多少层状态”。

**原地算法**通常允许常数个辅助单元。只数递归代码中有几个显式变量而忽略调用栈，就可能误判为原地算法。快速排序即使在原数组上划分，递归栈仍要计入：普通实现平均 $O(\log n)$，最坏 $O(n)$；第 8 章会解释两者分别对应怎样的划分。

## 1.5 由导图辨清对象、结构、类型与运算

**定义 1.5.1** (数据结构的三要素)

一个数据结构要同时回答：对象中有哪些元素，元素之间是什么逻辑关系，需要提供哪些运算。存储结构负责把这些关系及操作落实为计算机中的表示与实现。数据项是构成元素的不可分割单位；记录由多个数据项构成，却仍可作为一个元素整体处理。

数据类型由一个值的集合及定义在该集合上的操作组成。原子类型的值不再分解，结构类型的值可分解为成分；抽象数据类型进一步强调对使用者可见的对象、关系和操作语义，将实现细节藏在接口之后。逻辑结构与存储结构不是一一对应：同一逻辑结构可以有多种表示，同一种链式表示也能用于线性表、树、图。

| 判断维度 | 需要说明什么 | 例子 | 不能直接推出什么 |
|---|---|---|---|
| 逻辑结构 | 元素间关系 | 序列、层次、任意关联 | 是否使用连续地址 |
| 存储结构 | 地址与链接的表示 | 顺序、链式、索引、散列 | ADT 允许的操作一定相同 |
| 运算定义 | 输入、输出与前后条件 | 取第 i 个元素、插入 | 具体执行多少次访问 |
| 运算实现 | 在已定表示上怎样执行 | 数组寻址、逐结点扫描 | 另一种表示下代价也相同 |

## 1.6 用一个完整算法连接特性、正确性与代价

**先写接口契约，再写循环**

以求数组最大值为例：合法输入为长度 $n \geq 1$ 的有效数组；返回值应等于数组中某个元素，且不小于任意元素。允许空输入时，必须把失败作为接口的一部分；不能凭空取一个数充当所有整数数组的最大值。

不变量·已处理区的最大值

进入第 i 轮前，best 只承诺等于前 i 个元素的最大值，并没有承诺覆盖未扫描区。证明每轮将承诺扩展一个元素，比直接说 “循环完一定正确” 多出了保持关系这一关键环节。

```cpp
#include <cstddef>
bool maximum(const int* a, std::size_t n, int& answer) {
    if (n == 0 || a == nullptr) return false;
    int best = a[0];
    for (std::size_t i = 1; i < n; ++i)
        if (a[i] > best) best = a[i];
    answer = best;
    return true;
}
```

上面的 C++17 接口以布尔返回值区分 “空数组没有最大值” 和 “最大值恰为 0”，失败时不修改 answer。若 a 非空，还要求它实际指向至少 n 个有效元素；仅检查指针非空并不能检验其可访问长度。后续指针操作同样必须满足有效结点前提。

**扫描最大值的循环证明**

进入第 $i$ 轮前，不变量为 `best=max(a[0..i-1])`。初始化 $i = 1$ 时，唯一已处理元素为 $a[0]$；比较 $a[i]$ 后取较大者，不变量扩展到前 $i+1$ 个元素。$i$ 严格递增且上界为 $n$，故循环终止；终止时处理全部元素，返回正确最大值。非空输入恰有 $n-1$ 次元素大小比较，所以时间为 $\Theta(n)$，额外存储为 $\Theta(1)$。

□

## 1.7 渐近分析的运算规则及适用条件

**定理 1.7.1** (加法取主阶，嵌套先计实际次数)

若两段代码依次执行，时间为 $T_1(n) + T_2(n)$，可按最大数量级归并；若一段工作被重复 $k(n)$ 次且每次代价有统一紧界，才可直接相乘。内层范围随外层变化时，先写求和。常数底的对数只相差常数因子。

常见增长次序（$c > 1$ 为固定常数）为

$$1 \prec \log n \prec n \prec n \log n \prec n^2 \prec n^3 \prec c^n \prec n!.$$

$O$ 上界并不唯一：线性算法当然也属于 $O(n^2)$，但题目通常要求最紧的常见阶。最好/最坏描述同规模输入的选择，$O/\Omega/\Theta$ 描述函数之间的界，不能将两组概念机械配对为 “O 只表示最坏”。

空间必须数**同一时刻仍存活的状态**。两段代码顺序使用并释放临时数组时，峰值可取各段最大值；两个数组同时保留时才相加。循环内每轮复用一个长度 $n$ 的临时数组，即使循环 $n$ 轮，峰值仍为 $\Theta(n)$ 而非 $\Theta(n^2)$。递归每层若分配不同规模的数组，必须沿一条同时活跃的调用

链求和，而非只用层数乘最外层空间。

**错项排除·效率不是唯一质量**

一个输出错误的程序即使时间和空间都很少，也没有解决目标问题。遇到用效率“全面衡量优劣”的说法，应先检查正确性等前提；正确性、可读性、健壮性不能被遗漏。

## 1.8 历史错题指向的辨析

算法必须具备五项基本特性；正确性、可读性、健壮性以及时空效率则是进一步评价设计的维度。CPU 实际耗时受机器与实现影响，并非只由抽象算法决定；也允许以更多空间换取更少时间。配套单选对“衡量优劣”的概括存在措辞歧义，复习时应掌握这几个层次的区别，不把时空效率当作全部质量条件。做复杂度题则先找真正的停止量，列累计和或等比和，避免将变量最终值直接当作循环次数。

## 1.9 主动回忆自测

第 1 题为配套题，其余为自拟补充。先遮住答案，说明停止条件、存活空间与表示关系。

**合上正文，沿导图复述**

1. 【2019】设 n 是描述问题规模的非负整数，程序段 `x=0; while(n>=(x+1)*(x+1)) x=x+1;` 的时间复杂度是（　）。A. $O(\log n)$　B. $O(n^{1/2})$　C. $O(n)$　D. $O(n^2)$
2. 一段算法循环调用一个无递归函数 $n$ 次，每次函数使用 10 个整数局部变量。仅考虑这些局部变量，辅助空间是 $O(n)$ 还是 $O(1)$？
3. 一个对象采用链式存储，能否据此断定它的逻辑结构是一棵树？
4. 数据项、数据元素、数据对象之间是什么关系？ADT 是否规定必须用数组实现？
5. 算法的五项特性与正确性、健壮性有什么区别？“输入”必须从键盘读入吗？
6. 单个循环变量连续翻倍，和外层翻倍但内层执行 i 次，两者为什么不是同一个阶？
7. 顺序执行两段各需 n 个临时单元的代码，峰值空间一定是 2n 吗？
8. 递归每次将规模减半、每层只用常数空间，时间是否一定是对数阶？还缺什么信息？

### 1.9.1 核对答案

**先核条件，再核结论**

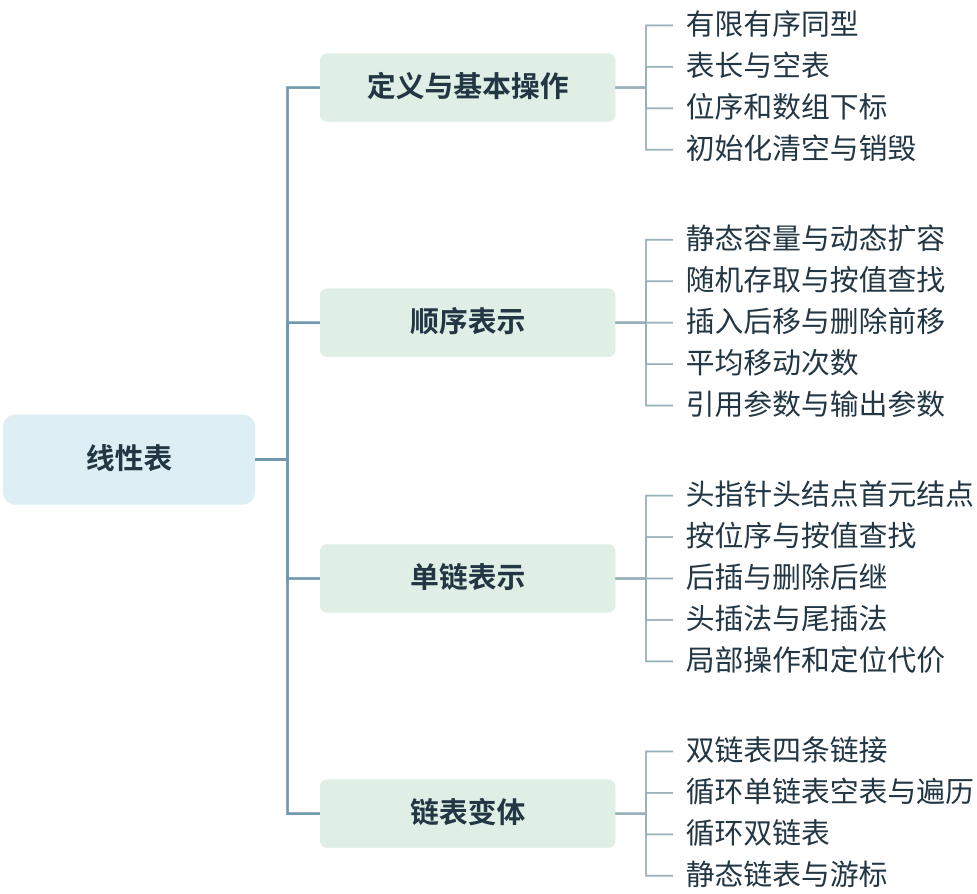
1. **B**。次数恰为 $\lfloor\sqrt{n}\rfloor$：终止时 $x^2 \le n < (x+1)^2$，仍为平方根数量级，但停止方程与例 1-2 的累计和不同。2. $O(1)$：前一次调用返回后，局部空间可以复用。3. 不能。链式存储只是一种表示方法，线性表、树和图都可以采用。
2. 元素是处理的基本单位，由数据项组成；对象是同性质元素的集合。ADT 规定抽象语义而不限定表示。
3. 五项特性描述作为有限求解算法的要求；正确性等描述设计质量。输入可由参数、已存数据等提供，并不限定交互方式。
4. 前者轮数为对数；后者基本操作数为 1+2+4+$\cdots$，是线性量级。
5. 不一定。若前段的临时空间结束后可复用，峰值只需 n 个量级；若两段结果必须同时保留才相加。两者渐近阶均为 Θ(n)，但实际峰值不同。
6. 还要知道每层产生几个子问题、每个调用做多少非递归工作。两个半规模递归和一个半规模递归具有不同调用总量。

**最小反例·零不能当通用初值**

数组只有一个元素 $-3$ 时，若 best 初始化为 0 并只在出现更大元素时修改，最后返回 0 就出错。非空数组应从真实元素初始化；空数组另行定义失败结果。

# 线性表 2

**知识导图**




**最小反例·容量不等于表长**

容量为 100 的数组可以表示空表，此时 length=0；也可只保存一个元素。仅给数组上界不能推断有效元素数。插入判断必须同时检查位序是否合法与是否尚有空位。

**知识导图（续）**




**错项排除·随机存取不等于随机查值**

顺序表给定位序后可用地址公式直接访问；只给目标值时，尚不知道它在哪个位置。无序表按值查找仍需逐个比较，不能把随机访问的 O(1) 套到所有查找上。

同一个线性表可以放在连续数组中，也可以由分散的结点连接起来。前者依靠数组下标定位，插入和删除常要搬移元素；后者把先后关系放在链接中，定位常要逐结点进行。分析操作复杂度时，必须把“找到位置”和“修改结构”分别计算。

**错项排除·不搬移不等于常数时间**

链表删除可只改链接，不需要移动其他元素的数据；但若还要从头定位前驱，操作总时间依然可能为线性。做题先拆分定位成本和局部结构修改成本。

## 2.1 位序、下标和基本操作

线性表是具有相同数据类型的 $n$ 个元素的有限序列。$n=0$ 时为空表。非空表的第一个元素没有直接前驱，最后一个没有直接后继，其余元素各有一个直接前驱和一个直接后继。这里的“相同类型”不要求元素的值互不相同。

长度为 $n$ 的表，已有元素的位序是 $1\sim n$；在第 $i$ 个位置插入新元素，合法范围是 $1\le i\le n+1$。最后一个位置表示追加。若用 `A[0..n-1]` 保存，位序 $i$ 对应下标 $i-1$。查找失败的信号不能与下标 0 混淆，例如按位序返回 0 表示失败，或按下标返回 $-1$ 表示失败。

本章约定单链表有头结点 `L`，首元结点为 `L->next`。头结点用于统一第一个位置的插入与删除，不保存有效表元素；**头指针**是指向它的指针变量，二者不是同一对象。无头结点的版本也可以实现相同运算，但修改第一个结点时必须能够修改头指针。

## 2.2 顺序表的寻址、插入和删除

若每个元素占 $w$ 个地址单位，首元素地址为 $b$，则第 $i$ 个元素地址为 $b+(i-1)w$。计算地址只需常数次算术运算，故可随机存取。“随机”指任意位置可以直接定位，与随机数无关。按值查找仍要比较元素，未排序时通常需要 $O(n)$ 时间。

在下标 $k$ 处插入，要把 `A[k..n-1]` 各后移一位；必须**从后向前**移动，否则后一个元素可能已被前一个覆盖。删除下标 $k$ 的元素，则把 `A[k+1..n-1]` 各前移一位，按从前向后的顺序执行。

```
insert(A, n, capacity, k, x):
    if k<0 or k>n or n==capacity: return false
    for j=n down to k+1: A[j]=A[j-1]
    A[k]=x
    n=n+1
    return true
```

插入移动 $n-k$ 个元素，追加时为 0；删除移动 $n-k-1$ 个元素，删除末元素时为 0。等概率在 $n+1$ 个插入位置选择时，平均移动 $n/2$ 个；等概率删除 $n$ 个已有元素时，平均移动 $(n-1)/2$ 个。平均数的分母不同，来自合法位置数不同。

### 2.2.1 例 2-1　没有给存储方式时能否确定移动数

**例 2.2.1** (配套例题)

〖2.3-012〗在线性表 $a_0,a_1,\ldots,a_{100}$ 中，删除元素 $a_{50}$ 需要移动（　）个元素。

A. 0　　B. 50　　C. 51　　D. 0 或 50

**解析**。题目没有限定存储方式。顺序表中，后面的 $a_{51}\sim a_{100}$ 共 50

个元素前移；链表中改变链接即可，无需搬移其他结点的数据，故答案为 **D**。删除前若还要从头寻找该结点，链表访问时间仍可能是 $O(n)$；“不移动数据” 不等于 “全过程常数时间”。

静态数组的容量固定，不意味着表长固定。动态数组可以在空间不足时申请更大的连续区并搬移旧元素，但单次扩容可能需要 $O(n)$ 时间；分析某一次插入时不能忽略这次复制。

**不变量·搬移保护未读区**

插入向后搬移时，从后往前处理能确保源元素尚未被覆盖；删除向前搬移则从前往后读。记住需要保护的未读区，就能由目标下标推回正确循环方向。

## 2.3 顺序表的完整状态与核心实现

**定义 2.3.1** (表长、容量和初始化)

顺序表的状态不只是数组，还包括当前有效长度。容量是最多容纳的元素数，表长是已经存放的元素数，满足 $0 \le length \le capacity$。初始化令表长为 0；清空只撤销有效元素，销毁还要按存储方式释放资源。静态数组离开作用域由语言回收，动态申请区则按分配方式释放。

下例统一使用 C++17、位序从 1 开始，内部数组从 0 开始。L 以引用传入，是为了修改调用者的表长和元素；删除的输出参数仅在成功时被赋值。

```
constexpr int CAP = 100;
struct SeqList { int data[CAP]; int length = 0; };
bool insertAt(SeqList& L, int pos, int x) {
    if (pos < 1 || pos > L.length + 1 || L.length == CAP)
        return false;
    const int k = pos - 1;
    for (int j = L.length; j > k; --j)
        L.data[j] = L.data[j - 1];
    L.data[k] = x;
    ++L.length;
    return true;
}
```

```
bool eraseAt(SeqList& L, int pos, int& removed) {
    if (pos < 1 || pos > L.length) return false;
    const int k = pos - 1;
    removed = L.data[k];
    for (int j = k; j + 1 < L.length; ++j)
        L.data[j] = L.data[j + 1];
    --L.length;
    return true;
}
```

**为什么移动方向不能互换**

插入时，待保留的旧元素位于 $A[k..n-1]$。从后向前移，在写入 $A[j]$ 时，旧 $A[j]$ 已经被保存到更后的位置；因此不会丢失仍需搬移的元素。若从前向后做 $A[j+1]=A[j]$，旧 $A[j+1]$ 会在下一轮读取之前被覆盖。删除正好相反，目标地址比源地址小，故从前向后移动能保护未读的旧后缀。

□

## 2.4 单链表：先保留哪条链接

结点由数据域与后继指针域构成。不同结点不要求物理相邻；同一个结构体结点所含的各域作为一个整体分配，因此不能把“结点之间地址分散”误说成“一个结点内部各域随意分散”。为了读取第 $i$ 个元素，须从链头沿 `next` 走 $i-1$ 步，所以链表不支持按位序的常数时间随机访问。

已知有效结点 `p`，在其后插入新结点 `s`：先令 `s->next=p->next` 保存旧后继，再令 `p->next=s`。若先把 `p->next` 改成 `s`，随后从 `p->next` 取得的已不是旧后继，可能产生自环并丢失后半链。

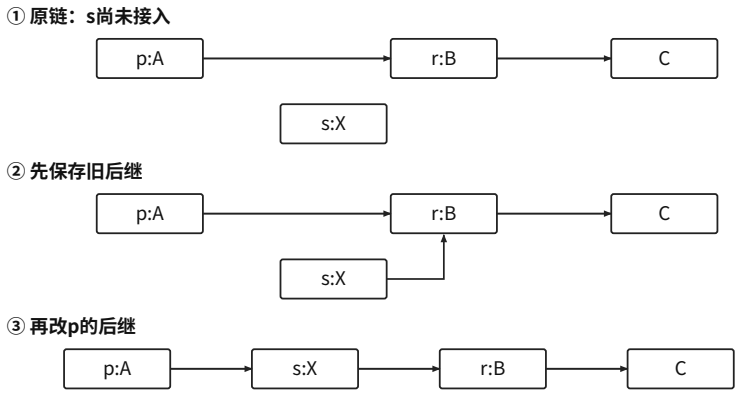


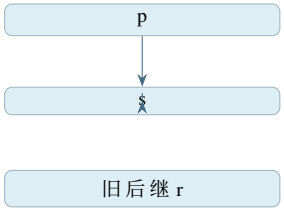

Figure 2.1: 插入前、保存旧后继、接入新结点；虚线表示待改链接

删除 `p` 的后继时，先检查 `p->next!=NULL`，再用 `q=p->next` 暂存待删结点，执行 `p->next=q->next`，最后释放 `q`。释放后不能继续读取 `q->next`。若只给待删结点的指针而不给其前驱，通常还要从头寻找前驱；用“复制后继数据再删后继”的办法不适用于尾结点，还可能改变题目要求保留的结点身份。

头插法不断把新结点接到头结点之后，输入次序会逆转；尾插法保存尾指针，每次接到尾部，能保留输入次序。两者若都维护所需指针，建立 $n$ 个结点的链表均为 $O(n)$；若每插入一个元素都重新从头找尾部，才会累积到 $O(n^2)$。

**最小反例·两句交换会丢链**

p 原后继为 r。若先写 p->next=s，再写 s->next=p->next，第二句读到的是 s，形成自环，r 不再可从 p 到达。先暂存旧链接，是所有原地指针改写的基本纪律。




错误顺序使 s 自环，旧后继 r 不可达

### 2.4.1 原地逆置：未处理部分不能丢

维护 `prev` 为已逆置部分的新首结点，`cur` 为未处理部分的首结点。每次先暂存 `nxt=cur->next`，再把 `cur->next` 指向 `prev`，随后推进两个工作指针。

```
void reverseList(Node *L) {       // 带头结点
    Node *prev=NULL, *cur=L->next;
    while (cur!=NULL) {
        Node *nxt=cur->next;
        cur->next=prev;
        prev=cur;
        cur=nxt;
    }
    L->next=prev;
}
```

每轮只移动一个结点到逆置区。空表时循环不执行，头结点仍指向

空；单结点时其 next 仍为空。时间 $\Theta(n)$，辅助空间 $\Theta(1)$。递归逆置虽然也能线性运行，却要保留递归栈，不能与迭代实现的空间混算。

### 2.4.2 例 2-2　如何一次扫描找到倒数第 $k$ 个结点

**例 2.4.1** (配套例题)

配套例题：已知一个带有表头结点的单链表，结点结构为 data | next，只给出了头指针 list。在不改变链表的前提下，设计尽可能高效的算法，查找倒数第 $k$ 个位置上的结点（$k$ 为正整数）。若成功，输出该结点的 data 值并返回 1；否则只返回 0。要求说明设计思想、实现步骤，并用 C、C++ 或 Java 描述算法，在关键处加注释。

**解析**。倒数位置由尾部决定，但链表不能直接向前访问。让两个指针保持固定间隔：fast 先从首元结点前进 $k$ 步，再让 fast 与 slow 同步前进。若表长为 $n$，fast 到达空位置时共走了 $n$ 步，slow 只走了 $n-k$ 步，正好指向第 $n-k+1$ 个结点。

**不变量·固定间隔保持到尾**

fast 先走 k 步后，两指针始终相隔 k 条后继步。当 fast 到空位置时，slow 距空位置也恰为 k 步，故指向倒数第 k 个结点。先走阶段遇空可直接识别 k 超过表长。

```c
int searchK(Node *list, int k) {
    if (k<=0) return 0;
    Node *fast=list->next, *slow=list->next;
    for (int j=0; j<k; ++j) {
        if (fast==NULL) return 0;   // 表长不足 k
        fast=fast->next;
    }
    while (fast!=NULL) {
        fast=fast->next;
        slow=slow->next;
    }
    printf("%d", slow->data);
    return 1;
}
```

以长度 5、$k=2$ 为例，先让 fast 指向第 3 个结点，slow 留在第 1 个；同步经过“4 与 2—5 与 3—空与 4”，第 4 个即倒数第 2 个。$k=n$ 时第一阶段恰好走到空位置，第二阶段不执行，返回首元结点；$k>n$ 时在第一阶段报失败。两个指针合计走 $O(n)$ 步，辅助空间 $O(1)$，没有修改任何链接。

## 2.5 双链表、循环链表与静态链表

双链表为每个结点增加 prior，以便从当前结点访问直接前驱。已知 p 后，删除 p 可在 $O(1)$ 时间内同时接好两侧；但这不消除按位序寻找 p 的成本。

### 2.5.1 例 2-3　把结点插到双链表结点之前

**例 2.5.1** (配套例题)

〖2.3-015〗在双链表中向 p 所指的结点之前插入结点 q，正确操作是（　）。

```c
A. p->prior=q; q->next=p;
   p->prior->next=q; q->prior=p->prior;
```

```
B. q->prior=p->prior; p->prior->next=q;
   q->next=p; p->prior=q->next;
C. q->next=p; p->next=q;
   q->prior->next=q; q->next=p;
D. p->prior->next=q; q->next=p;
   q->prior=p->prior; p->prior=q;
```

最小反 例·旧前驱必须还能取

只rp有、两个相邻结点时，p在前q插的最终关系r是qp↭。若第一句覆p->prior盖，之后再从它读取就失r去；四条目标链接全部成立才算插入完

**解析**。假设 p 有有效前驱（可以是头结点）。记旧前驱为 r，结果必须同时满足 r->next=q、q->prior=r、q->next=p、p->prior=q。D 在最后才修改 p->prior，因此前三句还可以取得旧前驱，答案为 **D**。A 的第一句过早覆盖旧前驱；B 最后把 p->prior 设成了 p 自身；C 既改错方向，又使用尚未设置的 q->prior。

写程序时也可先 q->prior=p->prior; q->next=p;，再改两侧结点的链接。若 p 是不带头结点链表的首元结点，前驱为空，须单独更新头指针，不能直接解引用这个空前驱。

**循环单链表**把尾结点的 next 指回链头。带头结点时，空表满足 L->next==L；L==NULL 只说明没有有效头结点，并非这个模型的空表条件。遍历应在重新遇到头结点时结束，而不是等待 NULL。

只给尾指针 rear 的无头循环单链表，可以用 rear->next 取得首元结点；两个非空循环单链表头尾相接时，交换各尾部的后继链接即可完成 $O(1)$ 拼接。若只给头指针且没有尾链接，要先找到尾部，通常需要线性时间。若存在头结点，拼接时须处理多余头结点，不能把它当作数据留在两表之间。

循环双链表的头结点同时连接首尾，空表时 next 与 prior 都指回头结点。**静态链表**用数组保存结点，用整数下标代替机器指针；逻辑后继由游标决定，不要求结点下标相邻。还须维护空闲结点链或其他管理空闲位置的结构，数组开得大不等于每个位置都是有效数据。

## 2.6 双指针、归并与原地处理

这些方法把题目的约束变成指针之间的不变关系。使用前先确认能否修改原数据、是否有序、是否共享结点，以及是否需要保留重复元素。

### 2.6.1 例 2-4　数组循环左移怎样避免反复搬移

**例 2.6.1** (配套例题)

配套例题：将 $n$ $(n>1)$ 个整数存放在一维数组 $R$ 中。设计一个时间、空间都尽可能高效的算法，将序列循环左移 $p$ $(0<p<n)$ 个位置，即把 $(X_0,\dots,X_{n-1})$ 变为 $(X_p,\dots,X_{n-1},X_0,\dots,X_{p-1})$。给出思想、C/C++ 或 Java 实现，并说明时间与空间复杂度。

**解析**。题目要求两个连续区段交换位置，同时保持各段内部次序。设前 $p$ 个元素为 $A$，其余为 $B$。分别逆置得 $A^RB^R$，再整体逆置，即得 $BA$。两个区段各被逆置两次，内部方向恢复，而相对位置交换。

```
void reverse(int R[], int l, int r) {
    while (l<r) {
        int t=R[l]; R[l]=R[r]; R[r]=t;
        ++l; --r;
    }
}
```

```
void rotateLeft(int R[], int n, int p) {
    reverse(R,0,p-1);    reverse(R,p,n-1);
    reverse(R,0,n-1);
}
```

例如 abcdefgh 左移 3 位，依次得到 cbadefgh、cbahgfe…defghabc。总交换次数不超过 $n$，时间 $O(n)$；只用常数临时变量，空间 $O(1)$。若将首元素移到末尾重复 $p$ 次，时间为 $O(np)$，当 $p$ 与 $n$ 同阶时会退化。三次逆置可处理 $p=1$ 和 $p=n-1$，无需另写搬移循环。

错项排 除·字符相同不等于共

两个结点都存字i符，也可能位于不同地址。公共后缀要求从某处起共享同一串结点，应比较指针而不data是；共享链不能再按两条独占链分别释

### 2.6.2 有序归并为什么可以一次扫描

两个递增表的当前首元素中，较小者一定是剩余全部元素中最小的。输出它并推进对应指针，重复直到某表为空，再接上另一个表的剩余部分。合并长度为 $m,n$ 的两个表需要 $O(m+n)$ 时间；数组版本通常另开输出区，链表版本在允许破坏原表时可直接重接结点。

比较相等元素时，若希望保持原相对次序，先取左表元素。若题目要求集合并集而不是多重集合合并，则相等时只输出一次并推进两个指针；这项条件会改变算法，不能只改题目的答案。

### 2.6.3 例 2-5　相同后缀是相同字符还是相同结点

**例 2.6.2** (配套例题)

配套例题：采用带头结点的单链表保存单词，两个单词可共享相同后缀的存储空间。例如 loading 与 being 共享从字符 i 开始的结点。str1、str2 指向两个头结点，结点为 data | next。设计时间上尽可能高效的算法，找到共同后缀的起始结点，给出思想、程序及时间复杂度。

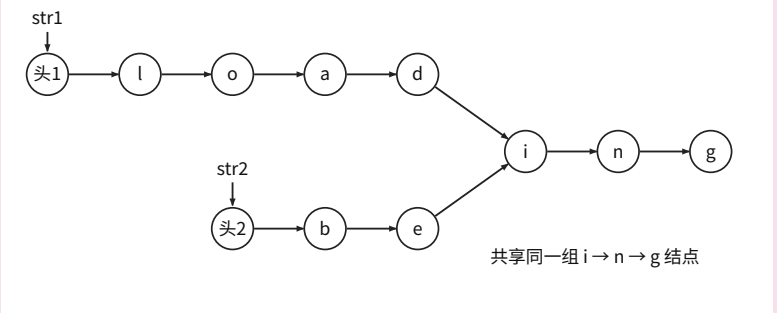



Figure 2.2: 两个前缀分别连接到同一个 i 结点，随后共用 n、g 结点

**解析**。共享指的是地址相同，不能以 p->data==q->data 判断。单链表一旦在某结点汇合，其后继也相同，所以两表从公共起点到末尾的长度相等。先各扫描一次求长 $m,n$，让较长表先走 $|m-n|$ 步，两指针便处于距表尾相同的位置；此后同步前进，第一次地址相等的位置就是答案。

```
Node *commonSuffix(Node *str1, Node *str2) {
    int m=0,n=0;
    for (Node *p=str1->next; p; p=p->next) ++m;
    for (Node *q=str2->next; q; q=q->next) ++n;
    Node *p=str1->next, *q=str2->next;
    while (m>n) { p=p->next; --m; }
    while (n>m) { q=q->next; --n; }
```

```
    while (p!=q) { p=p->next; q=q->next; }
    return p;                    // 无公共结点时二者同时为 NULL}
```

不变 量 · 头结点不随表空而消

有效头结点始终存在，数据结点 L->next从开始。空表并不意味L着为空；统一前驱定位pos=0时对应头结点，才能让第一个位置的插入与其他位置共用同样两步链

图中两表长度分别为 7 和 5，先让长表跨过 l,o，再比较 a 与 b、… 与 e，最后到达同一个 i。若两表内容相同却各自分配结点，则直到… 位置才相等，返回空。总时间 $O(m+n)$，辅助空间 $O(1)$，适用于有… 无环单链表。

### 2.6.4 例 2-6 主元素为什么还要再计数

**例 2.6.3** (配套例题)

配套例题：整数序列 $A=(a_0,\ldots,a_{n-1})$ 满足 $0\le a_i<n$。某值出现次数超过 $n/2$ 时，称它为主元素。例如 $(0,5,5,3,5,7,5,5)$ 的主元素为 5，而 $(0,5,5,3,5,1,5,7)$ 没有主元素。设计尽可能高效的算法，存在主元素时输出该值，否则输出 $-1$；给出思想、程序及时间、空间复杂度。

**解析**。每次删除两个值不同的元素，不会消灭一个原本占严格多数的值：若删去一个多数元素和一个其他元素，多数仍超过剩余的一半；若删去两个其他元素，多数优势更大。用一个候选值和计数模拟这种抵消。计数为 0 时选新候选；相同则加 1，不同则减 1。

```
int majority(int A[], int n) {
    int candidate=0, count=0;
    for (int i=0; i<n; ++i) {
        if (count==0) { candidate=A[i]; count=1; }
        else if (A[i]==candidate) ++count;
        else --count;
    }
    int actual=0;
    for (int i=0; i<n; ++i)
        if (A[i]==candidate) ++actual;
    return actual>n/2 ? candidate : -1;
}
```

抵消只保证“若有主元素，最后候选一定是它”，没有保证主元素存在。例如 1,2,3 最后仍可能留下候选 3，它只出现一次，所以必须第二次扫描核实。两个线性扫描合计 $O(n)$ 时间，额外空间 $O(1)$。题目给出的值域也允许用计数数组，但需 $O(n)$ 辅助空间；此处没有必要为省下一次扫描而付出这部分空间。

## 2.7 单链表的接口契约与基本操作

**先定位前驱，再改链接**

把按位序插入/删除拆成两步：先从有效头结点开始走到第 $pos-1$ 个结点，再执行局部接入或摘除。前驱定位可能是线性时间；局部链接改写是常数时间。以下代码要求链表有限、无环、结点独占所有权，头结点始终有效；共享后缀仅做只读查找，不能分别销毁两条共享链。

```
struct Node { int data; Node* next; };
// 返回第pos个结点；pos=0返回有效头结点。
```

```
Node* nodeAt(Node* L, int pos) {
    if (pos < 0) return nullptr;    Node* p = L;
    while (p != nullptr && pos > 0) {
        p = p->next;
        --pos;
    }
    return p;
}
```

**课程连接·表示影响访问局部性**

连续数组通常能利用缓存中的相邻数据，分散结点可能增加缓存未命中。渐近复杂度相同也不保证实际用时相同；比较表示时同时检查数据规模、访问方式和存储开销。

```
bool insertNode(Node* L, int pos, int x) {
    if (pos < 1) return false;
    Node* p = nodeAt(L, pos - 1);
    if (p == nullptr) return false;
    Node* s = new Node{x, p->next};
    p->next = s;
    return true;
}
bool eraseNode(Node* L, int pos, int& removed) {
    if (pos < 1) return false;
    Node* p = nodeAt(L, pos - 1);
    if (p == nullptr || p->next == nullptr) return false;
    Node* q = p->next;
    removed = q->data;
    p->next = q->next;
    delete q;
    return true;
}
```

new 申请失败时，标准 C++ 抛出异常；此时接入语句尚未执行，旧链不变。上面的布尔失败只表示非法位序等接口条件失败，并没有把内存分配异常改成 false。销毁独占链时逐个保存后继再 `delete`，最后才释放头结点；不能混用 `malloc/free` 与 `new/delete`。

初始化可用 `Node* L = new Node{0, nullptr};`，其中头结点的 data 不计入表。按值查找从 `L->next` 开始，比较值并沿 next 前进；查找不存在时返回 nullptr。求表长必须实际数数据结点，而不是将头结点计入。

## 2.8 结构选择与操作代价对照

| 操作条件 | 顺序表 | 单链表（有效头结点） | 选择时还需检查 |
|---|---|---|---|
| 按位序取值 | $O(1)$ | $O(n)$ | 是否要求随机访问 |
| 无序按值查找 | $O(n)$ | $O(n)$ | 值是否有序、是否另有索引 |
| 按位序插入/删除 | 最坏 $O(n)$ | 定位最坏 $O(n)$ | 是否已给有效前驱 |
| 已知前驱后的局部修改 | 仍可能搬移后缀 | $O(1)$ | 申请/释放与结点身份 |
| 存储开销 | 连续容量可能有空闲 | 每结点含链接域 | 局部性、碎片和最大规模 |

顺序表逻辑相邻通常也物理相邻，有利于连续访问；链表用额外链接换取接入灵活性，不能由此得出“链表一定更省空间”。静态链表虽然使用数组分配结点，逻辑后继仍由游标指定，所以不具备按逻辑位序直接寻址的能力。

## 2.9 连续删除与有序去重的线性写法

**删除后留在前驱，保留后才前进**

删除单链表所有值为 x 的结点时，用 p 指向已处理部分的最后一个保留结点。若 p 后继应删除，就摘除后继续检查 p 的新后继；否则 p 向前走。相邻多个 x 正是不能在每次删除后都移动 p 的原因。

**不变量·删后检查新后继**

若当前前驱后连续两个结点都等于 x，删掉第一个后马上前进，就可能跳过第二个。前驱只在保留结点时前进，才能让每个新后继都得到检查。

```
void eraseAll(Node* L, int x) {
    Node* p = L;
    while (p->next != nullptr) {
        if (p->next->data == x) {
            Node* q = p->next;
            p->next = q->next;
            delete q;
        } else {
            p = p->next;
        }
    }
}
```

顺序表有序去重可让读指针从左到右扫描，写指针只扩展“已保留的互异前缀”。若当前值不同于最后一个保留值才写入；每个元素只处理常数次，总时间 $O(n)$，额外空间 $O(1)$。未排序数组中，相邻不重复不能保证全局不重复，不能悄悄去掉有序条件。

## 2.10 主动回忆自测

以下为原创补充，检查边界和不变式。

1. 允许修改单链表，要求删除所有值为 $x$ 的结点。删除一个结点后，指向前驱的指针是否应立即后移?
2. 两个无环单链表长度相同，尾结点地址相同，能否断定它们一定有公共后缀?
3. 顺序表去除相邻重复元素时，为什么可用“已保留长度”作为写指针，而不必每删一个元素都搬移后半表?

**答案。** 1. 不应立即后移；新的后继还可能等于 $x$，应继续检查当前前驱的后继。只有保留一个结点时前驱才推进。2. 能；至少尾结点本身就是公共后缀，公共起点还可能更早。3. 扫描指针只读未处理区，写指针只向前形成已保留区；每个元素最多被检查和写入常数次，总时间 $O(n)$。若原表无序，这只能消除相邻重复，不能保证全表唯一。

### 2.10.1 补充自拟回忆题

**从边界恢复算法**

4. 长度 n 的顺序表允许在哪些位序插入，在哪些位序删除？各自等概率模型的分母是什么?
5. 有效头结点、头指针、首元结点如何区分？带头结点空表中哪一个为空?
6. 只给单链表尾结点指针，能否总在 O(1) 时间内删除该结点并保留其他结点身份?

7. 主元素候选的计数归零意味着此前完全没有主元素吗？为什么最后还要核计数？
8. 两个内容相同却分别分配的单链表，公共后缀算法返回什么？适用前提是什么？

**补充题核对答案**

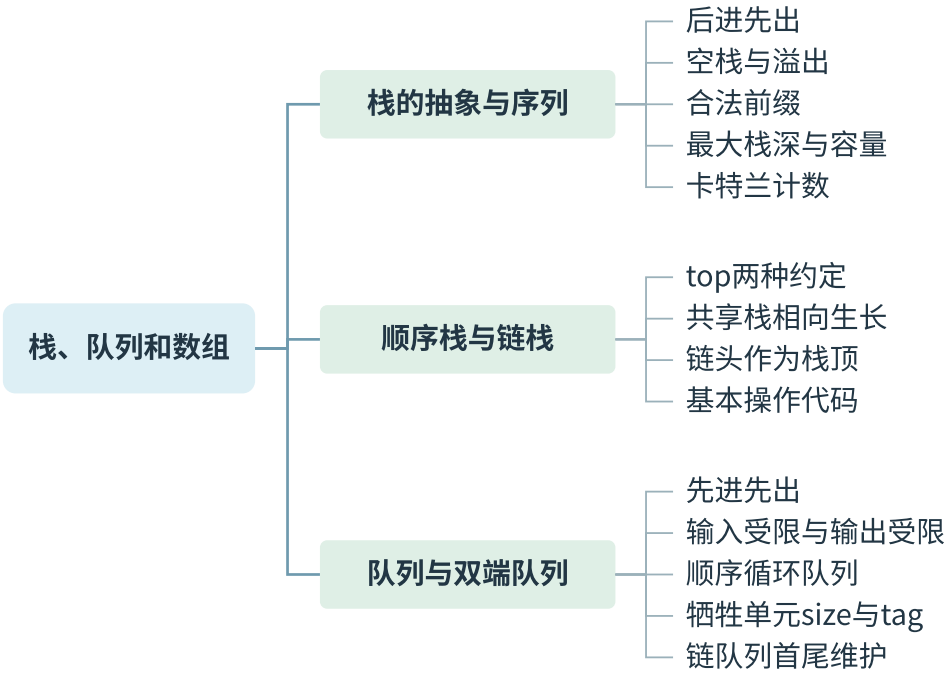
4. 插入位序 1 到 n+1，删除 1 到 n；平均插入移动数除以 n+1，平均删除移动数除以 n。
5. 头指针是地址变量，头结点是哨兵对象，首元结点是第一个有效元素。有效头结点存在时，空表是 L->next 为空。
6. 不能。单链表尾结点无后继可复制，且没有直接前驱链接，通常要从头找前驱。
7. 不意味着不存在严格多数；抵消保证 “若存在，最后候选是它”，不能反推候选一定占多数，必须再次扫描。
8. 返回 nullptr。地址相等才表示共享，前提是有限无环单链表且头结点和链接有效。

**最小反例·剩下候选仍需复核**

序列 1、2、3 经抵消可留下 3，但 3 只出现一次，未超过一半。抵消给出的是存在多数时的必要候选，并不证明多数存在；第二次计数补上充分性检验。

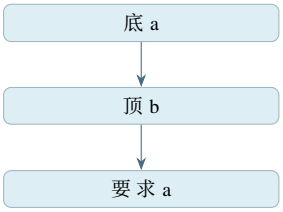
# 3 栈、队列和数组

**知识导图**



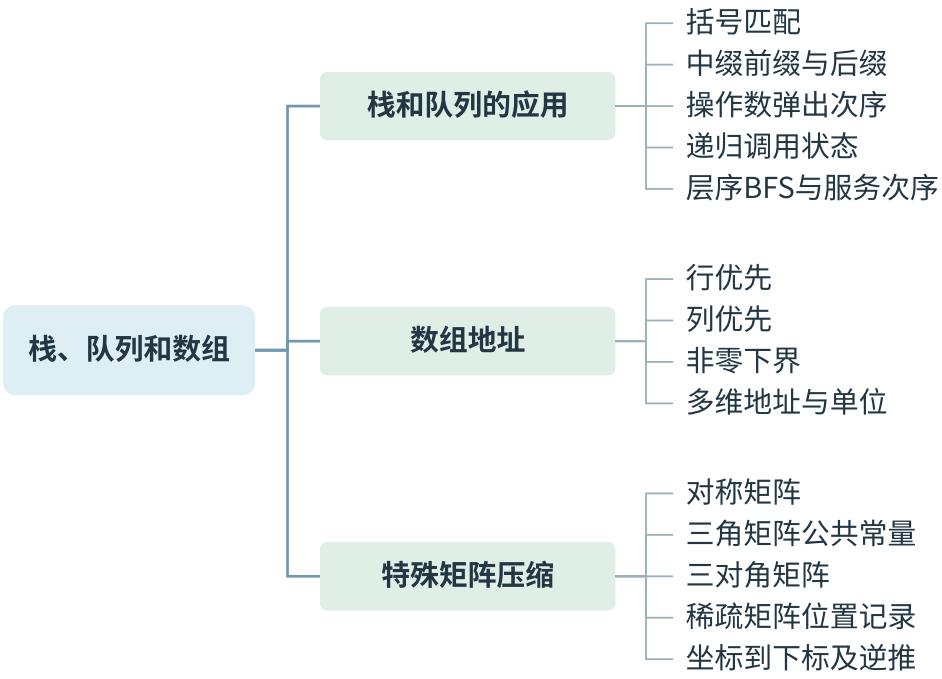

**最小反例·数量相同仍不合法**

顺序输入 a、b、c，要先输出 c 就必须把三者全部入栈；弹出 c 后栈顶为 b，接着要求 a 便失败。输出 c、a、b 与输入含相同元素，却违反栈顶约束。




c 已弹出，b 挡在 a 上方

**知识导图（续）**




**错项排除·共享栈只在相邻时满**

两栈从数组两端向中间增长，未用单元恰位于两栈顶之间。两栈顶相邻已经没有空位；等到相等才判满，就已让两个栈占用同一个位置。

栈与队列都保存线性序列，差别在于允许从哪一端加入和取出。判断一个操作序列是否合法，必须保留操作过程中的状态；只检查输入、输出是否含有相同元素，不能排除违反栈顶或队头约束的情况。

**错项排除·双端队列要看受限端**

输入受限和输出受限双端队列是两种不同约束。普通栈的合法输出在一般双端队列中能实现，但不能倒推出任意指定输出都能实现；仍要跟踪两端状态。

## 3.1 栈的后进先出与合法序列

**栈**只允许在同一端插入和删除，这一端称为栈顶，另一端称为栈底。入栈称 `push`，出栈称 `pop`。栈为空时不能出栈；固定容量栈满时不能继续入栈。若中途不出栈，一批元素的出栈次序是入栈次序的逆序；若允许交替入栈和出栈，输出就可能有多种。

检验给定输出的方法是按目标逐个模拟：若当前栈顶不是所需元素，继续把尚未入栈的元素依次压入；一旦所需元素到顶，就弹出。若输入已用尽而栈顶仍不符，输出不合法。这个过程也给出实现该输出所需的最小容量，即模拟中出现的最大栈深。

### 3.1.1 例 3-1　队列不会改变栈的输出次序

**例 3.1.1** (配套例题)

配套例题：栈 $S$ 和队列 $Q$ 初始均为空，元素 `a,b,c,d,e,f,g` 依次入栈 $S$。每个元素出栈后立即进入队列 $Q$，7 个元素出队顺序为 `b,d,c,f,e,a,g`，则栈容量至少是（　）。

A. 1　B. 2　C. 3　D. 4

**解析**。队列保持进入它的先后次序，因此它的输出就是栈的输出。按目标模拟，栈内状态按栈底到栈顶写：

| 需要输出 | 必须先执行的入栈 | 弹出后栈内剩余 |
|---|---|---|
| b | a、b | a |
| d | c、d | a、c |
| c | 无 | a |
| f | e、f | a、e |
| e | 无 | a |
| a | 无 | 空 |
| g | g | 空 |

压入 d 或 f 后，栈深达到 3；表中过程也说明 3 个单元足够，答案为 **C**。不能把队列当作另一处可以随意调整次序的缓冲区。

若只用入栈 `I`、出栈 `O` 表示操作，一段从空栈出发并回到空栈的合法序列须满足：两类操作的总数相等，且每个前缀中 `O` 的个数不超过 `I`。对 $n$ 个不同元素，合法出栈序列数为卡特兰数 $C_n = \frac{1}{n+1}\binom{2n}{n}$；这一公式用于计数，不代替具体容量与给定序列的模拟。

输入为 `a,b,c,d` 时，若输出以 `c,d` 开头，弹出 c 后栈中为 a、b，随后必须压入并弹出 d，剩余只能 b、a，因此仅有 `c,d,b,a` 一种。若题目指定的是栈的输出而让输入未知，也要反推中间栈顶。例如第 3 个输入是 1，输出从 1 递增，1 弹出时第 2 个输入压在第 1 个输入上方；若第 1 个是 2，就要先弹出上面的另一个值才能到 2，与“接下来输出 2”矛盾，所以第 1 个不可能是 2。

## 3.2 队列与双端队列

**不变量·尾指针不能悬空**

带头结点链队列始终让 rear 指向最后一个数据结点，空时指向头结点。删去唯一数据结点后，rear 必须回到 front；否则下次入队会经已释放地址写内存。

**队列**只在队尾加入、在队头删除，遵循先进先出。它适合保存等待依次处理的对象，如按发现顺序等待扩展的图顶点。队头与队尾是逻辑角色，不能凭数组下标的大小判断先后：循环队列绕回后，队尾下标可以小于队头下标。

**双端队列**允许在两端插入和删除。输入受限双端队列只允许一端输入，两端均可输出；输出受限双端队列允许两端输入，只允许一端输出。判断输出时应明确属于哪一种，不能直接套普通队列或普通栈的限制。双端队列能够模拟栈与队列，但这不表示它能按任意次序输出一组顺序输入的元素。

## 3.3 顺序栈、共享栈与循环队列

顺序栈存于 `A[0..M-1]`。本书令 `top` 指向当前栈顶，初值 $-1$：空条件 `top==-1`，满条件 `top==M-1`；入栈为 `A[++top]=x`，出栈为 `x=A[top--]`。若题目改用 “`top` 指向下一个空位”，空条件就为 0，入栈与出栈的增减位置也随之改变。先画一个元素的状态，再写公式，通常比背四个表达式可靠。

两个栈共享一个数组时，可让一端从左向右长、另一端从右向左长。初值 `top1=-1, top2=M`，栈满当且仅当 `top2-top1==1`。未用空间是两栈顶之间的单元，某个栈已用多少并不单独决定全局是否栈满。入栈前必须先检查两栈顶是否相邻，不能等两个栈顶已经重合才报告满。

### 3.3.1 循环队列先立约定再计算

采用长度为 $M$ 的数组，令 `front` 指向队头元素，`rear` 指向下一次入队位置。出队后 `front=(front+1)%M`，入队后 `rear=(rear+1)%M`。若只有这两个下标，空与完全占满都会出现 `front==rear`，无法区分。

**牺牲一个单元**时，约定空条件为 `front==rear`，满条件为 `(rear+1)%M==front`，可保存的最大元素数是 $M-1$。当前长度为

$$\text{len} = (rear - front + M) \bmod M.$$

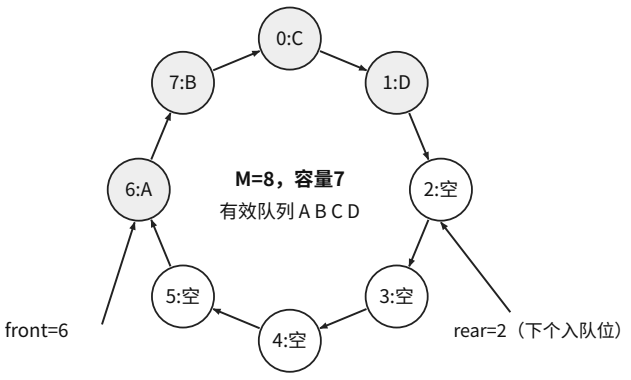



Figure 3.1: 数组环绕时，下标大小不代表队列先后；本图采用牺牲一个单元

若要求 $M$ 个单元全部可用，可以增加 `size` 计数，或增加标志，区分最近一次使二者相等的是入队还是出队。此时 “空” 和 “满” 的判断依赖新增成员，不能仍照搬牺牲单元的满条件。

### 3.3.2 例 3-2　换了 front 含义，长度公式是否改变

**例 3.3.1** (配套例题)

〖3.2-006〗循环队列存于数组 `A[21]`，`front` 指向队首元素的前一个位置，`rear` 指向队尾元素。当前 `front=8, rear=3`，队列长度为（　）。

A. 5　B. 6　C. 16　D. 17

**解析**。有效元素从下标 9 开始，经过 9 至 20，再经过 0 至 3，共 $12+4=16$ 个，答案为 **C**。公式仍可写成 $(3-8+21) \bmod 21 = 16$，是因为与前一约定相比，两个指针的含义都错开了一个位置。若只改变一个指针的含义，公式就未必不变。数组是 `A[0..n]` 时实际有 $n+1$ 个单元，取模也必须使用 $n+1$。

## 3.4 链栈与链队列

链栈通常把链头作为栈顶，头插与头删都是 $O(1)$。动态分配可随数据量申请结点，因而通常不受预先给定的小容量数组限制；但入栈仍可能因可用内存不足而失败，出栈时也仍可能遇到空栈。

单链队列把链头作为队头，链尾作为队尾，并保存两个指针。若反过来从单链表尾部删除，必须找到尾结点的前驱，出队会变为 $O(n)$。带头结点的链队列空时 `front==rear`，二者都指向头结点。入队时接到 `rear->next` 并更新 `rear`；出队时删除 `front->next`。

```
dequeue(Q):
    if Q.front==Q.rear: return 失败
    p=Q.front->next
    x=p->data
    Q.front->next=p->next
    if Q.rear==p: Q.rear=Q.front
    释放 p
    return x
```

最后一个数据结点出队时，必须让尾指针退回头结点；否则它指向已释放的空间。非空且不止一个元素时，无需改变尾指针，所以对 “删除是否修改头尾指针” 这类问题，答案取决于被删除的是否为最后一个元素。

只带队首指针的非循环双链表，要在另一端入队仍需找尾部；循环双链表则可通过 `front->prior` 直接取得尾部。比较结构时，应检查有没有可到达另一端的短链接，而不是看到 “双链” 就认定所有操作都为常数时间。

## 3.5 多维数组的地址怎样计算

数组同一维度的下标取值范围固定，每个元素大小相同。设二维数组的行下标为 $l_1 \sim u_1$，列下标为 $l_2 \sim u_2$，列数 $c = u_2 - l_2 + 1$，行数 $r = u_1 - l_1 + 1$，每元素占 $w$ 字节，首元素地址为 $b$。

**课程连接·偏移先统一地址单位**

数组公式先数元素，再乘每元素宽度。计组中的字节编址与按字编址会改变地址单位；同样的元素偏移，在两种编址方式下不能直接共用一个数值。

按行优先时，在 $(i,j)$ 前有 $i-l_1$ 个完整行及本行的 $j-l_2$ 个元素，故

$$\text{Addr}(A[i,j]) = b + [(i-l_1)c + (j-l_2)]w.$$

按列优先时，先数完整列：

$$\text{Addr}(A[i,j]) = b + [(j-l_2)r + (i-l_1)]w.$$

例如 `A[2..4][-1..2]` 按行优先存放，每元素 4 字节，`A[2][-1]` 的地址为 1000。`A[4][1]` 前有 $2\times4+2=10$ 个元素，字节地址为 1040。下标差先得到“元素数”，最后乘元素宽度才得到字节偏移；按字编址的题则要统一成字地址再计算。

三维数组按行优先时，最右下标变化最快：`A[i][j][k]` 的相对元素位置为 $((i-l_1)n_2+(j-l_2))n_3+(k-l_3)$。可以逐维理解，不必另背一个没有下标原点的公式。

**错项排除·三对角首尾行少一个**

n 至少为 2 时，首行和末行各只保存 2 个位置，中间行才各 3 个。若把第 i 行之前全部按 3 个计，会从第一行就多算；先列 2+3(i−2) 再加本行偏移。

## 3.6 对称、三角、三对角与稀疏矩阵

压缩存储利用元素间的确定关系，避免重复保存。**对称矩阵**满足 $a_{ij}=a_{ji}$，保存一侧三角即可；访问另一侧时交换 $i,j$。**三角矩阵**的三角区之外，元素全部相同，通常还要额外保存这个公共常量，它未必为 0。**三对角矩阵**只在主对角线及相邻两条对角线上可能非零，共有 $3n-2$ 个需要保存的位置。

下三角按行保存、原矩阵与数组下标均从 0 开始时，位置 $(i,j)$ $(i\ge j)$ 之前有 $1+2+\cdots+i=i(i+1)/2$ 个元素，故数组下标为 $i(i+1)/2+j$。若原矩阵从 1 开始而数组从 0 开始，公式相应变成 $i(i-1)/2+j-1$。推导的顺序始终是“前面完整行或列数＋本行或列内偏移”。

### 3.6.1 例 3-3　三对角矩阵的起止行不能一律算 3 个

**例 3.6.1** (配套例题)

配套例题：100 阶三对角矩阵 $M$ 的元素 $m_{i,j}$ $(1\le i,j\le 100)$ 按行优先依次压缩存入下标从 0 开始的一维数组 $N$。$m_{30,30}$ 在 $N$ 中的下标为（　）。

A. 86　B. 87　C. 88　D. 89

**解析**。第一行有 2 个元素，第 2 至 29 行各有 3 个，共 $2+28\times3=86$ 个在第 30 行之前。第 30 行依次保存 $m_{30,29},m_{30,30},m_{30,31}$，目标是本行第 2 个，因此下标为 $86+1=87$，答案为 **B**。这里 87 是从 0 开始的下标，目标是第 88 个被保存的元素。

### 3.6.2 例 3-4　先用对称性换到实际保存的半边

**例 3.6.2** (配套例题)

配套例题：10×10 对称矩阵 $M$ 的上三角元素 $m_{i,j}$ $(1\le i\le j\le 10)$ 按列优先存入 C 语言一维数组 $N$。$m_{7,2}$ 在 $N$ 中的下标为（　）。

A. 15　B. 16　C. 22　D. 23

**解析**。$m_{7,2}=m_{2,7}$，实际寻找第 7 列第 2 行。前 6 列各存 $1,2,\dots,6$ 个，共 21 个；本列目标前还有第 1 行 1 个元素，所以从 0 开始的下标为 $21+1=22$，答案为 **C**。不能先把下三角坐标直接代入上三角公式，再试图以加减 1 修正。

上三角若改为按行优先，各行长度为 $n,n-1,\dots,1$；第 $i$ 行之前有 $(i-1)(2n-i+2)/2$ 个保存位置。下三角若按列优先，各列长度同样递减，但列内偏移为 $i-j$。公式外形相近，决定它的是保存方向与下标原点。

**稀疏矩阵**的非零位置没有固定三角规律，需要连同位置一起保存。三元组顺序表记录 `(行,列,值)`，还须保存矩阵行列数与非零元数；十字链表为同一个非零元同时设置行链接和列链接。压缩后不能再按普通二维数组地址公式随机访问任意元素，查找与修改成本随表示方法而变。

**最小反例·括号计数看不到嵌套**

`([)]` 中每一类左右括号数都相等，但第三个字符要求圆括号配对，栈顶却是方括号。配对必须遵守最近未完成对象先结束，不能在扫描后任意重排。

## 3.7 表达式、递归与层次处理

### 3.7.1 括号与表达式中的“最近一个尚未完成对象”

左括号入栈；遇右括号时，栈顶必须是对应种类的左括号，匹配后弹出；扫描完后栈必须为空。`([)]` 中括号数量相等，但遇到第一个右圆括号时栈顶为左方括号，已经失败。这个过程正好需要最后遇到且尚未配对的对象，所以选择栈。

后缀表达式中，操作数先入栈，遇二元运算符时先弹出的数是**右操作数**，后弹出的是左操作数，计算后将结果入栈。减法、除法不能颠倒。合法的二元后缀表达式在遇到每个运算符时，栈中至少有两个可用值，最终恰好留下一个结果。

### 3.7.2 例 3-5　弹出次序怎样影响减法

**例 3.7.1** (配套例题)

配套例题：栈 $S_1$ 保存整数，$S_2$ 保存运算符。函数 F() 依次：（1）从 $S_1$ 弹出操作数 a、b；（2）从 $S_2$ 弹出运算符 op；（3）执行 b op a；（4）把结果压入 $S_1$。初始 $S_1$ 自底向顶为 5、8、3、2，$S_2$ 自底向顶为 *、−、+。调用 3 次 F() 后，$S_1$ 栈顶值是（　）。

A. $-15$　　B. 15　　C. $-20$　　D. 20

**解析**。第一次弹出 2、3，用加法得到 $3+2=5$，整数栈变为 5、8、5。第二次弹出 5、8，用减法得到 $8-5=3$，栈变为 5、3。第三次得到 $5\times 3=15$，答案为 **B**。若第二次按弹出先后计算 $5-8$，就会得到错误的负数。

中缀转后缀可用运算符栈：操作数直接输出；左括号入栈；右括号触发输出直到匹配左括号；遇运算符时，先输出栈顶中优先级更高的运算符。对通常左结合的 `+`、`-`、`*`、`/`，同级也先弹出；若加入右结合运算符，同优先级的处理规则需要调整。

例如 $x+y(z-u)/v$（其中相邻表示乘法）先把括号内写成 `zu-`，乘除按左结合形成 `yzu-*v/`，最后与 x 相加，得 `xyzu-*v/+`。这里的输出顺序表达运算依赖，而不是把原式的符号按种类重新排列。

### 3.7.3 递归返回前保存了什么

bool不变 top量·含义决定增减位

topbool指栈顶时，空状态−1为，入栈先加再top写；指下一空位时，空状态0为，入栈先写再加。用空栈和单元素两幅状态检查，比只push背语句更能应对改约定

递归调用栈保存尚未完成的调用。调用树的一个结点表示一次调用，不能把函数返回值当作调用次数。对于

```
int F(int n) {
    if (n<=3) return 1;
    return F(n-2)+F(n-4)+1;
}
```

令 $C(n)$ 表示含当前调用在内的调用次数，则 $n \le 3$ 时 $C(n)=1$，否则 $C(n)=1+C(n-2)+C(n-4)$。因此 $C(4)=C(2)+C(0)+1=3$，$C(6)=1+3+1=5$，$C(8)=1+5+3=9$。式中返回值的“+ 1”不是另一次函数调用；次数递推中的“+ 1”是当前调用本身。此过程对应〖3.3-006〗的递归调用模型。

C/C++ 对加号两侧函数调用的求值先后并不一概保证从左到右。若题目问“第几个被调用”，应检查答案是否依赖这个先后，不能把一种习惯顺序当作语言保证。调用总数与最终结果在本例不受这个先后影响。另如〖3.3-007〗把递归式改为 `func(x-2)+func(x-4)`、基例返回 2，并计算 `func(func(5))`：必须先完成内层 func(5)，它和 func(3)、func(1) 共 3 次调用，结果为 4，所以第 4 次是外层 func(4)；内层两个基例谁先执行不影响这个结论。

层次处理需要先进先出的队列：先发现的一层结点必须先扩展，后发现的下一层排在后面。第 5 章层序遍历与第 6 章 BFS 都用这条约束，但图还需 `visited` 防止同一顶点反复入队。

## 3.8 从导图恢复栈与队列的核心接口

**定义 3.8.1** (先约定指针指向什么，再写边界)

顺序栈中的 top 可以指栈顶元素，也可以指下一空位；循环队列中的 front/rear 也有不同语义。表示约定决定空满条件、增减时机和长度公式。同一个变量名并不保证同一种表示，改题时先画空状态和单元素状态。

以下栈使用“top 指当前栈顶”的约定；下标从 0 开始，空栈 top 为 $-1$。

```
constexpr int SMAX = 100;
struct Stack { int data[SMAX]; int top = -1; };
bool push(Stack& s, int x) {
    if (s.top == SMAX - 1) return false;
    s.data[++s.top] = x;
    return true;
}
bool pop(Stack& s, int& x) {
    if (s.top == -1) return false;
    x = s.data[s.top--];
    return true;
}
```

循环队列使用 “front 指队头元素，rear 指下一入队位” 的约定，牺牲一个单元；数组长度固定为 8，最大有效长度为 7。

```
constexpr int QMAX = 8;
```

```
struct RingQueue { int data[QMAX]; int front = 0, rear = 0; };
bool enqueue(RingQueue& q, int x) {    if ((q.rear + 1) % QMAX == q.
   front) return false;
    q.data[q.rear] = x;
    q.rear = (q.rear + 1) % QMAX;
    return true;
}
bool dequeue(RingQueue& q, int& x) {
    if (q.front == q.rear) return false;
    x = q.data[q.front];
    q.front = (q.front + 1) % QMAX;
    return true;
}
```

**不变量·先换到已保存半边**

对称矩阵访问未存的半边时，先交换 i、j，再按保存方向计数。坐标转换解决的是表示对应；数组从 0 还是 1 起始解决的是编号，两步不能互相替代。

**环形长度公式来自有向距离**

从 front 沿数组下标递增方向走到 rear，步数就是有效元素数。未绕回时为 rear−front，绕回时为 M−front+rear，合并为 $(rear - front + M) \bmod M$。牺牲一个单元将步数限制在 0 至 M−1，从而只让 0 表示空；若允许 M 个元素，模运算会把长度 M 也变成 0，必须增加 size 或标志保存这一个区别。

□

| 环形队列策略 | 空条件 | 满条件 | 可用容量 |
|---|---|---|---|
| 牺牲一个单元 | front=rear | (rear+1) mod M=front | M−1 |
| 增加 size 计数 | size=0 | size=M | M |
| 增加最近操作 tag | front=rear 且 tag 为出队 | front=rear 且 tag 为入队 | M |

tag 方案初始设为 “出队” 态，初始化即表示空；成功入队/出队后更新 tag。计数方案每次成功修改长度，失败操作不能改 size。链队列没有预设数组容量，但申请新结点仍可能失败，不能将 “不判数组满” 理解为内存无限。

## 3.9 矩阵压缩：由坐标到下标，再从下标恢复坐标

**完整行数加行内偏移**

每次先写清：矩阵坐标从 0 还是 1 开始、压缩数组从 0 还是 1 开始、上/下三角、行/列优先。若是对称矩阵，先把目标换到实际保存的半边。最后数前面完整行或列，再加本行/列内偏移；不要用最后加减 1 修补错误的保存方向。

对于坐标从 0 开始、按行保存下三角的矩阵，令 $T_i = i(i+1)/2$，则 $k = T_i + j$。反向给定有效数组下标 k 时，先找到唯一满足 $T_i \le k < T_{i+1}$ 的 i，再令 $j = k - T_i$。数学上

$$i = \left\lfloor \frac{\sqrt{8k+1}-1}{2} \right\rfloor, \qquad j = k - \frac{i(i+1)}{2}.$$

例如 k=8，因 $T_3 = 6 \le 8 < T_4 = 10$，得 $i = 3, j = 2$。实现时若用浮点平方根，应以整数三角数边界复核，防止舍入把边界分到相邻行。

坐标从 1 开始、压缩数组从 0 开始时，上三角行优先下标为

$$k = \frac{(i-1)(2n-i+2)}{2} + (j-i), \quad 1 \le i \le j \le n.$$

下三角列优先则为

$$k=\frac{(j-1)(2n-j+2)}{2}+(i-j),\quad 1\le j\le i\le n.$$

三角矩阵的公共常量区可另存一个值，所需空间为 $n(n+1)/2+1$；对称矩阵只保存一侧即可，不需要这个公共常量单元。三对角矩阵 $n\ge 2$ 时按行存 3n−2 个有效位置，坐标从 1、数组从 0 的统一公式为 $k=2i+j-3$，前提是 $|i-j|\le 1$ 且坐标不越界。n=1 时只有一个元素，应按单元素矩阵处理，不再套首尾行各 2 个的解释。

稀疏矩阵中零元位置不具有固定三角规律。三元组的 “下标” 表示第几个非零记录，不能直接由坐标用普通二维地址公式得到；修改原来的零元为非零元时，可能需要插入记录并调整次序或链接。

**错项排除·FIFO 只是具体策略**

队列 ADT 规定先进先出，操作系统的 “就绪队列” 名称却不保证所有调度算法都按 FIFO 选择。先来先服务可借 FIFO 说明，优先级等策略需要额外排序或选择规则。

## 3.10 典型应用的失败条件

中缀表达式将运算符写在左右操作数之间；前缀写在两者之前，后缀写在两者之后。前缀求值从右向左扫描，遇运算符先弹出的值是左操作数、后弹出的值是右操作数；后缀从左向右扫描则先弹出右操作数。方向与角色要一起记。

例如 $x+y(z-u)/v$ 按通常左结合规则，前缀为 `+ x / * y - z u v`，后缀为 `x y z u - * v / +`。两种形式都用位置表达运算依赖而省去括号；不是简单地把所有运算符搬到字符串开头或末尾。

> **按尚未完成对象的次序选结构**
>
> 最近进入且尚未结束的对象要先处理，用栈：括号配对、函数调用、表达式中等待右操作数的运算。最早进入且尚未服务的对象先处理，用队列：层序遍历、BFS、先来先服务。操作系统的就绪队列只是管理等待对象，实际调度策略未必总是 FIFO。

后缀表达式求值必须同时检查：遇二元运算符时至少有两个操作数、除数合法、扫描末尾恰好留下一个结果。仅仅 “最终留下一个数” 不足以排除中途栈下溢。例如 `1 + 2` 按后缀语法扫描，在加号处就没有两个数；`1 2 3 +`则扫描后还剩两个值。

**例 3.10.1** (自拟：括号数量正确仍可失败)

输入 `([)]` 中左右圆括号和左右方括号数量分别相等。用栈检查时，在哪一个字符首次失败?

前两个字符依次入栈，栈顶为 `[`；读到第三个字符`)`时，它要求栈顶为 `(`，因此立即失败。只统计每类括号数量看不到嵌套关系，错误发生在第三个字符，不能等扫描完再随意重新配对。

## 3.11 主动回忆自测

第 2 题引自王道书，其余为原创补充，检查表示约定。

1. 数组有 8 个单元、牺牲一个单元的循环队列，`front=6,rear=2`，长度是多少？还能再入队几个元素?
2. 〖3.1-025〗顺序共享栈 `Share[0..n-1]`，第一个栈顶 top1 初值 −1，第二个栈顶 top2 初值 n，栈满条件是（　）。A. `top2-top1==1`

B. `top1-top2==1`　C. `top1==top2`　D. 都不对

3. 下标从 1 开始的三对角矩阵，若压缩数组也从 1 开始，非边界行元素 $a_{i,j}$ 的位置如何计算?

**答案**。1. 长度 $(2-6+8) \bmod 8 = 4$，最大容量 7，还能入队 3 个。2. **A**，两栈顶相邻时已无可用单元，不能等栈顶重合才判满。3. 前面有 $2+3(i-2) = 3i-4$ 个元素，本行从 $j = i-1$ 开始，故从 1 开始的位置为 $3i-4+[j-(i-1)]+1 = 2i+j-2$；前提为 $2 \le i \le n-1$ 且 $|i-j| \le 1$，首尾行须按实际保存位置解释。

**最小反例·同一对下标可能空也可能满**

数组长度 2、front=rear=0 可表示刚初始化的空队列，也可在允许全用两个单元时表示已入队 2 个元素。只有两个模下标丢失了一圈的信息，必须牺牲单元或增加状态。

### 3.11.1 补充自拟回忆题

**先画状态，再作判断**

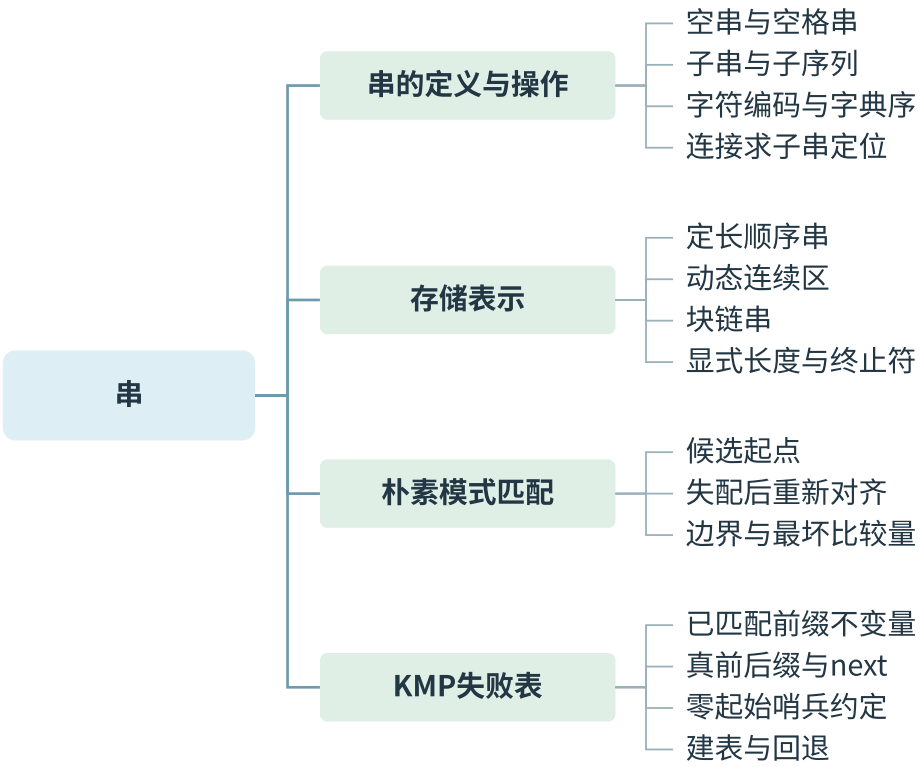
4. 入栈顺序为 a、b、c，输出 c、a、b 是否合法？第一次失败时栈顶是谁?
5. 带头结点链队列删除最后一个数据结点时，front 和 rear 应指向哪里?
6. 同一个 n 阶对称矩阵，为什么下三角行优先和上三角列优先有相同的三角数结构？坐标如何转换?
7. 后缀表达式 `8 3 - 2 /`的值是多少？若按整数除法计算，先弹出的数是什么角色?
8. 递归访问 n 个结点，最大同时活跃调用数一定为 n 吗？应由哪个结构参数决定?

**补充题核对答案**

4. 不合法。弹出 c 后栈中为 a、b，想输出 a 时栈顶是 b，而输入已用尽。
5. front 仍指有效头结点，rear 必须退回 front；头结点的 next 为空。rear 不能留在已释放的数据结点上。
6. 一边各行保存 1、2、⋯ 个元素，另一边各列也保存 1、2、⋯ 个。先用 aij=aji 交换坐标，再按对应行或列求偏移。
7. 先计算 8−3=5，再作 5/2，整数结果为 2；每次先弹出右操作数，除法的除数是先弹出的 2。
8. 不一定。由最长未返回调用链决定；树遍历通常取树高 h，最坏链形树才可达到 n。

# 4 串

**知识导图**




**最小反例·空格仍是字符**

空串长度为 0，一个空格构成的串长度为 1。仅凭打印后看起来空白无法判断串是否为空；接口判空应检查长度或表示规则，而不是观察屏幕上的可见字形。

**知识导图（续）**




**错项排除·一次候选失败才移起点**

朴素法在某候选起点内逐字符比较，失配后起点才加 1；主串和模式的工作指针可以回退。不能把朴素法的“重试下一个起点”与 KMP 保持 i 不动混为同一规则。

模式匹配要在主串中寻找一段连续字符，使它与模式串逐字符相同。朴素方法失败后重试下一个起点；KMP 利用已经相等的部分决定怎样移动模式串。二者处理的是同一个匹配问题，差别在于是否重复比较已经确认相等的字符。

**不变量·回退保留相等边界**

失配时已知 T[i−j..i−1] 等于 P[0..j−1]。新候选只可保留旧段中同时是模式前缀的后缀；next[j] 给出最长可保留长度，主串当前字符尚未被丢弃。

## 4.1 串、子串与下标

**串**是由零个或多个字符组成的有限序列，串长为字符个数。长度为 0 的是空串；只含空格的串仍含字符，不是空串。**子串**必须由原串中连续的一段字符构成，允许不连续选取字符得到的是子序列。两个串相等，须长度相等且每个对应字符相等。

本章令主串为 $T[0..n-1]$，模式串为 $P[0..m-1]$，匹配位置返回从 0 开始的下标。题目若使用从 1 开始的位序，最终位置加 1，但中间 `next` 数组还涉及失败哨兵的约定，不能只把表中某一列机械加 1。空模式是否匹配应由接口约定；以下算法把空模式匹配位置约定为 0，主要讲解 $m>0$ 的情形。

串可用定长数组、动态分配的连续存储区或块链存储。C 语言中以 `\0` 结束的字符串是一种具体表示；结束符本身不计入通常的串长。使用显式长度的串也可以包含值为 0 的字符。求子串、连接、替换与模式匹配是不同运算，不能把“在主串中找到与模式串相同的一段字符”称为串连接。

## 4.2 朴素匹配怎样回退

先把模式首字符与主串起点 $s=0$ 对齐，从头逐一比较。若某处不同，则把起点改为 $s+1$，重新从模式首字符比较。若模式全部相等，返回 $s$；若对每个满足 $s \le n-m$ 的起点都失败，则不存在匹配。

```
naive(T, n, P, m):
    for s=0 to n-m:
        j=0
        while j<m and T[s+j]==P[j]: j=j+1
        if j==m: return s
    return -1
```

当 $m>n$ 时，没有候选起点，直接失败。一次尝试最多比较 $m$ 次，总共最多 $n-m+1$ 个起点，所以最坏时间为 $O((n-m+1)m)$，常简记为 $O(nm)$；辅助空间为 $O(1)$。

最坏情况来自“前面大段都相同，到末尾才失败”。例如主串连续很多个 a，模式为 `aaaab`。每次对齐都要重新比较一段 a，而这些比较并不能排除紧接着的下一个起点。若每次第一个字符就失配，比较次数则近似候选起点数。不能把最坏上界当成每次的实际比较次数。

## 4.3 KMP 的 next 为什么只依赖模式串

设当前已经匹配 $j$ 个字符，即 $T[i-j..i-1]=P[0..j-1]$，现在 $T[i] \ne P[j]$。主串这段已知内容就是模式的前 $j$ 个字符。要让模式右移后仍保留尽可能多的已匹配字符，新的模式前缀必须等于旧匹配段的后缀。

因此，要在 $P[0..j-1]$ 中找最长的一对相等真前缀与真后缀，设其长度为 $k$。真前缀和真后缀都不允许取整个串。右移后，旧段最后 $k$ 个

字符与新模式前 $k$ 个字符已经相等，下一步只需比较 $T[i]$ 与 $P[k]$。主串指针 $i$ 保持不变，模式指针从 $j$ 退到 $k$，模式整体右移了 $j-k$ 位。

错项排除 · next 不含当前失配字符

本章 next[j] 研究 P[0..j−1] 的真前后缀，而不是 P[0..j]。例中失配下标 5 之前是 abaab，最长边界 ab 长 2；把 c 算进去会让表错位。

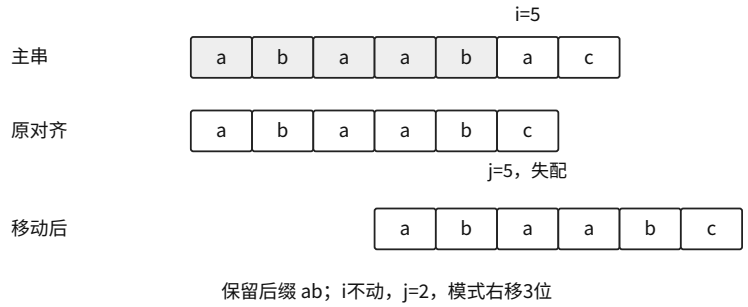



Figure 4.1: 已匹配段 abaab 的后缀 ab 与模式前缀 ab 对齐，主串当前字符保持原位

### 4.3.1 本书的 next 定义

对 $j \geq 1$，`next[j]` 等于 $P[0..j-1]$ 的最长相等真前后缀长度；令 `next[0]=-1` 作哨兵。`j=0` 已失配时，模式没有可保留的前缀，应把主串推进一位再从模式首字符开始；代码借 `j=-1` 统一完成这个动作。

以 `P=abaabc` 为例：

| 下标 j | 0 | 1 | 2 | 3 | 4 | 5 |
|---|---|---|---|---|---|---|
| P[j] | a | b | a | a | b | c |
| next[j] | −1 | 0 | 0 | 1 | 1 | 2 |

求 `next[5]` 时，考察的是位置 5 之前的 `abaab`，不把 `P[5]=c` 算进去。`abaab` 的最长相等真前后缀是 `ab`，长度 2，所以失败后去 `P[2]` 继续比较。若把 `next` 误算成“包含当前位置的前后缀长度”，就会把表错开一格。

```
buildNext(P, m):
    next[0]=-1
    i=0; k=-1
    while i<m-1:
        if k==-1 or P[i]==P[k]:
            i=i+1; k=k+1
            next[i]=k
        else:
            k=next[k]
```

建表函数的前提是 $m > 0$；空模式直接返回匹配位置 0，不调用建表函数。求表时同样是在模式内部做匹配：已知长为 $k$ 的相等前后缀，若下一字符也相等，就扩展一位；不等时，退到这段前缀自己的更短边界继续尝试。不能每次失配都直接把 $k$ 清零，否则会丢掉仍可复用的边界。

```
kmp(T, n, P, m, next):
    if m==0: return 0
    i=0; j=0
    while i<n and j<m:
        if j==-1 or T[i]==P[j]:
            i=i+1; j=j+1
        else:
            j=next[j]
```

```
    if j==m: return i-m
    return -1
```

主串指针只前进，不回退；模式指针可回退。每次字符相等时二者同时增加，失败时只有 $j$ 减少；$j$ 总减少量受此前总增加量约束。因此匹配扫描为 $O(n)$，求表为 $O(m)$，总时间 $O(n+m)$，辅助数组占 $O(m)$。“每个主串字符只能比较一次”并不成立：同一个主串字符可能与模式的多个位置比较，只是总比较量仍为线性。

**不变量·只跳过必然失败的位置**

若 P[next[j]] 与已经失配的 P[j] 相同，那么同一主串字符与它再比较仍失败。nextval 压缩这类失败链；只有获得这一字符相等条件，才能证明跳过不会漏解。

### 4.3.2 例 4-1　第一次失配后哪个指针改变

**例 4.3.1** (配套例题)

配套例题：字符串 s 为 `abaabaabacacaabaabcc`，模式串 t 为 `abaabc`。采用 KMP 匹配，第一次出现失配 `s[i]≠t[j]` 时，`i=j=5`，下次开始匹配时，i 和 j 分别是（　）。

A. i=1，j=0　B. i=5，j=0　C. i=5，j=2　D. i=6，j=2

**解析**。由“i=j=5 时第一次失配”可知本题使用从 0 开始的下标：前面已匹配 `abaab`，当前主串字符 a 与模式字符 c 不同。已匹配段的后缀 `ab` 可与模式前缀 `ab` 重合，因此 `j=next[5]=2`，i 仍为 5，答案为 **C**。此后比较的是同一个主串 a 与模式第 3 个字符 a；这一步若相等，才把 i、j 同时加 1。

若使用从 1 开始的 `next[1]=0` 约定，相同位置对应的指针是 `i=j=6`，失败后 `j=3`。它与从 0 开始的“5 退到 2”表达同一次对齐，不能把表的哨兵 0 和字符下标 0 当成同一个值。

### 4.3.3 nextval 省掉哪些一定失败的比较

若当前 `P[j]` 已与主串字符失配，而 `P[next[j]]==P[j]`，退到该位置再比较仍必然失败。可以继续沿失败链接退到不同字符，或直接退到哨兵。把这种合并后的失败位置存入 `nextval`。

在本章 0 起始约定下，先求普通 `next`，再从小到大求：

```
nextval[0]=-1
for j=1 to m-1:
    k=next[j]
    if k>=0 and P[j]==P[k]: nextval[j]=nextval[k]
    else: nextval[j]=k
```

这项优化只跳过“退过去仍因字符相同而失败”的比较，并不能任意加大右移距离；过度右移会漏掉合法匹配。某些实现把前缀函数记为 `pi` 或 `lps`，其第 j 项包含当前字符。读题时先确认定义，再换算表，不能仅凭数组名断定语义。

### 4.3.4 例 4-2　修正后最长能移动几位

**例 4.3.2** (配套例题)

配套例题：KMP 使用修正后的 next 数组进行模式匹配，模式串 S=`aabaab`。当主串中某字符与 S 中某字符失配时，S 向右滑动的最长距离是（　）。

A. 5　B. 4　C. 3　D. 2

**解析**。按本章定义，逐位置求表并计算 `j-nextval[j]`：

| j | 0 | 1 | 2 | 3 | 4 | 5 |
|---|---|---|---|---|---|---|
| S[j] | a | a | b | a | a | b |
| next[j] | −1 | 0 | 1 | 0 | 1 | 2 |
| nextval[j] | −1 | −1 | 1 | −1 | −1 | 1 |
| 右移距离 | 1 | 2 | 1 | 4 | 5 | 4 |

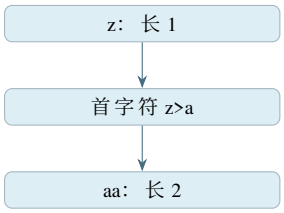
最长距离为 5，答案为 **A**。例如在 j=4 的 a 处失配，普通失败链接退到 j=1 的 a，再到 j=0 的 a，都会与当前主串字符重复失配；修正表直接到 −1。下一轮主串也前进一位，于是新对齐起点相对旧起点实际前进 5 位。

### 4.3.5 比较次数要与实际实现对应

普通 next 与 nextval 的字符比较次数可能不同；程序里对 `j==-1` 的判断不是一次字符比较。若题目要求“单个字符之间的比较次数”，只记录真正执行 `T[i]==P[j]` 的次数。一次匹配成功后是否继续查重叠出现，也取决于任务：只找首次出现可立即返回；要找全部出现，须保留末尾可重用边界，不能一律把模式指针清零并跳过已读位置。

**最小反例·长串未必大**

按通常英文编码的字典序，z 大于 aa，虽然前者只有一个字符。第一次不同字符就决定比较结果；只有短串完全等于长串前缀时，串长才参与最后判断。




字典序先看首个不同字符，z 大于 aa

## 4.4 字符编码、串比较与基本操作

**定义 4.4.1** (字符序列与表示单位)

串的逻辑元素是字符，串长是所约定字符单位的个数。编码把字符映射到计算机表示；比较、求长和取子串必须使用一致的单位。408 中的英文字符示例通常按单字节处理，不能据此把任意 UTF-8 文本的字节数直接当作用户看到的字符数。

串比较通常采用字典序：从首字符起寻找第一个不同位置，按该字符的编码序比较；若较短串的全部字符都等于较长串的对应前缀，则短串较小。长度不同不意味着长串一定大，例如按通常英文编码序，`z` 大于 `aa`。比较返回值一般只约定负、零、正，不保证恰为 −1、0、1。

赋值、判空、求长、清空、连接、求子串、串比较和定位是不同操作。连接把两串首尾接起来；求子串提取一个连续区间；定位返回模式首次出现的位置。复制输出区时还要考虑容量与源/目标是否重叠，不能只改元数据长度而不复制内容。

**例 4.4.1** (自拟：区分空串、空格串与前缀关系)

设 A 为空串，B 包含一个空格，C=`ab`，D=`abc`。哪些是空串？按字典序比较 C 与 D，并说明 D 中连续子串 `bc` 的位置。

只有 A 长度为 0；B 含一个字符。C 是 D 的真前缀，故 C 小于 D。`bc` 在 D 中的起始下标为 1，若题目用从 1 开始的位序则为 2。空串可视为任意串的子串，但接口仍须约定定位返回值；本章返回首次合法起点 0。

## 4.5 定长、动态与块链表示

课程连 接 · 编码单位决定下

字符编码把符号落实为存 UTF-8储。的一个字符可能占多个字节，字节下标不总是字符位置；本章英文题按单字节模型分析，换输入模型时先确定元素单

定长顺序串预分配最大容量，并另存实际长度；超容量时必须定义失败或截断行为。截断会改变运算结果，不能将它悄悄当作完整连接成功。堆分配顺序串按实际需要申请连续区，长度或容量改变时可能重新申请和复制，旧地址不保证保持有效。

块链串让一个结点保存多个字符，减少每字符一个指针的开销；末块若未用满，需要有效长度或明确的填补规则。链式表示适合某些局部接入操作，却不能因此宣称按字符下标访问为常数时间。C 风格以终止符`\0`判结束的字符串和“数组 + 显式长度”的串是不同表示；前者不能把中途的`\0`作为普通有效字符继续处理，后者可以。

| 表示 | 如何知道长度 | 主要边界 | 随机访问 |
|---|---|---|---|
| 定长数组 + 长度 | 显式 length | 最大容量、结果截断 | 字符下标 $O(1)$ |
| 动态连续区 + 长度 | 显式 length | 申请失败、扩容复制 | 字符下标 $O(1)$ |
| 块链存储 | 总长度及末块信息 | 跨块定位、未用字符位 | 通常需沿块查找 |
| C 终止符串 | 扫描到`\0` | 终止符与有效容量 | 已知下标可访问，求长需扫描 |

## 4.6 求子串的安全边界与实现

> **区间先统一成起点加长度**
>
> 用 0 起始下标 pos 与长度 len 表示半开区间 $[pos, pos+len)$。合法条件为 $0 \le pos \le n$、$0 \le len \le n - pos$；pos=n 只允许取空子串。与检查 pos+len 不越界相比，检查 len≤n−pos 还可避免实现中加法溢出。

```cpp
#include <string>
bool subString(const std::string& s, int pos, int len,
               std::string& result) {
    if (pos < 0 || len < 0) return false;
    const std::size_t first = static_cast<std::size_t>(pos);
    const std::size_t count = static_cast<std::size_t>(len);
    if (first > s.size() || count > s.size() - first)
        return false;
    result = s.substr(first, count);
    return true;
}
```

这里用标准字符串演示逻辑操作和边界，元素仍按字节处理。求长和下标的表示单位必须一致；失败时不修改 result，成功提取空串则会把 result 置空，两种结果不能只靠返回串内容区分。

## 4.7 KMP 的可执行实现与线性界

以下 C++17 实现与正文的 0 起始、`next[0]=-1`一致，前提是主串和模式串长度均可用 int 表示，且单个元素采用单字节比较。空模式单独成功，不访问 next[0]。

```cpp
#include <vector>
```

```cpp
std::vector<int> buildNext(const std::string& p) {
    const int m = static_cast<int>(p.size());
    std::vector<int> next(m, -1);    if (m == 0) return next;
    int i = 0, k = -1;
    while (i + 1 < m) {
        if (k == -1 || p[i] == p[k]) {
            ++i; ++k;
            next[i] = k;
        } else {
            k = next[k];
        }
    }
    return next;
}
```

```cpp
int kmpFirst(const std::string& text, const std::string& p) {
    const int n = static_cast<int>(text.size());
    const int m = static_cast<int>(p.size());
    if (m == 0) return 0;
    if (m > n) return -1;
    const auto next = buildNext(p);
    int i = 0, j = 0;
    while (i < n && j < m) {
        if (j == -1 || text[i] == p[j]) {
            ++i; ++j;
        } else {
            j = next[j];
        }
    }
    return j == m ? i - m : -1;
}
```

**允许重复比较，为何总量仍是线性**

每次成功匹配或哨兵推进都会使 i 增加，发生次数最多 n；其余迭代只令 j 严格下降。j 的增加总量受上述推进次数限制，而 j 最低为 $-1$，因此全部下降量也为 $O(n)$，下降迭代数不会超过该量。两类迭代合计 $O(n)$。建表时将 i、k 作同样分析，i 最多推进 m$-1$ 次，k 的回退受其此前增长限制，故预处理为 $O(m)$，总时间 $O(n+m)$。

□

本证明没有声称每个主串字符只比较一次。例如同一个 `text[i]` 可在 j 依次回退时与多个模式字符比较；真正受控的是总的推进与回退数量。计算字符比较次数时，`j==-1` 这样的整数判断不是字符比较；不同 next/nextval 实现也可能有不同具体次数。

## 4.8 由历史错题检查 KMP 的单调量

配套做题本第 4 章 4.2 第 2 题问 “指示主串的指针” 的特点。标准 KMP 中主串指针可以前进，也可以在模式回退时保持原位，但不会变小。因此应选 “不会变小”，而不是把 “可以前进、可以不动” 误解成 “可以回退”。模式指针才沿 next 失败链接回退，二者角色必须分清。

## 4.9 主动回忆自测

以下为原创补充，用于检查失败表定义及边界。

1. 对模式 `aaaa`，按本章约定写普通 next 与 nextval。

**错项排除·不会变小才是准确表述**

标准 KMP 中 i 在匹配成功或哨兵推进时变大，在模式回退时不动。因此 “不会变大” 错误，“可能变小” 也错误；主串指针的单调不减与模式指针的失败回退必须分开看。

2. 某主串字符与模式 `abab` 的最后一个 b 失配，普通 next 应退到哪里？主串指针是否前进？
3. 主串长 $n$、模式长 $m$，只执行一次 KMP 匹配但预先已有可用 next 表，扫描部分复杂度是多少？

**答案**。1. next 为 `-1,0,1,2`，nextval 为 `-1,-1,-1,-1`。2. `next[3]=1`，先退到模式下标 1，主串指针不动；若使用 nextval，因两个位置都是 b，可直接退到 0 继续。3. $O(n)$；总算法连同求表才记为 $O(n+m)$，表的存储仍要计入空间开销。

**不变量·半开区间分清空与越界**

从 pos 取 len 个字符表示半开区间 [pos,pos+len)。pos 等于 n 时只可取长度 0；检查 len 不超过 n−pos，既表明边界，又避免先计算 pos+len 可能产生的整数溢出。

### 4.9.1 补充自拟回忆题

**按导图解释对象、表示与失败转移**

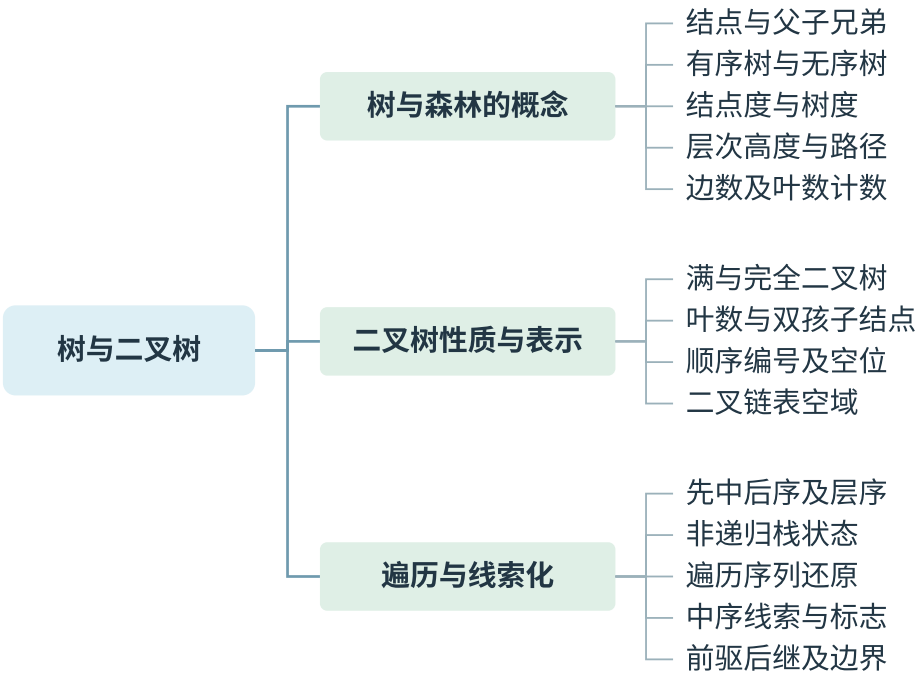
4. 字典序中 `ab` 与 `b` 谁大？是否只按串长比较？
5. 求子串 pos=n、len=0 合法吗？pos=n、len=1 呢？
6. 长为 m 的模式比主串长时，为何可以直接返回失败？空模式为什么要另行约定？
7. 本章 next[j] 是否包含 P[j]？若把它当作包含当前位置的前缀函数会错在哪里？
8. nextval 把多个同字符失败位置跳过，为什么仍不遗漏合法匹配？

**补充题核对答案**

4. b 大于 ab，因为第一处 a 与 b 已决定大小；长度只在较短串完全是较长串前缀时决定次序。
5. 前者合法，表示末端空子串；后者越界。
6. 非空模式必须落在主串的一个等长连续区间，m>n 时没有候选起点；空模式不需要字符比较，接口须约定返回值，本章返回 0。
7. 不包含 P[j]，只研究 P[0..j−1]。当作包含当前位置的前缀函数，会使失败表语义错一格，改变回退位置。
8. 已知当前主串字符不等于 P[j]；若更短候选位置仍是同一字符，比较必然失败，可继续回退。只跳过能证明失败的位置，不是任意加大位移。

# 树与二叉树 5

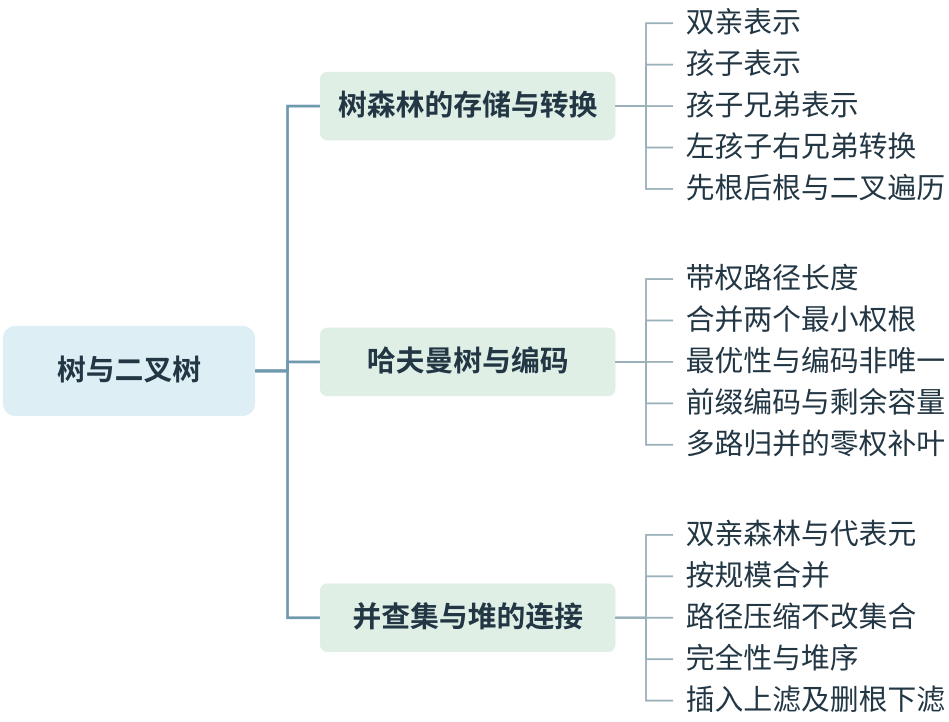
**知识导图**




**课程连接·目录树和进程树**

操作系统的目录层次、进程派生关系可用树表达。硬链接若允许同一对象有多个上级入口，目录对象关系就不能直接套用“除根外唯一双亲”；先核对对象与链接的含义。

知识导图（续）




**最小反例·度为 2 未必是二叉树**

最小情形只有根与一个孩子：普通无序树不区分这个孩子在左还是在右，二叉树却区分两种位置。因此“每结点至多两个孩子”尚未表达二叉树的全部结构语义。

树用父子关系组织层次。二叉树在此基础上区分左、右位置，遍历把结构转成一个处理次序；哈夫曼树、并查集和堆则各自增加约束，分别处理编码、集合合并和最值访问。它们共享树的表示工具，但各自的性质不能互相替代。

## 5.1 导图主干与阅读路径

- **树与森林的概念**：结点与父子兄弟；有序树与无序树；结点度与树度；层次高度与路径；边数及叶数计数。
- **二叉树性质与表示**：满与完全二叉树；叶数与双孩子结点；顺序编号及空位；二叉链表空域。
- **遍历与线索化**：先中后序及层序；非递归栈状态；遍历序列还原；中序线索与标志；前驱后继及边界。
- **树森林的存储与转换**：双亲表示；孩子表示；孩子兄弟表示；左孩子右兄弟转换；先根后根与二叉遍历。
- **哈夫曼树与编码**：带权路径长度；合并两个最小权根；最优性与编码非唯一；前缀编码与剩余容量；多路归并的零权补叶。
- **并查集与堆的连接**：双亲森林与代表元；按规模合并；路径压缩不改集合；完全性与堆序；插入上滤及删根下滤。

先按导图说清对象与条件，再沿相应正文执行算法或推导。章末先遮住答案回忆各主枝；不能只凭图中文字是否熟悉判断掌握。

## 5.2 树的结点、度、层次与路径

**定义 5.2.1** (树及结点的度)

**有序树**规定同一结点各子树的先后，交换两个孩子可能改变树；**无序树**不区分这种先后。森林是零棵或多棵互不相交的树，删去一棵非空树的根后，各子树构成森林；在一片森林上加新根、把各棵树根接为孩子，又得到一棵树。普通树的有序性与“关键字从小到大”无关。

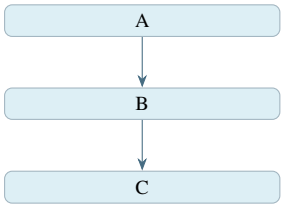
非空树有且仅有一个根。除根外每个结点恰有一个双亲，根到任意结点只有一条简单路径；树的各子树互不相交。一个结点的孩子数称为该结点的**度**，树的度是所有结点度的最大值。度为 0 的结点称为叶结点。树中“度”通常指数孩子的个数，与无向图把关联边条数称为度的约定不同。

本书把根记为第 1 层，树高为最大层号，空树高度为 0。结点的深度按根起的边数计，因此根深度为 0，某结点层号等于深度加 1；结点高度从该结点到最深后代的结点层数计。题目改为“根高度 0”时，要整体换算，不能只改其中一处。

路径长度以边数计，不以沿途结点数计。树的路径长度是根到各结点路径长度的总和；树高由最长的一条路径决定，二者不同。孩子、子孙、兄弟也要区分：兄弟共有同一个双亲，与堂兄弟不是同一个概念。

**错项排除·叶公式只用了二叉条件**

证明 $n_0 = n_2 + 1$ 时没有用到“完全”或“满”。若选项声称任意二叉树还必有至多一个度为 1 结点，就偷偷增加了完全性；一条三结点单支链即可排除。




三结点单支链有 2 个度 1 结点；叶数公式仍成立。

### 5.2.1 例 5-1　怎样从非叶结点数求叶结点数

**例 5.2.1** (配套案例)

【配套例题】一棵度为 4 的树，含 20 个度为 4 的结点、10 个度为 3 的结点、1 个度为 2 的结点、10 个度为 1 的结点。叶结点数为（　）。

A. 41　B. 82　C. 113　D. 122

**解析**。设叶数为 $n_0$。一方面总结点数为 $n = n_0 + 20 + 10 + 1 + 10 = n_0 + 41$；另一方面每条边从一个双亲指向一个孩子，边数是度数总和 $4 \times 20 + 3 \times 10 + 2 + 10 = 122$。树有 $n - 1$ 条边，所以 $n_0 + 40 = 122$，得 $n_0 = 82$，答案为 **B**。

一般地，令 $n_i$ 为度为 $i$ 的结点数，由 $\sum_i n_i - 1 = \sum_i i n_i$ 得

$$n_0 = 1 + \sum_{i \geq 2} (i-1) n_i .$$

度为 1 的结点不出现在右端，是因为在一条边上再串入一个单孩子结点，结点数、边数各增加 1，叶数不变。因此只给定 $n_2, n_3$ 和叶数，未必能确定全树结点总数；还可能插入任意多个度为 1 的结点。

每结点至多有 $m$ 个孩子的树，第 $i$ 层至多有 $m^{i-1}$ 个结点，高度 $h$ 至多容纳 $(m^h - 1)/(m-1)$ 个结点（$m > 1$）。求最小高度时，找使容量至少达到 $n$ 的最小整数 $h$。例如至多 3 叉、50 个结点，高 4 时最多 40 个，高 5 时最多 121 个，所以最小高度 5。这里必须把“度为 3”理解为全树最大孩子数 3，不能要求每个非叶结点都恰有 3 个孩子。

## 5.3 二叉树、满二叉树与完全二叉树

二叉树的每个结点至多有两个孩子，而且左右有别。只有一个孩子时，左孩子与右孩子形成不同结构；二叉树也不是简单地把“度不超过 2 的普通树”换个名字。普通树的单孩子若不规定位置，就不区分左、右这两种形态。

高度为 $h$、每一层都达到最大结点数的二叉树称为**满二叉树**，共有 $2^h - 1$ 个结点。**完全二叉树**按满二叉树从上到下、从左到右编号，已有结点恰占前 $n$ 个编号。最后一层可以不满，但只能缺右侧连续的一段；其余各层则必须排满。

“每个非叶结点恰有两个孩子”只说明不存在度为 1 的结点，未必是这里定义的满二叉树。叶结点可能分布在不同层。使用外文资料中的 full、perfect 等名称时，也要先看定义。

二叉树第 $i$ 层至多 $2^{i-1}$ 个结点。非空二叉树有 $n_0 = n_2 + 1$，由上一节公式直接得到。高度为 $h$ 时，结点数满足 $h \leq n \leq 2^h - 1$；给定 $n$，最小高度为 $\lceil \log_2(n+1) \rceil$，最大为 $n$。例如 50 个结点，最小高度 6。

完全二叉树有 $\lfloor n/2 \rfloor$ 个非叶结点、$\lceil n/2 \rceil$ 个叶结点，最多只有一个度为 1 的结点，且它只能有左孩子。普通二叉树不能沿用这些完全性结论。

**定理 5.3.1** (二叉树的叶与双孩子结点)

任意非空二叉树中，若 $n_i$ 表示度为 $i$ 的结点数，则 $n_0 = n_2 + 1$。

**按结点与按孩子数边**

**不变量·编号表达位置而非秩**

顺序存储中下标表达满二叉树的位置，删除缺失位置后不能把剩余元素紧凑前移而继续沿用 $2i, 2i+1$。链表遍历得到的次序也不是这种位置编号。

结点总数 $n = n_0 + n_1 + n_2$，边数一方面为 $n-1$，另一方面为 $n_1 + 2n_2$。令两式相等并消去 $n_1$，即得 $n_0 = n_2 + 1$。证明只需二叉树，不要求完全、满或平衡。

□

**最小反例·先序加后序的歧义**

最小反例为 A 与其唯一孩子 B：B 在左或在右，先序都是 AB，后序都是 BA。缺少中序时，序列没有给出这个单孩子的左右位置；重复值还会带来身份歧义。

## 5.4 顺序编号与链式表示

完全二叉树从 1 开始编号，结点 $i$ 的双亲为 $\lfloor i/2 \rfloor$，左、右孩子为 $2i, 2i+1$，但只有编号不超过 $n$ 的位置才存在。若下标从 0 开始，孩子公式改为 $2i+1, 2i+2$，非根结点的双亲为 $\lfloor (i-1)/2 \rfloor$。

任意二叉树也可以按满二叉树位置编号，但缺失的位置必须留空。因此实际结点少，顺序存储占用的空间未必小。高度 $h$ 的一条全右链，从 1 开始编号时最后编号为 $2^h - 1$，只存 $h$ 个结点却可能需要指数级下标范围。

### 5.4.1 例 5-2　任意形态的顺序存储需要预留多少

**例 5.4.1** (配套案例)

【配套例题】对于任意一棵高度为 5、有 10 个结点的二叉树，采用顺序存储结构保存，每个结点占 1 个存储单元，只存放结点数据信息。存放该二叉树需要的存储单元数量至少为（　）。

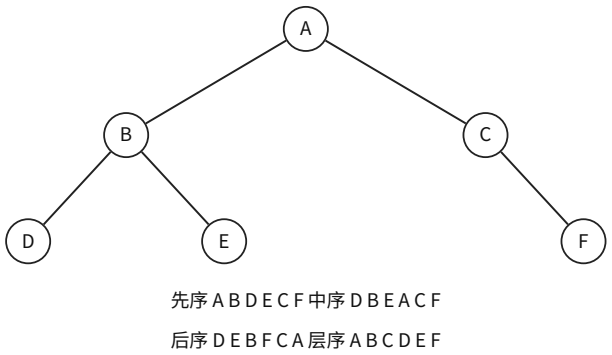
A. 31　　B. 16　　C. 15　　D. 10

**解析**。本题要求容量能够保存这种约束下的任意形态。高度 5 的满二叉树位置编号到 31；合法的 10 结点形态完全可能包含第 31 号位置（保留通往该位置的祖先，再安排其余结点）。因此为保证任意形态可保存，至少预留 31 个位置，答案为 **A**。并非每一棵给定的 10 结点树都实际用到下标 31；不能把 “保证任意形态” 读成 “在所有形态中挑一个最省空间的”。

二叉链表为每个结点保存左、右孩子指针。$n$ 个非空结点共有 $2n$ 个孩子指针域，真正指向孩子的有 $n-1$ 个，所以空域为 $n+1$ 个。若再增加双亲指针，根的双亲域为空，其他 $n-1$ 个双亲域非空；总空域为 $3n - 2(n-1) = n+2$。某个结点被多少指针指向，不只由它自己有几个孩子决定：孩子的双亲域也会指回它。

## 5.5 遍历、还原与树形递归

先序、中序、后序分别把 “访问根” 的位置放在左、右子树递归的前、中、后；三个次序都先处理左子树再处理右子树。对下图：




**不变量·栈保存尚未完成的祖先**

非递归中序栈中结点的左侧处理尚未完全结束，弹出后才访问根并转入右子树。后序还要记录右子树完成状态，不能把中序的弹栈时机直接挪用。

Figure 5.1: 同一棵二叉树的先中后序及层序；空的左孩子不应被当作另一个结点

先序为 A B D E C F；中序为 D B E A C F；后序为 D E B F C A；层序为 A B C D E F。先序在深入子树之前处理结点，适合复制或输出结构；后序在左右结果齐备后处理根，适合求子树大小、高度、释放结点。访问位置应服从依赖，不能只因为某种遍历最常见就选它。

```c
int height(Node *t) {
    if (t==NULL) return 0;
    int hl=height(t->left), hr=height(t->right);
    return 1+(hl>hr ? hl : hr);
}
```

这段代码每个结点只处理一次，时间 $O(n)$；调用栈深为 $O(h)$。求高度时，叶结点返回 1，与本章高度约定一致。交换所有左右子树可以递归地先交换两个子树内部，再交换两个孩子指针，从而把中序序列整体反向；交换不改变根的身份，也不改变哪些结点是叶结点。

### 5.5.1 先序加中序为什么能还原

假设结点标识互不相同。先序第一个元素为根，在中序中找到根后，左侧全属于左子树，右侧全属于右子树；由左子树的长度，能把先序余段分成左右两段，再递归。上图根为 A，中序左段 D B E 长 3，故先序左段是 B D E，根 B 又把它分成 D 和 E。

中序加后序也能唯一还原，根取后序最后一个；先序加后序一般不能区分一个单孩子在左还是在右。只给含 $n$ 个不同标识的先序序列，形态数仍为卡特兰数 $C_n$，例如 4 个元素为 14 种。结点值重复时，“在中序中找到根的位置” 可能不唯一，前述唯一性需要额外身份信息。

非递归中序遍历用显式栈保存未完成的祖先：沿左链入栈，到空位置时弹栈访问，再转到其右子树。上图先依次压入 A、B、D，弹出 D 后没有右孩子，接着弹出 B 并转到 E；E 处理完才回到仍在栈中的 A。

```
inorder(root):
    p=root; 建空栈 S
    while p 非空 or S 非空:
        while p 非空:
            push(S,p); p=p->left
        p=pop(S); visit(p)
        p=p->right
```

非递归后序还需分辨右子树是否已经处理，可保存最近访问结点 `last`。从左子树返回时先看栈顶；它有尚未处理的右子树，就进入右子树，不能立即弹根。

错项排除 · 先看 tag 再沿指针

线索填写后，一条非空指针可能指向祖先或兄弟方向的前后继。若忽略 tag 把它当孩子继续递归，就可能绕回已访问位置；``指针非空'' 不等价于 ``存在子树''。

```
postorder(root):
    p=root; last=NULL; 建空栈 S
    while p 非空 or S 非空:
        if p 非空:
            push(S,p); p=p->left
        else:
            q=top(S)
            if q->right 非空 and last!=q->right:
                p=q->right
            else:
                visit(q); last=pop(S)
```

`last` 只在一次完整子树处理后更新，因此它等于 `q->right`，表示右子树根已经最后访问过。每个结点各入栈、出栈一次，时间仍为 $O(n)$，栈最多同时保存一条祖先路径，空间为 $O(h)$。

层序遍历把根入队，反复出队访问并依次入队左右孩子。判定完全二叉树可在层序遍历中设置 “已经遇到空孩子” 标志：一旦出现空位置，此后的孩子位置不得再出现非空结点。只排除 “有右孩子而无左孩子” 还不够，还要排除遇到首个空孩子位置后又出现非空孩子的情况；一个结点只有左孩子，本身未必违反完全性。

### 5.5.2 例 5-3　中序前驱和后继在哪一侧

**例 5.5.1** (配套案例)

【配套例题】p、q、v 都是二叉树 T 中的结点，v 有两个孩子，T 的中序遍历序列形如 …,p,v,q,…。正确的是（　）。

A. p 没有右孩子，q 没有左孩子　B. p 没有右孩子，q 有左孩子

C. p 有右孩子，q 没有左孩子　D. p 有右孩子，q 有左孩子

**解析**。v 有左子树，紧邻它之前的 p 必在左子树的最右端；若 p 还有右孩子，应先访问那棵右子树，p 就不可能紧邻 v。对称地，q 是右子树的最左端，不可能有左孩子。答案为 **A**。这里的 “最右端” 指沿右孩子指针一直走到不能再走的位置，不能按图上哪个结点画得最靠右来判断。

> **遍历问题先固定三个条件**
>
> 先写清访问根的位置、左右孩子顺序、空树处理。还原问题再检查结点身份是否唯一；若给先序与中序，根由先序首项决定，子树边界由中序决定，不能按数值大小猜左右。

层序遍历的队列表示 “已发现但尚未处理” 的结点。以本章图为例，A 出队后 B、C 入队，接着 B 出队才把 D、E 加入队尾，所以 C 在 D 之前处理。

```
levelOrder(root):
    if root为空: return
    Q入队root
    while Q非空:
        p=Q出队; visit(p)
        if p.left非空: Q入队p.left
        if p.right非空: Q入队p.right
```

时间 $O(n)$；队列峰值由树的宽度决定，最坏 $O(n)$。不能把递归遍历的 $O(h)$ 调用栈结论搬到层序队列。

不变 量 · 转换不改变数据结

孩子兄弟转换保留全部数据结点，但改变链接解释。原树根无兄弟，所以转换后二叉根无右孩子；森林的多个根由右链串起，不能把右链误认成原树的后

## 5.6 线索二叉树：区分孩子链接与遍历线索

二叉链表有许多空孩子指针。把空左域改记某遍历次序下的前驱，把空右域改记后继，便得到相应的**线索二叉树**。新增 `ltag`、`rtag` 区分该域是孩子还是线索；本书约定 0 表示孩子，1 表示线索。寻找子树时必须先看标志，不能沿一条线索继续递归，否则可能回到祖先形成循环。

中序线索树中，若结点右域为线索，可直接到中序后继；若右域为孩子，后继是右子树中沿真实左孩子链到达的第一个结点。前驱同理：左域为线索则直接取，否则到左子树最右端。图中每个结点的 L、R 分别表示左、右指针域，其后数字为对应的 `ltag`、`rtag`。E 的 R1 指向 A，C 的 L1 也指向 A；两条线索各有独立的起点和箭头落点。D 的 L1 和 F 的 R1 则分别接空前驱、空后继端点，本图采用无头结点、首尾取空的约定。

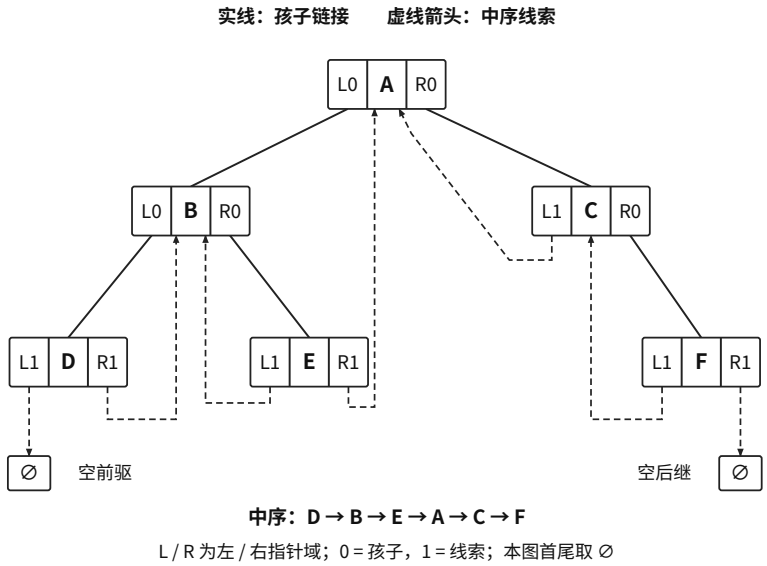



Figure 5.2: 虚线为中序线索，实线为孩子；首尾边界单独标出

$n+1$ 个原空孩子域均可改为线索域，其中首结点的前驱和末结点的后继需约定为空或指向专设头结点。题目说 “线索数为 $n+1$” 时通常统计这些线索域，包含首尾边界；若只统计实际指向普通结点的线索，则边界两项另算。加头结点后，其额外指针也不应混入原树的 $2n$ 域计数。

构造中序线索时，用 `pre` 保存刚访问过的结点，初值为空。处理当前结点 p 时，若其左域为空，填入 pre 并设置左线索标志；若 pre 的右域为空，填入 p 并设置右线索标志，随后令 pre=p。递归只沿真实孩子执行，避免把刚填的线索又当作子树。

```
thread(p):
    if p==NULL: return
    if p->ltag==0: thread(p->left)
    if p->left==NULL:
```

```
        p->left=pre; p->ltag=1
    if pre!=NULL and pre->right==NULL:
        pre->right=p; pre->rtag=1    pre=p
    if p->rtag==0: thread(p->right)
```

调用前所有原孩子标志为 0。图中先访问 D，其空左域指向空前驱；随后访问 B，D 的空右域才补为指向 B 的线索。访问 E 时填左线索指向 B，访问 A 时再把 E 的空右域补为 A。遍历结束后，pre 为最后结点 F，将其空右域标为后继线索，线索目标按约定为空或为头结点。若使用头结点，首结点空前驱也应统一改指头结点。

先序、后序线索的前驱后继依照各自次序，不可沿用中序规则。后序中，若叶结点 X 存在左兄弟 Y，则 X 是双亲的右孩子；左子树和右子树都处理完后才访问双亲，因此 X 的后序后继是双亲，而不是左兄弟。一般后序线索树若无双亲信息，不一定能像中序那样方便地只凭孩子和线索完成全部后继访问。

**课程连接·编码和外排使用同一棵树**

哈夫曼叶权可表示字符频数，也可表示外部归并段长。每次参加合并使该叶再负担一次权值，故总合并长度等于 WPL；第 8 章的最佳归并由此得到。

## 5.7 普通树的三种存储

**双亲表示法**为每个结点保存双亲的下标，根用 $-1$ 等特殊值标识。找双亲 $O(1)$，找全部孩子通常需扫描整个数组。它适合从结点反复向根追踪，却不便于列举孩子。

**孩子表示法**为每个结点保存孩子链表，便于依次访问所有孩子；反查双亲时若没有另外保存双亲信息，仍需搜索。**孩子兄弟表示法**只保存两个链接：`firstChild` 指向第一个孩子，`nextSibling` 指向下一个兄弟。它能用两个指针表示孩子数不定的树，却不能把 `nextSibling` 理解成普通树的孩子。

选择表示时，应先看主要方向：查双亲、枚举孩子或遍历同层兄弟，分别依赖不同的链接。多保存一个双亲域会增加空间，但上溯不必再搜索，代价降低。

## 5.8 树、森林怎样转换为二叉树

采用 “左孩子、右兄弟” 约定：普通树第一个孩子成为二叉树左孩子，后续兄弟依次以右孩子相连。原树的根没有兄弟，故其对应二叉树的根没有右孩子。森林的各棵树根互为兄弟，再沿右链连接，第一棵树的根成为二叉树的根。

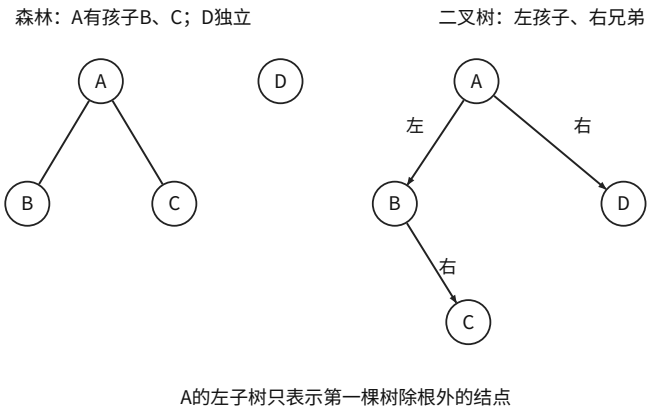



Figure 5.3: 普通树的孩子关系转成左孩子，兄弟关系转成右孩子

逆转换时，从某结点的左孩子出发沿右链走，沿途结点就是它的全部孩子；根及其右链上的各结点分别作为森林各棵树的根。转换改变了“左右链接的解释”，没有增加或删除数据结点。森林有 4 棵树、第一棵含 $a$ 个结点时，二叉树根的左子树只含第一棵树除根外的 $a-1$ 个结点，其他树由根的右子树表示。

**错项排除·最优不意味着唯一**

左右子树交换不会改变叶深和 WPL；同权候选的选择也可产生不同树。应排除 “不同编码必有不同平均码长”，同时避免把哈夫曼最优性扩大成完全或平衡。

### 5.8.1 例 5-4　二叉树隔两条链接，在森林中是什么关系

**例 5.8.1** (配套案例)

【配套例题】将森林转换成对应二叉树。若二叉树中 u 是 v 的父结点的父结点，则原森林中 u 与 v 可能的关系是（　）。

I. 父子关系　II. 兄弟关系　III. u 的父结点与 v 的父结点是兄弟关系

A. 只有 II　B. I 和 II　C. I 和 III　D. I、II 和 III

**解析**。从 u 到 v 恰经过两条二叉树链接。左—右表示 “先到 u 的第一个孩子，再到其下一个兄弟”，v 仍是 u 的孩子，I 可能；右—右表示连续两个兄弟链接，u、v 互为兄弟，II 可能。左—左表示祖孙，右—左表示 u 的兄弟的孩子，它们都不能产生 III 所说的父结点关系。因此答案为 **B**。应按每条链接的语义推回森林，不能把二叉树的双亲直接当作原树的双亲。

## 5.9 树与森林的遍历怎样对应

普通树的先根遍历先访问根，再依次先根遍历各子树；后根遍历先按顺序遍历各子树，再访问根。采用孩子兄弟转换后，树的先根次序对应二叉树先序，树的后根次序对应二叉树中序。原因是二叉树左链接深入第一个孩子，而右链接串联后续兄弟。

森林先序依次先根遍历各棵树，对应二叉树先序；森林中序先按中序处理第一棵树的子树森林，再访问该树根，最后处理其余树，对应二叉树中序。森林遍历要按定义递归到各棵树，并非在整片森林开头统一访问一次根。

层次遍历普通树时，出队一个结点后，按孩子链的次序把它的所有孩子入队；兄弟链接只用于枚举孩子，不能把整条兄弟链当成下一层的结点序列。

## 5.10 哈夫曼树、WPL 与前缀编码

给每个叶结点一个权值 $w_i$，其深度为 $d_i$，**带权路径长度**为 $\mathrm{WPL}=\sum_i w_i d_i$。权值通常为频数或概率，内部结点不直接加入这项 “叶权乘深度” 的求和。求给定叶权下最小 WPL 的二叉树，可每次选当前两个最小权值合并为新结点，再把权值和放回候选集合，直到仅剩根。

为什么合并两个最小值：在一棵最优树中，可以让最深的一对兄弟叶承载两个最小权值而不增大 WPL；把它们收缩成一个权值为和的叶后，剩余结构仍应最优，否则替换后能得到 WPL 更小的树。每次合并都使这两棵子树中所有叶的深度增加 1，因此合并权值之和也等于最终 WPL。

### 5.10.1 例 5-5　加权平均码长怎样从合并过程得到

**不变量·压缩路径不压缩集合**

路径压缩把沿途结点直接挂根，保留每个元素最终代表及集合划分。根的负规模只在合并两个不同根时累加；重复合并同一集合不能再增加规模。

**例 5.10.1** (配套案例)

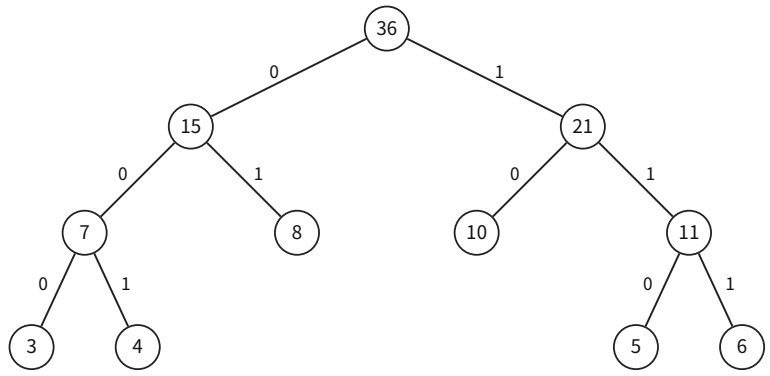
【配套例题】6 个字符出现频次分别为 3、4、5、6、8、10，为它们构造哈夫曼编码，加权平均长度为（　）。

A. 2.4　B. 2.5　C. 2.67　D. 2.75

**解析**。每轮重新从当前候选中取最小的两个，不能把原始序列分成相邻对后一次配完。

| 本轮合并 | 新权值 | 合并后候选权值 |
|---|---|---|
| 3 与 4 | 7 | 5、6、7、8、10 |
| 5 与 6 | 11 | 7、8、10、11 |
| 7 与 8 | 15 | 10、11、15 |
| 10 与 11 | 21 | 15、21 |
| 15 与 21 | 36 | 36 |

WPL 为 $7+11+15+21+36=90$，频次总和 36，所以平均码长为 $90/36=2.5$ 位/字符，答案为 **B**。不能把总码长除以 6，因为各字符出现频次不同。




WPL = 7+11+15+21+36 = 90；平均码长 90/36 = 2.5

Figure 5.4: 同一合并过程构造的哈夫曼树；左右分支的 0、1 可对换

沿根到叶的 0、1 路径形成码字。字符只放在叶上，所以任何字符码都不是另一个字符码的前缀，这称为**前缀编码**。编码可因相等权值的选取、左右分支交换而不唯一，WPL 仍可相同。哈夫曼树没有度为 1 的内部结点，若有 $n$ 个叶，内部结点 $n-1$、总结点 $2n-1$；它未必完全、未必平衡，$n$ 个叶的最大高度可达 $n$。

### 5.10.2 例 5-6　已占用的码字怎样限制剩余容量

**例 5.10.2** (配套案例)

〖5.5-004〗哈夫曼编码长度不超过 4，已有两个字符编码为 1 和 01，还最多能对几个字符编码?

A. 2　B. 3　C. 4　D. 5

**解析**。以 1 开头的整棵子树已被码字 1 占用，以 01 开头的子树也被占用。剩余码字只能位于 00 开头的子树。长度至多 4，00 之后最多

还放 2 位，最多形成 `0000`、`0001`、`0010`、`0011` 四个叶，答案为 **C**。使用较短的剩余码会占用其后代位置，不会增加字符总数。

推广到 $m$ 叉最优归并/编码时，每次合并最小的 $m$ 个权值。若叶数不满足满 $m$ 叉树的条件 $(n-1)\bmod(m-1)=0$，可补最少的零权虚叶。补齐后的叶数记 $n'$，内部结点数为 $(n'-1)/(m-1)=\lceil (n-1)/(m-1)\rceil$；不能直接把二叉的“内部结点 $n-1$”照搬。这里内部结点都按 m 叉合并计算，零权虚叶不代表额外字符。第 8 章最佳归并树会再次使用这项结构关系。

> **把哈夫曼的贪心选择写成可重复步骤**
>
> 候选森林初始为各带权叶；重复取两个最小权根，创建权值为两者之和的新根，并将它放回候选集合。每轮候选数减少 1，故 $n$ 个符号需 $n-1$ 轮。优先队列实现总时间 $O(n\log n)$，树与候选存储 $O(n)$；单符号编码的码长需另作约定。

```
huffman(weights):
    Q=以权值为键的小根优先队列
    while size(Q)>1:
        a=extractMin(Q); b=extractMin(Q)
        z=new Node(weight=a.weight+b.weight,left=a,right=b)
        insert(Q,z)
    return Q唯一根（若原输入为空则返回空）
```

## 5.11 并查集：用根结点代表一个集合

并查集维护若干互不相交的集合，主要支持两个操作：`find(x)` 查 x 所在集合的代表元，`union(x,y)` 合并两个集合。它不要求维护集合内部元素的顺序。用数组 `parent` 表示一片双亲森林，根存负数表示集合规模，非根存双亲下标。

例如初始 6 个单元素集合：`parent=[-1,-1,-1,-1,-1,-1]`。合并 0 与 1 后可记 `parent[0]=-2,parent[1]=0`；再合并 2 与 3 得 `parent[2]=-2,parent[3]=2`；把这两集合合并时，把根 2 接到根 0，根 0 规模记 $-4$。查找 3 时沿 $3\to2\to0$，不能把直接双亲 2 误当作最终代表。

**按规模合并**让较小树根接到较大树根，避免不断形成长链；**路径压缩**在查找后把沿途结点直接连到根。路径压缩不改变集合划分，只改变表示。两项同时使用时，连续操作的摊还复杂度极低，可用反阿克曼函数表示；单次查找仍须按当时实际路径执行，不能声称每次严格只访问一个结点。

```
find(x):
    if parent[x]<0: return x
    parent[x]=find(parent[x])
    return parent[x]

unite(x,y):
    rx=find(x); ry=find(y)
    if rx==ry: return
    若 parent[rx]>parent[ry] 则交换 rx、ry
    parent[rx]=parent[rx]+parent[ry]
    parent[ry]=rx
```

Kruskal 选边时，若两个端点代表元相同，加边会在现有森林中形

**错项排除·堆序没有兄弟顺序**

小根堆的根最小，两个孩子之间却没有规定。删除根时必须先选更优孩子，再和候选比较；只因左孩子更早出现在数组，就总往左走，会破坏右侧堆序。

成环；代表元不同才合并。这里代表的是当前已选边连成的分量，不是原图预先固定的一个编号。

**错项排除·堆序没有兄弟顺序**

小根堆的根最小，两个孩子之间却没有规定。删除根时必须先选更优孩子，再和候选比较；只因左孩子更早出现在数组，就总往左走，会破坏右侧堆序。

## 5.12 堆的结构、调整与优先队列

**二叉堆**同时满足完全二叉树的形状和堆序。大根堆中每个双亲关键字不小于孩子，小根堆中不大于孩子。于是根保存全局最大值或最小值；兄弟间没有规定，整个数组也不必有序。堆中沿根到叶的关键字单调，二叉搜索树则按左右子树区间有序，二者不是同一个约束。

本书堆用下标从 1 开始的数组。小根堆的最大值一定可以在叶中找到，叶下标为 $\lfloor n/2 \rfloor + 1 \sim n$；不能确定是哪一个叶。非根结点若比所有后代都小，仍不保证它比别的子树中的元素小。

插入时先放到数组末尾，保证完全性，再与双亲比较并向上调整。删除根时，用末元素填根、缩小堆范围，再从根向下调整：选择较优孩子，与待调整的值比较，必要时交换并继续。每次最多跨越树高，时间 $O(\log n)$。原有左右子树必须已是堆，下滤才只需沿一条路径。

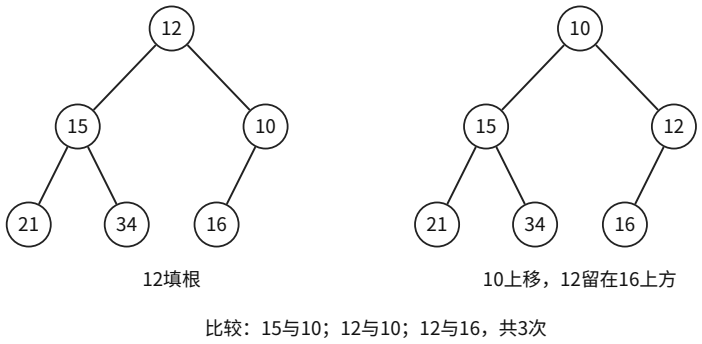



Figure 5.5: 小根堆删根后，用末元素 12 填根，再与更小孩子 10 比较并下滤

建堆从最后一个非叶结点 $\lfloor n/2 \rfloor$ 开始逆序下滤。叶子本身已是堆，靠下的内部结点移动距离短；按各高度结点数求和，建堆总时间为 $O(n)$，不是把每个结点都按最坏树高估成 $O(n \log n)$。逐个插入也能建堆，但对应另一种过程和复杂度上界。第 8 章将用同一调整规则执行完整堆排序。

### 5.12.1 堆下滤的代码与不变式

以下大根堆采用 1 起始数组。调用前，左右子树各自满足堆序，只可能根位置过小；每轮把较大孩子提升，尚未解决的缺口沿一条链向下移动。

```
void siftDown(int A[], int root, int n) {
    int x=A[root];
    for (int c=2*root; c<=n; c=2*root) {
        if (c<n && A[c]<A[c+1]) ++c;
        if (x>=A[c]) break;
        A[root]=A[c];
        root=c;
    }
    A[root]=x;
}
```

向下每轮至少下降一层，最坏 $O(\log n)$，辅助 $O(1)$。只比较候选与左孩子会漏掉更大的右孩子；`c<n` 是检查右孩子确实存在。空堆不应调用此函数，外部循环从 $n/2$ 开始可自然跳过 $n=0,1$ 的情况。

| 结构 | 约束保存什么 | 操作与判断边界 |
|---|---|---|
| 普通二叉树 | 左右位置有别 | 遍历还原需序列与身份条件 |
| 完全二叉树 | 编号占据前 $n$ 个位置 | 适合数组；只排除右孩子孤立还不够 |
| 二叉堆 | 完全形状及双亲孩子间堆序 | 根为最值；兄弟不保证有序 |
| BST（第 7 章） | 全部左右子树的关键字区间 | 中序有序；不保证完全或平衡 |
| 哈夫曼树 | 叶权下的 WPL 最小 | 通常不是完全树；编码可不唯一 |

**不变量·压缩路径不压缩集合**

路径压缩把沿途结点直接挂根，保留每个元素最终代表及集合划分。根的负规模只在合并两个不同根时累加；重复合并同一集合不能再增加规模。

## 5.13 主动回忆自测

以下为自拟补充，针对表示与前置条件。

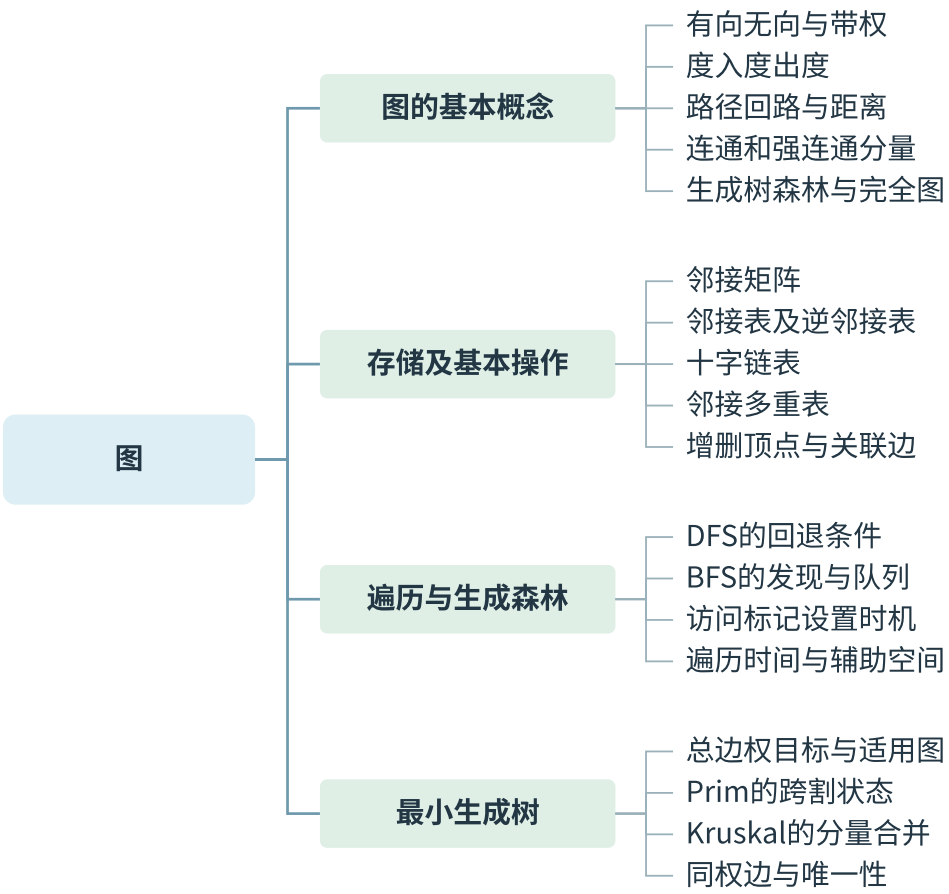
1. 一棵二叉树有 8 个度为 2 的结点、3 个度为 1 的结点，共多少结点？它是否一定为完全二叉树？
2. 一棵二叉树的先序和中序完全相同，结点值互异，所有非叶结点的孩子位置有什么限制？
3. 并查集中 `parent[4]=2,parent[2]=0,parent[0]=-5`，执行带路径压缩的 `find(4)` 后，哪些关系改变？
4. 关键字互不相同的大根堆中，第二大元素一定在哪些可能位置？
5. 为什么一棵 $n$ 结点二叉链表有 $n+1$ 个空孩子域？加上双亲域后怎样变化？
6. 普通树的后根遍历转换后对应二叉树哪一种遍历？为什么不是后序？
7. 权值 3、4、5 合并构造哈夫曼树，WPL 是多少？编码一定唯一吗？
8. 层序遍历与递归深度优先遍历分别由什么结构保存未处理状态？最坏辅助空间是否一定相同？

**答案**。

1. 由二叉树的性质 $n_0=n_2+1$，得叶结点数 $n_0=8+1=9$，所以总结点数为 $9+3+8=20$。这棵树**不是完全二叉树**：完全二叉树至多有 1 个度为 1 的结点，而题中有 3 个，不满足这一必要条件。
2. 只能有右孩子；若存在左子树，中序会先于根访问它，与先序矛盾。
3. 返回 0，`parent[4]` 改为 0，2 仍指向 0，根规模不变。
4. 根的孩子中；若某更深结点是第二大，其双亲严格大于它，除根外还会出现一个更大的结点，产生矛盾。存在相同值时，须先明确第二大是否按不同值计数，不能原样沿用结论。
5. $2n$ 个孩子域中 $n-1$ 个非空，余 $n+1$；再加 $n$ 个双亲域，除根外 $n-1$ 个非空，总空域 $n+2$。
6. 对应中序：左链递入孩子森林、右链枚举兄弟，全部孩子完成才访问本根；右孩子在转换语义中不是原树子女，不能按原二叉后序套用。
7. 先 3+4=7，再 5+7=12，WPL 为 7+12=19；左右 0/1 交换可改变编码而不改变 WPL。
8. 前者用队列、后者用调用栈；队列与宽度有关，调用栈与高度有关，形态不同可相差很大。

# 图 6

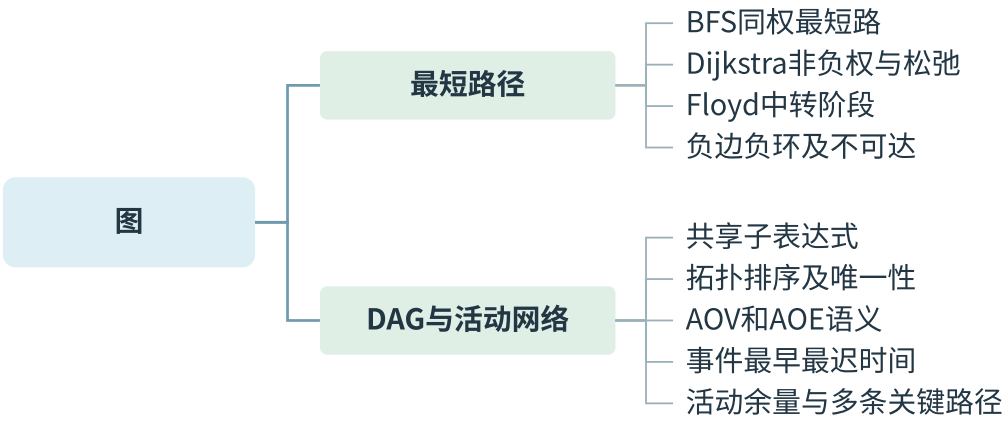
**知识导图**




**课程连接·网络拓扑只是图的一种模型**

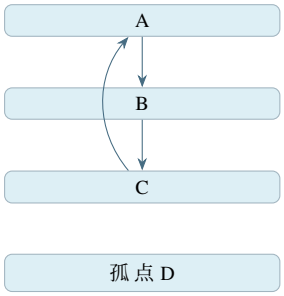
计算机网络可把路由器视为顶点、链路视为边，但边权可能表示时延、费用或带宽的变换。先明确权的含义再选择算法；同一个物理网络可以对应不同优化问题。

**知识导图（续）**




**最小反例·n 减 1 条边不够判树**

三点成环再加一个孤立点，共 4 顶点 3 边，恰有 $n-1$ 条边却不连通且有环。判树还要加连通或无环条件，不能只由数量推出结构。




4 顶点 3 边：三角形加孤点，满足 m=n−1 却不是树。

图的题目先辨认边的含义：一条边可以表示可通行的道路、需要铺设的线路，也可以表示任务之间的先后约束。最短路径、最小生成树、拓扑排序和关键路径分别优化或检查不同对象；选择算法前先把题目要求写成图上的条件。

**错项排除·零权边不是没有边**

带权邻接矩阵用无穷表示无边时，零是合法边权。若仍以 if(A[u][v]) 判断有边，就会漏掉零权边；AOV 的 0/1 邻接矩阵与这种表示的语义不同。

## 6.1 导图主干与阅读路径

- **图的基本概念**：有向无向与带权；度入度出度；路径回路与距离；连通和强连通分量；生成树森林与完全图。
- **存储及基本操作**：邻接矩阵；邻接表及逆邻接表；十字链表；邻接多重表；增删顶点与关联边。
- **遍历与生成森林**：DFS 的回退条件；BFS 的发现与队列；访问标记设置时机；遍历时间与辅助空间。
- **最小生成树**：总边权目标与适用图；Prim 的跨割状态；Kruskal 的分量合并；同权边与唯一性。
- **最短路径**：BFS 同权最短路；Dijkstra 非负权与松弛；Floyd 中转阶段；负边负环及不可达。
- **DAG 与活动网络**：共享子表达式；拓扑排序及唯一性；AOV 和 AOE 语义；事件最早最迟时间；活动余量与多条关键路径。

先按导图说清对象与条件，再沿相应正文执行算法或推导。章末先遮住答案回忆各主枝；不能只凭图中文字是否熟悉判断掌握。

## 6.2 顶点、边、连通性与路径

**定义 6.2.1** (图与本章模型)

图记为 $G=(V,E)$，$V$ 为顶点集合，$E$ 为边集合。无向边用无序顶点对表示，有向边（弧）用有序对表示。除另行说明，本章讨论无自环、无平行边的简单图，顶点数 $n=|V|>0$。无向简单图最多有 $n(n-1)/2$ 条边；有向简单图最多有 $n(n-1)$ 条弧。带权图的边还关联权值，权可以表示距离、费用或时间。

无向图顶点的度为关联边数，所有顶点度数和为 $2|E|$；有向图分别计入度、出度，各自总和都为 $|E|$。无向图度数为奇数的顶点个数必为偶数。路径长度若未加权，通常计边条数；带权长度计沿途权值和。简单路径不重复顶点，简单回路除首尾相同外不重复其他顶点。

无向图任意两顶点都有路径时称为连通图，其极大连通子图为连通分量；有向图中每对顶点相互可达时为强连通，极大强连通子图为强连通分量。单向可达不等于强连通。“极大” 指不能再加入原图其他顶点而保持相应性质，不是要求分量含全图最多的顶点。

连通无向图的**生成树**含全部顶点，连通且无环，因此有 $n-1$ 条边。它是极小连通子图：再删一边即不连通。生成树一般不是原图的连通分量，因为连通分量要保留该连通块原有的全部边，不能删到只剩一棵树。非连通图的每个分量各取一棵生成树，得到生成森林。

### 6.2.1 例 6-1 “有可能连通” 与 “保证连通” 不同

**例 6.2.1** (配套案例)

【配套例题】无向图有 7 个顶点，要保证图在任何情况下都连通，需要的边数最少是（ ）。

A. 6 B. 15 C. 16 D. 21

**解析**。6 条边可以组成一棵生成树，但也可以都集中在部分顶点上，不能保证连通。若图不连通，边数最多的分法是 6 个顶点组成完全图，剩 1 个孤立顶点，共 $\binom{6}{2} = 15$ 条边。超过 15 条就不可能仍不连通，故最少 16 条，答案为 **C**。

一般的 $n$ 顶点简单无向图，边数大于 $(n-1)(n-2)/2$ 可保证连通；边数至少 $n-1$ 只是连通的必要条件。若图有 $c$ 个连通分量，则每个分量至少要 “顶点数减 1” 条边，所以 $|E| \geq n - c$。若 $|E| < n-1$，必不连通；若 $|E| = n-1$，还需检查是否无环或连通，才能认定为树。

一张 $n$ 顶点、$n$ 条边的无向图必有环，因为无环森林最多有 $n-1$ 条边。但图有环并不保证连通，也不保证只有一个环。求给定边数下最多的连通分量数时，应让边尽量集中：例如 51 个顶点、21 条边，可在 7 个顶点上组成完全图，其余 44 个孤立，共 45 个分量；少于 7 个顶点最多 15 条边，无法容纳 21 条。

“完全图” 是边达到上限的图；“连通” 只要求路径存在。稠密与稀疏描述边数相对规模，具体选结构还要结合访问方式。对于 $n > 1$ 的强连通有向图，每个顶点入度、出度至少 1，所以至少 $n$ 条弧，有向环可达到下界；$n = 1$ 时零弧图也可按零长度路径视为强连通，不能忽略这个边界。

**有向树**常指恰有一个入度为 0 的根、其余顶点入度为 1、根沿弧可达全部顶点的有向无环图。仅仅有 $n-1$ 条弧还不够。图中两点的距离是最短路径长；不可达时记为无穷，不把 “没有路径” 误记为距离 0。

**不变量·一条边结点参与两条链**

邻接多重表让同一个无向边结点同时属于两个端点链。沿当前端点前进时选择其对应链接；这解释了为何可统一标记边，也解释了不能始终沿 ilink。

## 6.3 邻接矩阵、邻接表与基本操作

邻接矩阵 `A[u][v]` 表示是否存在 $u \to v$；带权时元素存放权值，用 $+\infty$ 表示不存在，不能把 0 既用作 “无边” 的标记，又用作零权边的权值。无向图矩阵对称，有向图不必。矩阵占 $\Theta(n^2)$ 空间，检查一条给定边为 $O(1)$，枚举某顶点邻边通常要扫整行 $O(n)$。

邻接表为每个顶点保存其邻接点链。无向图每条边通常出现两个表结点，有向图每条弧在出边表出现一次，空间均为 $\Theta(n + |E|)$。枚举邻边只访问实际表项，判断一条边是否存在则可能扫描相应链。逆邻接表以终点组织入边，方向与普通出边表相反。

下图同时用于说明存储表示与遍历。无向边为 `ab`、`ac`、`ae`、`be`、`cf`、`df`、`de`，各邻接表按字母升序排列。

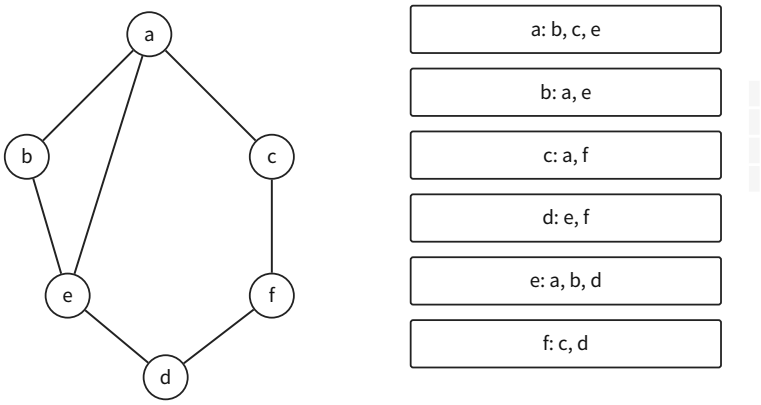



Figure 6.1: 本章遍历图；实线表示无向边，左侧表保存每个顶点的邻接点

对该图，a 行有 b、c、e 三个非零项，b 行有 a、e 两个。无向图中，顶点的度等于该行非零项的个数；有向图中，行的非零项数给出出度，列的非零项数给出入度。求有向图所有顶点的入度时，矩阵要扫描 $n^2$ 个位置，邻接表只需扫描所有表项，成本不同。

删除顶点会同时删除其所有关联边。无向邻接表不能只删除该顶点自己的链，还要清除其他顶点中指向它的表项；有向图则须同时考虑入边和出边。题目若给出 “只按出度计删除成本” 的实现，应检查它是否另有入边索引。

## 6.4 十字链表与邻接多重表

**十字链表**面向有向图，每条弧只建立一个弧结点，记录弧尾、弧头以及两个链接：一个沿相同弧尾串联出弧，一个沿相同弧头串联入弧。每个顶点分别保存第一条出弧、第一条入弧。这样同一条弧可从源顶点的出边链或终点的入边链到达，无需存两份弧数据。

**邻接多重表**面向无向图，每条边只建一个边结点，记录两个端点 `ivex,jvex`，并有 `ilink,jlink`，分别串入两个端点的关联边链。沿顶点 u 的链前进时，须先判断 u 是这条边的哪个端点，再选择对应链接；不能始终只走 `ilink`。这种表示便于给一条边设置统一访问标记、执行删除等边操作。

普通邻接表为每条无向边保存两个表项；邻接多重表中的一个边结点同时挂在两条链上。数边时若直接数链表项，两种结构得到的数目会相差常数倍。稀疏矩阵的十字链表也使用 “两种方向共享一个数据结点” 的思想，但它的两维是行与列，不是有向图的弧尾与弧头。

## 6.5 深度优先搜索与搜索森林

DFS 从一个顶点开始，标记并访问，再递归访问尚未访问的邻接点；当前顶点已没有未访问邻点时才回退。标记用于防止沿环重复访问。遍历整张图时，还须依次检查所有顶点，对仍未访问的顶点重新启动搜索；一趟 DFS 未必到达全图。

```
DFS(u):
```

不变 **DFS量**・何时才允许回

当前顶点仍有未访问邻点时必须深入，全部处理完才回退。输出相邻两点可以不直接相连，因为中间隐含了回退；仅检查输出相邻性会误排除合 DFS

```
    visited[u]=true
    visit(u)
    for v in u 的邻接点:        if not visited[v]: DFS(v)

traverse(G):
    全部 visited 初始化为 false
    for u in 全体顶点:
        if not visited[u]: DFS(u)
```

上图按字母顺序枚举邻点，从 a 开始，得到 `a,b,e,d,f,c`。首次到达新顶点的边形成 DFS 生成树；无向图的搜索树棵数等于连通分量数，有向图不能用树棵数直接认定强连通分量数。

### 6.5.1 例 6-2　什么时候允许回退去另一个分支

**例 6.5.1** (配套案例)

〖6.3-011〗无向图顶点为 `{a,b,c,d,e,f}`，边为 `{ab,ae,ac,be,cf,fd,ed}`。进行深度优先遍历，不能得到的序列为（　）。

A. acfdeb　B. aebdfc　C. aedfcb　D. abecdf

**解析**。D 访问到 `a,b,e` 时，e 的邻点 d 尚未访问，必须深入 d，不能先回退到 a 转去 c，因此 D 不可能。A 可沿 a→c→f→d→e→b；B 可先走 a→e→b，b 无未访问邻点后回到 e，再走 d→f→c；C 可先走 a→e→d→f→c，回退后再处理 b。答案为 **D**。输出序列中相邻的两个顶点不一定直接相连，因为中间可能发生回退；判断时真正要看的是，回退发生时当前顶点是否还有未探索的邻点。

采用邻接表时，每个顶点和表项只处理常数次，时间 $O(n+|E|)$；矩阵要对每个访问顶点扫一整行，时间 $O(n^2)$。递归栈最坏 $O(n)$，另有访问标志数组 $O(n)$。如果改为在递归返回前才输出顶点，得到的是完成次序；对 DAG，该完成次序的逆序就是拓扑序列，第 8 节会说明原因。

## 6.6 BFS 为什么得到无权最短路

BFS 用队列保存已发现、尚未扩展的顶点。起点先标记并入队；每次取出队头，将其尚未发现的邻点标记、记录距离并入队。标记应在**入队时**设置，若等到出队才标记，同一顶点可能被不同前驱重复入队。

```
BFS(s):
    dist[s]=0; visited[s]=true; enqueue(s)
    while 队列非空:
        u=dequeue()
        for v in u 的邻接点:
            if not visited[v]:
                visited[v]=true
                dist[v]=dist[u]+1; parent[v]=u
                enqueue(v)
```

**BFS 首次发现即最短**

队列保证按距离分层出队：距离 0 的顶点先于距离 1 的，距离 1 的又先于距离 2 的。首次发现 v 时，若还存在更短的路径，这条路径上 v 的前驱应位于更早的一层，早已被扩展，v 也就应更早被发现，矛盾。因此 `dist` 记录的是最少边数。

**最小反例·边最少而权更大**

三点 A、B、C，AB 与 BC 权 1，AC 权 3。BFS 从 A 一跳发现 C，少一条边却总权 3；经 B 两跳总权 2。因此 BFS 的分层证明不能覆盖不等权路径目标。

□

上图从 a 出发、邻点按字母枚举，出队访问序列为 `a,b,c,e,f,d`；b、c、e 距离 1，f、d 距离 2。该序列与 DFS 顺序不同，但也不唯一：同层邻接次序可改变，最少边数不变。带权不等长边时，少经过一条边未必总权更小，不能把这条证明直接推广为带权最短路。

邻接表与矩阵的时间同 DFS，队列和标记等辅助空间为 $O(n)$。有向图中，BFS 只沿弧的方向扩展；“无权最短路”并不要求图无向，但要求每条边的代价相同，只按边数计。

**把题意翻译成图算法的目标**

先确定有向还是无向、边权是否同一常数，再问目标是“全部顶点以最低总价接通”“某个起点到各点最短”“先后约束合法”还是“工程完成时间”。BFS 只保证同权边最短，MST 只控制整棵树总权；它们不能互相替代。

**错项排除·同权不等于 MST 必不唯一**

若所有边权互异，MST 一定唯一；逆命题不成立。一个本来就是树的连通图，即使边权相同，其生成树仍只有它本身，不能根据同权直接判不唯一。

## 6.7 Prim 与 Kruskal 在挑什么边

最小生成树（MST）要求连通全部顶点、无环，并使所选 $n-1$ 条边权和最小。它不要求从某个起点到每个顶点的路径最短。若图不连通，只能得到每个分量的最小生成森林。

Prim 维护一个已连接顶点集合 U，每次从“一端在 U、一端在外”的边中选最小边，加入新顶点。Kruskal 则把全部边按权非降序考虑，端点在不同当前分量时选入并合并，同一分量则跳过以避免成环。二者都利用割上的轻边性质，但状态不同：Prim 始终扩展一棵树，Kruskal 可能同时有多棵。

### 6.7.1 例 6-3　同权边怎样影响第二次选边

**例 6.7.1** (配套案例)

【配套例题】求下图最小生成树，可能为 Kruskal 第 2 次选中、但不可能为从 $V_4$ 开始的 Prim 第 2 次选中的边是（　）。

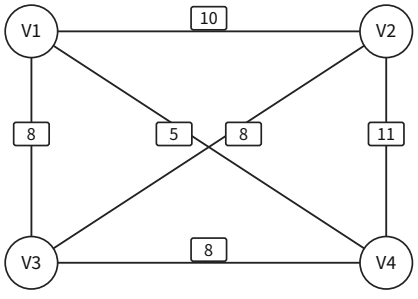



Figure 6.2: 四顶点带权无向图；三条权为 8 的边形成选择分支

A. $(V_1, V_3)$　B. $(V_1, V_4)$　C. $(V_2, V_3)$　D. $(V_3, V_4)$

**解析**。两个算法首先都选权 5 的 $(V_1, V_4)$。Kruskal 此后可选任意一条不成环的权 8 边，包括 $(V_2, V_3)$。Prim 此时 U 为 $\{V_1, V_4\}$，第 2 条

边必须跨越这个集合的边界，只能在 $(V_1, V_3)$ 与 $(V_4, V_3)$ 中选权 8 的边；$(V_2, V_3)$ 两端都在集合外，不能选。答案为 **C**。

继续执行时，Kruskal 若选 $(V_2, V_3)$，最后还要用一条权 8 边把两个分量连接，总权 21；Prim 选入 V3 后，再用 $(V_3, V_2)$ 权 8 连 V2，也是 21。同权边可能导致不同的 MST；全部边权互异足以保证 MST 唯一，但存在同权边时 MST 仍可能唯一。

朴素 Prim 配矩阵为 $O(n^2)$；邻接表配优先队列可降低稀疏图中选最小边的代价。Kruskal 主要花在边排序，常写 $O(|E| \log |E|)$，加并查集维护分量。具体选哪一个，还要看图的表示与选边的实现，不能只比较算法名字。

**不变量·Prim 的 key 不是距离**

Prim 只问一条跨割边接新点的代价，key 更新取边权；Dijkstra 问源点到新点的完整路径，需加已确定前缀。把两者公式混写可能算出另一种对象。

### 6.7.2 最小生成树的核心伪码

```
Prim(s):
    U={s}; key[v]=w(s,v)，无边记无穷
    重复直到选入全部顶点:
        u=U外key最小者
        若key[u]=无穷: 原图不连通，停止
        选入parent[u]到u的边; U加入u
        对每个U外邻点v:
            若w(u,v)<key[v]: 更新key[v]及parent[v]
```

这里 `key[v]` 始终是 U 到 v 的最轻跨割边，**不是**从 s 到 v 的路径长度，所以更新式没有 `key[u]+w(u,v)`。

```
Kruskal(G):
    将边按权非降排序；每个顶点初始化为单独集合
    对每条边(u,v,w):
        若find(u)!=find(v):
            选入此边; unite(u,v)
    若选入边数<n-1: 得到最小生成森林而非生成树
```

**为何跨割最轻边可以安全加入**

设已有选择能扩展到某棵 MST，考虑与现有分量相容的割及其一条轻边 $e$。若某 MST 不含 $e$，加入 $e$ 形成环，环上必有另一条跨该割的边 $f$，且 $w(e) \le w(f)$。删 $f$ 保连通且不增权，得到含 $e$ 的 MST；这说明选择可扩展，不表示所有 MST 都必须含 $e$。

□

## 6.8 Dijkstra 与 Floyd 的条件和中间状态

Dijkstra 求单源最短路径。维护已确定集合 S 与暂定距离 `dist[v]`；每次选择 S 外距离最小的顶点 u，确定它的最短距离，再用 `dist[u]+w(u,v)` 松弛其邻点。尚未确定的 `dist` 是已知可达路径的最好上界，不一定已是真正最短距离。

通用正确性要求边权非负：若能先经过一个当前距离更远的顶点、再经负边绕回 u，u 的真实距离就可能小于已确定的值，于是 u 被过早确定。出现负边不表示每个实例都必然算错，但标准算法不能保证正确。不可达顶点的距离保持无穷，所有剩余距离都为无穷时可停止，不能再把无穷当作普通数参与加法。

### 6.8.1 例 6-4　求出第二条最短路径后哪些值还可变

**不变量·已确定与暂定要分开**

Dijkstra 中暂定 dist 只是一条已知路径的最好上界。选出最小暂定点后才永久确定；过程题若提前使用未来松弛的值，就把最终答案错误放入了中间状态。

**例 6.8.1** (配套案例)

【配套例题】使用 Dijkstra 求下图从顶点 1 到其余顶点的最短路径，`dist` 依次保存到 2、3、4、5 的距离。求出第二条最短路径后，`dist` 内容为（　）。

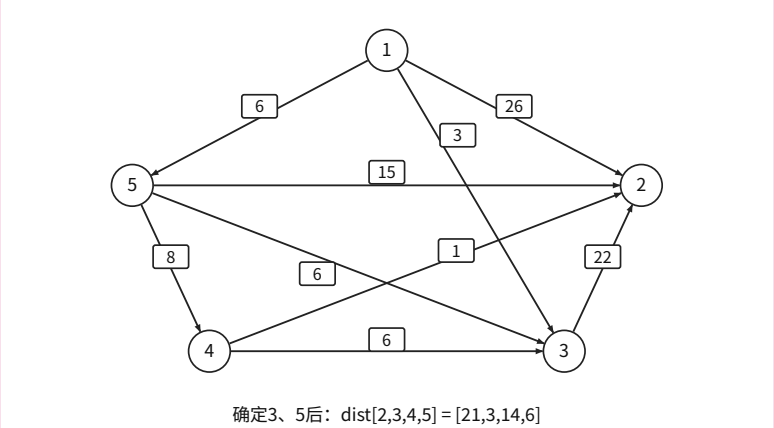



Figure 6.3: 原题有向边与权值；箭头方向参与松弛条件

A. 26、3、14、6　B. 25、3、14、6　C. 21、3、14、6　D. 15、3、14、6

**解析**。先把起点 1 确定，初始 `dist=(26,3,∞,6)`。第 1 条到其他顶点的确定路径是 1→3，长 3；用 3→2 的 22 更新到 2 距离为 25。随后选顶点 5，距离 6，第 2 条路径确定；由 5→2 的 15 把到 2 更新为 21，由 5→4 的 8 把到 4 更新为 14，得到 **(21,3,14,6)，答案 C**。

| 已确定的新顶点 | 暂定距离向量 (dist[2],dist[3],dist[4],dist[5]) |
|---|---|
| 1（初始化） | (26,3,∞,6) |
| 3 | (25,3,∞,6) |
| 5 | (21,3,14,6) |
| 4 | (15,3,14,6) |
| 2 | (15,3,14,6) |

答案要求的是第 2 条路径确定后，不是算法最终结果。顶点 4 随后才用其到 2 的权 1 把 21 改为 15，不能把这个将来更新提前放进答案。

Floyd 求任意顶点对之间的最短路。令 $D^{(k)}[i,j]$ 表示只允许编号不超过 k 的顶点作为内部中转时的最短距离，则

$$D^{(k)}[i,j] = \min\{D^{(k-1)}[i,j],\, D^{(k-1)}[i,k] + D^{(k-1)}[k,j]\}.$$

两种候选分别是不经过 k，或在 k 处分成两段。实现时中转顶点 k 必须放在最外层循环，才能对应这个阶段的含义。时间 $O(n^3)$，距离矩阵 $O(n^2)$；允许负边，但要求不存在影响所求路径的负权回路，否则不存在有限最短值。无向图中一条负权边就能往返构成负权闭合游走，不能直接套用通常的有限最短路模型。

矩阵版 Dijkstra 单源时间 $O(n^2)$；对每个源点重跑得到全源需 $O(n^3)$，不能仍写成 $O(n^2)$。BFS 求的是同权边下的最少边数，MST 最小化整棵树的边权总和，Floyd/Dijkstra 最小化指定端点之间的路径权和；三个目标要分别辨认。

### 6.8.2 最短路径的阶段循环

**课程连接·同步前驱约束就是偏序**

操作系统中多个事件的前驱关系可用 DAG 表示，零入度事件代表当前所有前驱已完成。拓扑排序只给合法先后，不计算耗时；带持续时间的工期还需 AOE 时间传播。

```
Dijkstra(s):
    dist[s]=0; 其余为无穷; S为空
    重复选择S外dist最小的u:
        若dist[u]=无穷: 停止
        S加入u
        对每条u到v，若v未确定且dist[u]+w(u,v)<dist[v]:
            更新dist[v]; parent[v]=u
```

最外层每次永久确定一个顶点，这是非负权前提的用处；已经确定的点不能靠未来负边回头改写。实现中先检查 `dist[u]` 有限，避免无穷哨兵加权溢出。

```
Floyd(D):
    D[i][i]=0; 其他位置按直接边初始化
    for k=0..n-1:
        for i=0..n-1:
            for j=0..n-1:
                若D[i][k]与D[k][j]均有限:
                    D[i][j]=min(D[i][j],D[i][k]+D[k][j])
```

k 在外层时，“已允许的中间顶点集合”才依次扩大。若结束后某 `D[v][v]<0`，说明存在负环；受该负环影响的点对没有有限最短距离，不能继续解释该数值为答案。

## 6.9 拓扑排序：由入度判断哪些任务可以先做

有向无环图（DAG）可以用顶点表示活动、弧表示先后约束，称为 AOV 网。拓扑序列要求对每条弧 $u \to v$，u 都排在 v 之前。算法先把所有入度为 0 的顶点作为候选，每次删除一个候选及其出弧，更新后继入度；若处理完的顶点数不足 n，则存在有向环。

有多个候选时可以选择不同的顺序，因此拓扑序列一般不唯一。对 DAG 做 DFS 并在退出时输出，每条弧的终点必先完成，得到的是逆拓扑序列；把该完成序列反向才是拓扑序列。若存在有向环，仅取得一个 DFS 完成次序并不能证明拓扑合法。

### 6.9.1 例 6-5　怎样判定唯一拓扑序列

**例 6.9.1** (配套案例)

【配套例题】载人航天工程包含众多子工程，为保证有序开展，需要明确前导子工程。这可抽象为有向图的拓扑序列问题。已知图 G 采用邻接矩阵存储：

```
typedef struct {
    int numVertices,numEdges;
    char VerticesList[MAXV];
    int Edge[MAXV][MAXV];
} MGraph;
```

设计 `int uniquely(MGraph G)` 判定 G 是否存在唯一拓扑序列，是则返回 1，否则返回 0。给出设计思想及带关键注释的 C

或 C++ 实现。

**解析**。每一轮的候选零入度顶点必须恰有一个。若图无环且某轮至少有两个，二者都可作为下一项，交换其先后可产生至少两个拓扑序列；若某轮没有候选而尚未处理完，则余图有环，无法完成。这两种情况都不满足唯一，均可返回 0。只有始终唯一且处理完全体顶点，才能返回 1。

```
int uniquely(MGraph G) {
    int n=G.numVertices, indeg[MAXV]={0};
    bool used[MAXV]={false};
    for (int u=0; u<n; ++u)
        for (int v=0; v<n; ++v)
            if (G.Edge[u][v]) ++indeg[v];
    for (int step=0; step<n; ++step) {
        int pick=-1;
        for (int v=0; v<n; ++v)
            if (!used[v] && indeg[v]==0) {
                if (pick!=-1) return 0; // 至少两个候选
                pick=v;
            }
        if (pick==-1) return 0;         // 剩余部分有环
        used[pick]=true;
        for (int v=0; v<n; ++v)
            if (G.Edge[pick][v]) --indeg[v];
    }
    return 1;
}
```

这里 AOV 邻接矩阵用 0/1 分别表示无边与有弧，不能直接把带权矩阵的“无边为无穷”搬进这个判断。总时间 $O(n^2)$，工作数组额外占 $O(n)$；题给的按值传参若实际复制整个矩阵，形参副本还占 $O(n^2)$，可按接口许可改为只读引用。若使用邻接表并维护零入度候选数，处理时间可为 $O(n+|E|)$。顶点已经按某个拓扑次序编号时，所有弧才保证由小编号指向大编号，邻接矩阵才呈严格上三角；任意编号的 DAG 不必原样为三角矩阵。

DAG 还可表达共享子表达式：相同子式只存一份，不同上层结点可指向同一子式。它表达计算依赖而非普通树的唯一双亲关系；不能再套用“除根外每结点恰有一个双亲”。

### 6.9.2 一般拓扑排序的队列实现

```
topological(G):
    统计所有顶点的入度；把入度0者入队Q
    count=0
    while Q非空:
        u=出队；输出u；count++
        对每条u到v：indeg[v]--
            若indeg[v]==0：v入队
    返回count==n
```

入度表示余图中尚未删除的前驱数，只有从 1 降到 0 时加入候选。若只关心任意拓扑序列，可以任选候选；要判断唯一，则每一步候选数都必须恰为 1，原例的代码正是在增加这个约束。

**错项排除·缩短一条关键活动未必够**

存在两条等长关键路径时，只缩短其中一条独有活动，另一条仍保持原工期。要检查所有当前关键路径是否都被影响，并在缩短后重新计算新关键路径。

## 6.10 关键路径：事件与活动的时间不同

**错项排除·有负边与一定算错不同**

负边意味着标准 Dijkstra 失去通用正确性保证，不代表每个含负边实例都会得错值。选项若用“只要有一条负边就必算错”的绝对语气，可用简单单路径反驳。

AOE 网用顶点表示事件、带权有向边表示活动，边权为持续时间。某事件要等全部进入活动都完成才能发生；它发生之后，其出边活动才能开始。以下假设该网无环、持续时间非负，且有统一的源点和汇点；若源点或汇点不止一个，可添加零时长的虚边统一。

事件最早发生时间 `ve[v]` 按拓扑序计算：从所有前驱中取 `ve[u]+w(u,v)` 的**最大值**，因为全部前导活动都得完成。令汇点的最迟发生时间等于总工期，事件的最迟发生时间 `vl[u]` 则按逆拓扑序计算，取所有后继的 `vl[v]-w(u,v)` 的**最小值**，以保证每条后续活动仍赶得上。

活动 $u \to v$ 的最早开始时间为 $e = ve[u]$，最迟开始时间为 $l = vl[v] - w(u,v)$。$l - e$ 是该活动的时间余量；余量 0 的活动为关键活动。关键路径是源到汇的一条最长带权路径，决定最短可完成工期；它不是边最多的路径。

### 6.10.1 例 6-6　多条关键路径时怎样缩短工期

**例 6.10.1** (配套案例)

【配套例题】下图 AOE 网包含 8 个活动。通过同时加快若干活动，可缩短整个工程工期。下列选项中，加快所列活动进度就可缩短工期的是（　）。

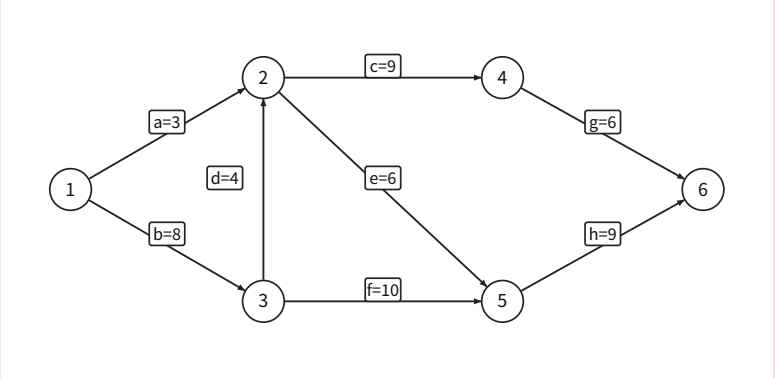



Figure 6.4: 箭头旁给出活动名和持续时间；d 的方向为 3→2

A. c 和 e　　B. d 和 c　　C. f 和 d　　D. f 和 h

**解析**。先按拓扑次序 1、3、2、4、5、6 算事件时间。

| 事件 | ve | vl |
|---|---|---|
| 1 | 0 | 0 |
| 2 | 12 | 12 |
| 3 | 8 | 8 |
| 4 | 21 | 21 |
| 5 | 18 | 18 |
| 6 | 27 | 27 |

例如 `ve[2]=max(0+3,8+4)=12`，不能仅沿 1→2 算成 3；`ve[5]=max(12+6,8+10)=18`。从汇点反推，`vl[2]=min(21−9,18−6)=12`，同时约束它通往 4 和 5 的两条后续活动。

活动 a 余量为 9，其余 b、c、d、e、f、g、h 均为关键活动。关键路径有三条：`b-d-c-g`、`b-d-e-h`、`b-f-h`，长度均 27。若只缩短其中一

条路径上的活动，其他路径仍可能保持 27。f 与 d 合起来覆盖三条关键路径：d 影响前两条，f 影响第三条；两者同时适当缩短，所有原关键路径都会变短。答案为 **C**。A、B 遗漏路径 b-f-h，D 遗漏路径 b-d-c-g。

增加关键活动时长会使包含它的路径变长，进而延长工期；缩短某一关键活动却未必缩短工期，因为还可能有不包含它的其他关键路径。持续缩短还可能使原非关键路径成为新的关键路径，必须重新计算，不能无限按原余量推断。

**错项排除·缩短一条关键活动未必够**

存在两条等长关键路径时，只缩短其中一条独有活动，另一条仍保持原工期。要检查所有当前关键路径是否都被影响，并在缩短后重新计算新关键路径。

### 6.10.2 关键路径的计算表怎样落到程序

```
criticalPath(G):
    取得拓扑序T；若不存在则模型非法
    ve[*]=0
    按T正序，对每条u到v：ve[v]=max(ve[v],ve[u]+w(u,v))
    D=ve[统一汇点]；vl[*]=D
    按T逆序，对每条u到v：vl[u]=min(vl[u],vl[v]-w(u,v))
    对每条活动u到v:
        e=ve[u]；l=vl[v]-w(u,v)
        若e==l：该边为关键活动
```

按 “全部前驱都完成” 取最大最早时间，按 “不拖延任何后继” 取最小最迟时间，方向不能交换。不能将 `ve[v]` 直接当作边 $u \to v$ 的活动最早开始时间，活动开始由弧尾事件 $u$ 决定。

| 任务 | 前提与优化对象 | 核心状态 | 典型边界 |
|---|---|---|---|
| BFS 最短路 | 同权边；最少边数 | 分层队列和距离 | 非等权不能直接用 |
| MST | 带权无向图；总树权最小 | Prim 跨割值或 Kruskal 分量 | 不保证源点路径最短 |
| Dijkstra | 非负权；单源路径和最小 | 已确定集合与暂定上界 | 负边破坏通用保证 |
| Floyd | 无相关负环；点对最短 | 允许的中转点集合 | 中转点循环在最外层 |
| 拓扑 | DAG；满足先后约束 | 余图入度 | 无候选而未结束表示有环 |
| 关键路径 | AOE DAG；总工期 | 事件 ve/vl 及活动 e/l | 多条关键路径需同时考虑 |

## 6.11 主动回忆自测

以下为自拟补充，检查算法选择的边界。

1. 图有 8 个顶点、7 条边，能否直接认定为树？若再给 “连通” 呢？
2. 有向图 A→B、A→C、B→D、C→D 有几个拓扑序列？若添加 B→C，怎样变化？
3. 一条边权变大，MST 总权与某一对顶点最短距离是否一定同时改变？
4. 一条关键活动缩短 1，工期是否必缩短 1？
5. 无向图三点 A、B、C，边 AB=1、AC=3、BC=1。从 A 作 BFS 得到的到 C 路径是否必为最短权路径？
6. Prim 的 `key[v]` 与 Dijkstra 的 `dist[v]` 分别保存什么？为什么松弛公式不同？
7. Floyd 把 `k` 放内层，仅执行一次三重循环还能直接沿用原证明吗？
8. DAG 中某轮有两个零入度顶点，能否仍有唯一拓扑序列？

**答案**。1. 不能；若无向且连通，7 条边恰够生成树，不可能再有环，故为树。2. 原有 A-B-C-D 与 A-C-B-D 两个；添加后仅 A-B-C-D。3. 不一定；该边可能不在任何当前采用的最优解中，两种优化目标也不同。

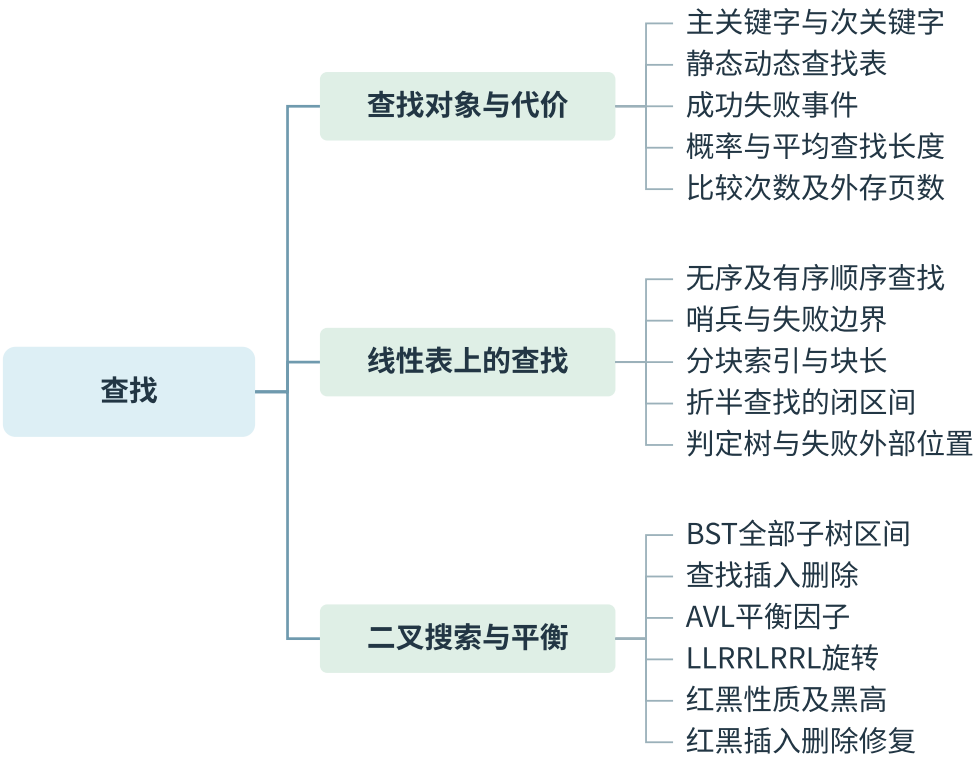
4. 不一定；有另一条不含该活动的同长关键路径时工期不变。
5. 不能；BFS 可一条边直接到 C，权为 3，实际 A-B-C 权为 2。它最少的是边数，不是总权。
6. 前者是连接现有树集合到 v 的最轻边；后者是从固定源点到 v 的已知路径最小上界，后者需累加已确定前缀距离。
7. 不能，阶段已不再对应统一的中转集合；必须重新证明算法，而非只因循环次数相同就认定等价。
8. 不能；在无环余图中这两个候选可分别作为下一项，并可扩展为合法序列，至少产生两种拓扑序。

**课程连接·同步前驱约束就是偏序**

操作系统中多个事件的前驱关系可用 DAG 表示，零入度事件代表当前所有前驱已完成。拓扑排序只给合法先后，不计算耗时；带持续时间的工期还需 AOE 时间传播。

# 查找 7

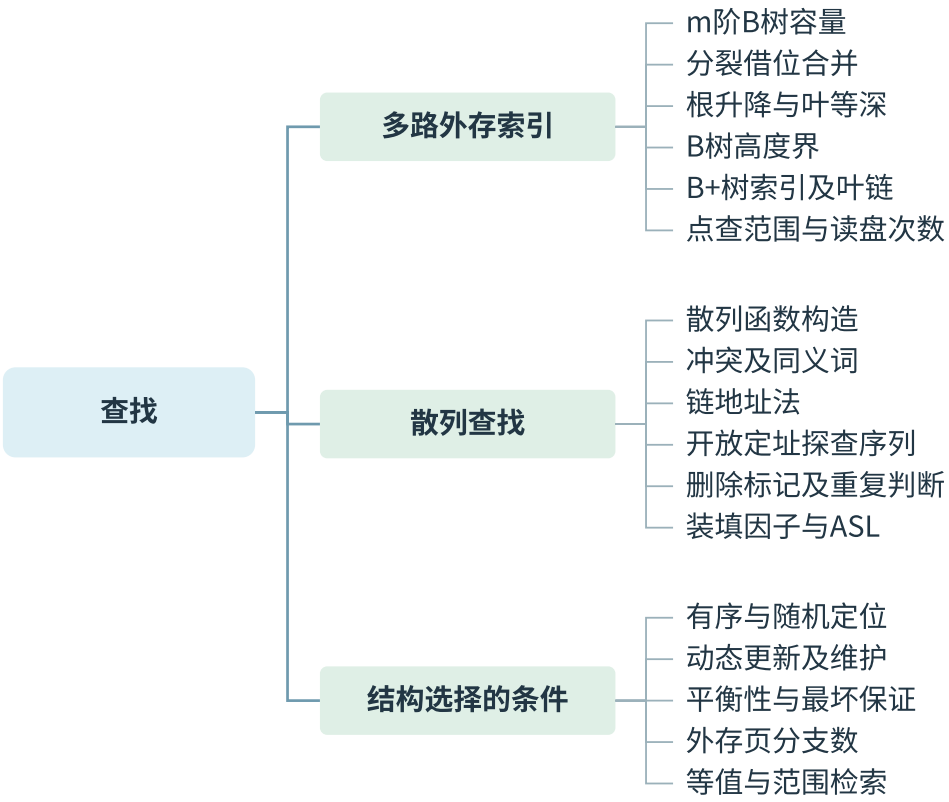
**知识导图**




**错项排除·先决定 ASL 的事件空间**

成功概率分配给已有记录，失败概率可能分配给关键字间隙或初始散列地址。先写清每项概率再计平均；不能看到有效记录有 n 个，就把所有失败长度都除以 n。

**知识导图（续）**




**不变量·mid 比较后必须排除**

闭区间中，相等已处理后，目标只能在 mid 左边或右边。若更新 high=mid 且区间仅两个元素，可能反复取得同一 mid 而不缩小；循环条件与更新式必须配套。

查找结构要回答两个问题：怎样缩小候选范围，插入或删除后怎样保持这种缩小范围的能力。顺序查找只依赖逐项比较；折半查找依赖顺序存储且有序；搜索树把有序关系放进分支；散列表则用关键字计算位置。外存索引还要考虑一次读入能取得多少个关键字。

**课程连接·比较少不代表访存少**

折半查找在数组上可常数时间定位中点；链表上虽能设计相同的比较决策，走到中点还要访问中间结点。算法分析要把抽象比较与实际存储访问分开。

## 7.1 导图主干与阅读路径

- **查找对象与代价**：主关键字与次关键字；静态动态查找表；成功失败事件；概率与平均查找长度；比较次数及外存页数。
- **线性表上的查找**：无序及有序顺序查找；哨兵与失败边界；分块索引与块长；折半查找的闭区间；判定树与失败外部位置。
- **二叉搜索与平衡**：BST 全部子树区间；查找插入删除；AVL 平衡因子；LLRRLRRL 旋转；红黑性质及黑高；红黑插入删除修复。
- **多路外存索引**：m 阶 B 树容量；分裂借位合并；根升降与叶等深；B 树高度界；B+ 树索引及叶链；点查范围与读盘次数。
- **散列查找**：散列函数构造；冲突及同义词；链地址法；开放定址探查序列；删除标记及重复判断；装填因子与 ASL。
- **结构选择的条件**：有序与随机定位；动态更新及维护；平衡性与最坏保证；外存页分支数；等值与范围检索。

先按导图说清对象与条件，再沿相应正文执行算法或推导。章末先遮住答案回忆各主枝；不能只凭图中文字是否熟悉判断掌握。

## 7.2 关键字、成功失败与平均查找长度

**定义 7.2.1** (查找表、关键字与动态性)

查找表是同一类型记录的集合，**关键字**是用于识别记录的数据项。主关键字唯一标识记录，次关键字可能对应多条记录。只进行查找、不改变记录集合的表称为静态查找表；还需要插入、删除记录的表为动态查找表，不能仅凭 “采用数组” 判断是静态还是动态。

**查找长度**通常指一次查找中关键字的比较次数，平均查找长度为 $\mathrm{ASL}=\sum_i p_i C_i$，其中 $p_i$ 为某一查找事件的概率，$C_i$ 为该事件的比较次数。成功 ASL 在表内各条记录之间分配概率；失败 ASL 在不存在的关键字可能落入的失败区间或初始散列地址之间分配概率，分母未必是表长。

树形查找中，访问一个含多关键字的 B 树结点可能只触发一次外存读，却在内存中比较多个关键字。因此 “读盘次数”“访问结点数”“关键字比较次数” 不是同一个量，题目要求哪个就统计哪个。

## 7.3 顺序查找、分块查找与提前结束

无序表从一端逐项比较，成功时停止，扫描完仍未找到则失败。等概率成功时，平均比较 $(n+1)/2$ 次；失败通常比较 n 次。哨兵可以统一循环终止条件、减少边界判断，但不会改变比较次数的数量级。若计入哨兵自身的关键字比较，次数还须按代码约定加上相应一项。

递增有序表从头查找时，当前值等于目标时应报告成功，当前值**大于**目标才可提前失败；把 “不小于” 一概判为失败会把相等的目标漏掉。

若仍逐个向后比较，等概率成功 ASL 与无序表相同，只是失败可能提前结束；有序并不自动意味着每种查找都使用折半。

分块查找把表分成若干块，块内可无序，块间有序：前一块的最大关键字小于后一块所有关键字。索引保存每块最大关键字和块起始位置，先定位块，再在块内顺序找。索引查找若分别采用顺序查找与折半查找，效率并不相同。

### 7.3.1 例 7-1　两级顺序查找怎样选择块长

**例 7.3.1** (配套案例)

【配套例题】查找表有 400 个元素，查找每个元素的概率相同，采用均匀分块。先顺序查找确定所在块，块内也顺序查找。为使查找效率最高，每块包含元素个数应为（　）。

A. 8　B. 10　C. 20　D. 25

**解析**。设每块 s 个，共 $b = 400/s$ 块。查成功时，索引平均比较 $(b+1)/2$ 次，块内平均比较 $(s+1)/2$ 次，总计

$$\mathrm{ASL} = \frac{400/s + s}{2} + 1.$$

两项乘积固定，$400/s + s$ 在 $s = \sqrt{400} = 20$ 时最小，答案为 **C**。四个选项均能整除 400，无需处理尾块。若索引改为折半，这个 “平方根块长” 的推导不再对应实际代价。

## 7.4 折半查找与判定树

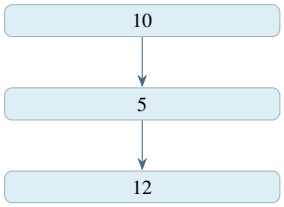
折半查找要求按关键字有序且能快速定位中点。本书在闭区间 `[low, high]` 中查找，取 `mid=low+(high-low)/2`，向下取整。目标比中点小时令 `high=mid-1`，比中点大时令 `low=mid+1`；中点已经比较过，更新区间时不能把它又留在候选范围内。

```
int binarySearch(int A[], int n, int key) {
    int low=0, high=n-1;
    while (low<=high) {
        int mid=low+(high-low)/2;
        if (A[mid]==key) return mid;
        if (A[mid]<key) low=mid+1;
        else high=mid-1;
    }
    return -1;
}
```

成功查找的比较次数等于对应内部结点的深度加 1。失败发生在候选区间为空时，空位置本身通常不做关键字比较，因此沿途比较次数等于经过的内部结点数。取上中点或下中点会改变具体路径，数量级仍是 $O(\log n)$。

**最小反例·孩子都合格，孙子越界**

根 10，左孩子 5，5 的右孩子 12。每个直接孩子比较都合格，但 12 位于 10 左子树而大于 10，违反全局区间；检验 BST 需中序或上下界递归。




10 的左子树里出现 12：直接父子比较仍可能全对。

### 7.4.1 例 7-2　失败查找的最大比较次数

**例 7.4.1** (配套案例)

【配套例题】长度为 16 的顺序表 L 按关键字有序，采用折半查找，查找一个不存在的元素，关键字比较次数最多为（　）。

A. 4　B. 5　C. 6　D. 7

**解析**。用从 1 开始的位序、向下取中点，查找一个大于全部元素的值：区间依次为 `[1,16]→[9,16]→[13,16]→[15,16]→[16,16]→ 空`，比较位置为 8、12、14、15、16，共 5 次，答案为 **B**。$\log_2 16 = 4$ 不是这里的最大比较次数；还需要最后一次区分第 16 个元素与它右侧的失败区间。

同一约定下，长度 11 时查第 11 个元素，比较位序为 6、9、10、11；长度 10、关键字依次 `b,c,d,e,f,g,q,r,s,t` 时查 b，比较 f、c、b。不能在不同题中改用向上取中点。

有序链表即使能逐步寻找中点，也没有数组的常数时间随机定位能力；普通折半查找的对数比较次数并不自动带来对数总访问时间。无序数组不满足排除半区的条件；静态链表虽然用数组存结点，逻辑顺序也不保证按下标连续。

**先写查找区间不变式再写边界**

闭区间折半查找循环开始时，若目标存在，它必在 `[low, high]`。比较过 `mid` 并排除相等后，下一区间必须去掉 `mid`；每轮区间严格缩小，`low>high` 才判失败。改为半开区间时，终止条件与更新式要一起改，不能从两份模板各抄一半。

**错项排除·删除传播与插入不同**

插入修复最小失衡子树后一般恢复原高度，更高祖先不再受本次增高影响。删除后局部修复可能继续降低高度，仍须向上；一处调整并非所有 AVL 更新的共同规则。

## 7.5 二叉搜索树的区间不变式

本书二叉搜索树（BST）默认关键字互异，左子树所有值小于根，右子树所有值大于根，左右子树分别满足同样性质。只检查根与两个直接孩子还不够；一个较深后代也可能越过祖先设定的区间。中序遍历严格递增是等价的整体判断。

查找从根出发，每次比较后进入相应子树；插入沿相同路径，到空位置新建叶结点。比较次数最多为树高，树可能退化成链，因此普通 BST 最坏为 $O(n)$，不能无条件写 $O(\log n)$。

删除叶结点直接断开双亲链接；只有一个孩子时让孩子接替位置；有两个孩子时，可把中序前驱（左子树最大值）或后继（右子树最小值）移到当前位置，再删除那个原结点。被选的前驱不会有右孩子，后继不会有左孩子，最后归结到前两种情况。若题目要求保留结点地址而非只保留关键字，那么复制数据与整体移接就是两种不同的实现，指针的改法也随之不同。

验证 BST 可以中序扫描并比较相邻值，也可以给每个结点传递允许区间 `(lower,upper)`。左递归把上界收紧到当前值，右递归把下界提高到当前值；这样不会漏掉“孩子合格但孙子越过祖先”的错误。当边界值可能等于整数类型极值时，用更宽类型或显式“有无边界”标记，不要随意把一个合法值当作无穷。

**定理 7.5.1** (BST 的中序判据)

关键字互异时，二叉树是递增 BST，当且仅当其中序遍历严格

递增。

**把全子树区间与中序顺序连接**

若为 BST，左子树各值小于根、右子树各值大于根，分别递归中序，整体严格递增。反之，中序严格递增使每个结点左子树的全部访问项均小于它，右子树全部项均大于它；递归对子树也成立，故满足 BST 的全局定义。证明涉及全部后代，不能简化为只比直接孩子。

□

**不变量·旋转保留中序次序**

右旋时 B 原右子树 T 的关键字介于 B 与 A 之间，转接为 A 左子树仍符合区间。若忘记保存 T，会丢失子树；若只交换数值，结点身份和其他链接语义也可能改变。

### 7.5.1 BST 查找、插入和删除的基本路径

```
search(root,key):
    p=root
    while p非空且p.key!=key:
        若key<p.key: p=p.left; 否则p=p.right
    return p

insert(root,key):
    沿同一比较路径找到空孩子位置
    若途中已有key: 按本章互异约定拒绝重复
    在空位置新建叶，接回父结点
```

删除的递归返回值表示 “删除后这棵子树的新根”，从而统一处理删除整树根与内部结点。

```
erase(t,key):
    if t为空: return 空
    if key<t.key: t.left=erase(t.left,key)
    else if key>t.key: t.right=erase(t.right,key)
    else:
        if t.left为空: 保存t.right，释放t，返回保存值
        if t.right为空: 保存t.left，释放t，返回保存值
        q=t.right; 沿q.left找到右子树最小结点
        t.key=q.key（记录有附加字段时一并复制）
        t.right=erase(t.right,q.key)
    return t
```

沿路每层常数工作，查找/插入/删除为 $O(h)$；递归删除额外栈 $O(h)$。这里复制记录后释放后继结点，不保留所有原结点地址；若题目约束结点身份，应改用移接方案。

## 7.6 AVL：先定位最小失衡子树

AVL 是每个结点左右子树高度差绝对值不超过 1 的 BST。令平衡因子为 “左高减右高”，合法值为 $-1$、0、1。插入后沿查找路径向上更新高度，找到离插入位置最近的失衡祖先 A，再根据新结点相对 A 及其较高孩子的位置分类。

LL（左孩子的左侧增高）对 A 右旋；RR 对 A 左旋；LR 先对 A 的左孩子左旋，再对 A 右旋；RL 先对 A 的右孩子右旋，再对 A 左旋。名称描述失衡路径，不能把 “LL” 误当作 “向左旋”。旋转必须保持中序次序：右旋时原左孩子 B 升为局部根，B 原来的右子树移成 A 的左子树，因为它的值介于 B 与 A 之间。

以下数值树用于观察旋转机制。插入前各树高度均为 3；灰底新结

点插入后，最小失衡子树高度升为 4。单旋图同时保留需要转接的中间子树：LL 中 40 由 30 的右子树改接为 50 的左子树，RR 中 40 的转接方向对称。

**错项排除·红叔要先变色**

红双亲配红叔时，旋转不是第一选择：把双亲与叔变黑、祖父变红，将冲突上移，再保证根黑。黑叔的内外侧旋转情形不能覆盖这个状态。

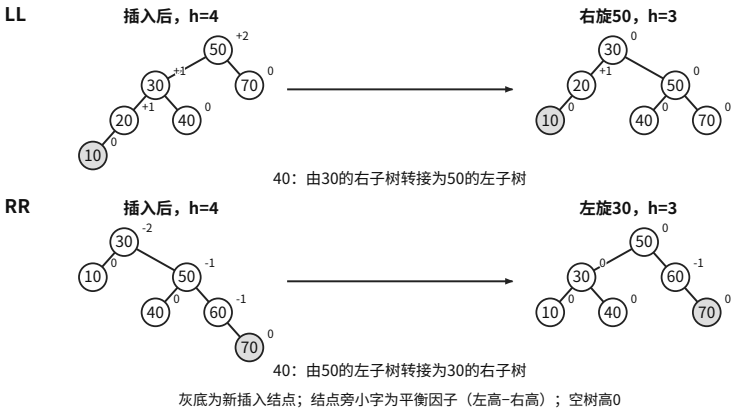



Figure 7.1: LL/RR 各一次旋转；结点旁为平衡因子，非空中间子树随旋转转接

LR 图在 70 的左孩子 30 的右侧插入 40，先左旋 30，再右旋 70；第一步把 40 转接为 30 的右孩子。RL 图在 50 的右孩子 90 的左侧插入 80，先右旋 90，再左旋 50；第一步把 80 转接为 90 的左孩子。**第一步后的树仍可能失衡**，双旋应检查完整两步，不能在中间态停下。四例都保持同一组关键字的升序中序，最终高度恢复为 3，所有平衡因子回到 −1、0、1。

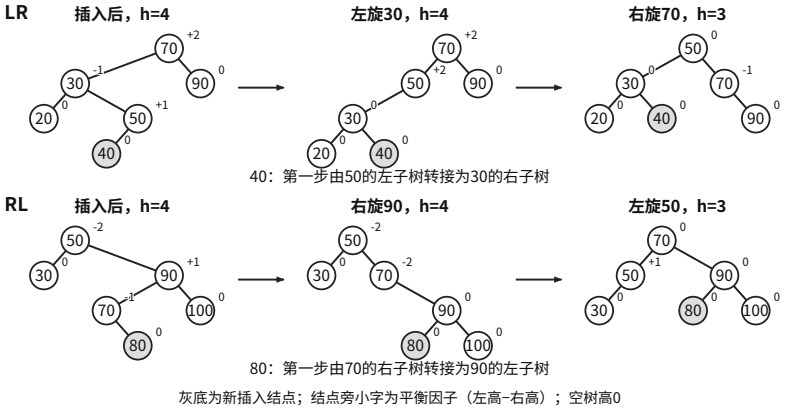



Figure 7.2: LR/RL 依次给出插入后、第一旋后和第二旋后三个状态

### 7.6.1 例 7-3　连续插入时在哪里旋转

**例 7.6.1** (配套案例)

〖7.3-003〗把 34、23、15、98、115、28、107 依次插入初始为空的平衡二叉搜索树，要求每次插入后保持 AVL 性质，给出每步结果。

**解析**。以下用 根（左子树,右子树）表示局部结构，⌀ 表示空。

| 插入 | 失衡处与调整 | 调整后整树 |
|---|---|---|
| 34 | 无 | 34 |
| 23 | 无 | 34(23,∅) |
| 15 | 34 处 LL，右旋 | 23(15,34) |
| 98 | 无 | 23(15,34(∅,98)) |
| 115 | 34 处 RR，左旋 | 23(15,98(34,115)) |
| 28 | 23 处 RL，先右旋 98，再左旋 23 | 34(23(15,28),98(∅,115)) |
| 107 | 98 处 RL，先右旋 115，再左旋 98 | 34(23(15,28),107(98,115)) |

插入 28 时，不能只旋转它的双亲 34：34 仍平衡，最先失衡的是 23，新增路径是 “23 的右—98 的左”，故为 RL。每次调整后复查中序始终为已插入关键字的升序，避免只把树形画平衡而破坏搜索关系。

最小失衡子树调整后通常恢复到插入前的高度，更高的祖先不会再因这次插入而失衡。删除则可能使子树高度继续缩短，需要一路向上检查，不能把插入时 “一处修复即可” 的结论直接用于删除。AVL 最小结点数满足 $N(h)=1+N(h-1)+N(h-2)$，由此树高是 $O(\log n)$，查找与更新路径才有对数保证。

固定高度下的形态数则是另一个量。按本书的规定，空树高 0、单根高 1，令 $C(h)$ 为恰高 h 的 AVL 形态数，$C(0)=C(1)=1$；恰高 2 的左右高度只能是 `(1,1)`、`(1,0)`、`(0,1)`，所以 $C(2)=3$。恰高 3 时，左右高度为 `(2,2)`、`(2,1)`、`(1,2)`，故 $C(3)=3^2+2\times 3\times 1=15$，对应〖7.3-013〗。这不等于固定结点数的卡特兰计数，也不等于最小结点数递推。

**不变量·B 树合法性有五项**

每次操作完成后同时检查结点内有序、孩子区间、键数上下限、孩子数、叶层深度。只画出平衡外形会漏掉关键字重复或孩子接错；临时溢出框不是最终合法状态。

### 7.6.2 例 7-4　中序为降序时搜索方向怎样变

**例 7.6.2** (配套案例)

【配套例题】一棵无重复关键字的 AVL 树，中序遍历得到降序序列。正确叙述是（　）。

A. 根的度一定为 2　　B. 最小元素一定是叶结点

C. 最后插入元素一定是叶结点　　D. 最大元素一定没有左子树

**解析**。本题的搜索方向与通常递增 BST 相反：左子树全大于根，右子树全小于根。最大值就是一直向左走到的那个结点，它没有左子树，答案为 **D**。AVL 约束不保证根一定有两个孩子；最小值结点可以有左孩子；新结点虽然最初作为叶插入，旋转后可能成为内部结点。不要用平衡性代替关键字的方向条件。

### 7.6.3 单旋代码为何必须保存中间子树

```
rotateRight(A):
    B=A.left; T=B.right
    B.right=A; A.left=T
    先更新A高度，再更新B高度
    return B

rotateLeft(A):
    B=A.right; T=B.left
    B.left=A; A.right=T
    先更新A高度，再更新B高度
    return B
```

调用者应把返回值接到原双亲对应链接；旋转整树根时更新根指

针。右旋中 $B < T < A$ 的区间关系保证 T 接到 A 左侧仍合法；只交换两个关键字不等同于链接旋转。

AVL 插入可在 BST 递归返回时逐层更新高度，若平衡因子超界，则按新增关键字落在重侧孩子哪一边选择 LL/RR/LR/RL。最少结点递推的边界为 $N(0) = 0, N(1) = 1$；删除时子树变矮可能继续向上传播，应沿整条祖先路径复查。

**最小反例·5 阶不能提前随意合并**

5 阶非根至少 2 键、最多 4 键。两个合法最小兄弟各 2 键，加父分隔项共 5 键，已经溢出；删除后某侧只剩 1 键，才能按 1+1+2=4 合并。阶数约定决定合并时机。

## 7.7 红黑树：颜色约束怎样限制高度

红黑树仍是 BST，另为结点标红或黑，采用以下约定：根为黑；所有空孩子视为黑色 NIL 叶；红结点的孩子均为黑；从任一结点到其所有后代 NIL 叶的路径，黑结点数相同。NIL 不是普通数据结点，画图时可以省略，但计算黑高时必须把它计入。

相邻红结点不允许，所以一条路径中红结点数不会超过相应黑结点数。本书把黑高 b 定义为：从当前结点向下到任一 NIL 的路径上，不计当前结点，但把终点 NIL 计入，统计所含的黑结点数；NIL 自身的黑高记 0。黑高为 b 的子树至少含 $2^b - 1$ 个内部结点。于是最长路径的长度至多约为最短路径的两倍，整体高度为 $O(\log n)$。它不要求每个结点左右高度差至多 1，比 AVL 的局部高度约束宽。

插入先按 BST 加入，通常把新结点染红，以免立即改变各路径黑结点数。双亲黑则完成；双亲红时，看祖父另一侧的叔结点：

- 叔为红：双亲与叔变黑、祖父变红，把问题上移到祖父，最后保证根黑。
- 叔为黑（含 NIL）：若形成内侧折线，先在双亲处旋转为外侧；再在祖父处反向旋转，并交换提升结点与原祖父的颜色，消除红红相邻。

例如黑根 10、红左孩子 5，插入红 1，叔为空黑，形成 LL：右旋 10 后 5 成为黑根，1 与 10 为红，黑路径数保持一致。若原来还存在红右孩子 15，则插入 1 先使 5、15 变黑，祖父 10 暂红，最后根 10 恢复黑；这个情境应变色，不应直接套前一个旋转。

删除时先按 BST 把待删结点化为至多只有一个非空孩子的情形。删红不减少路径黑高；删黑后，若有红孩子接替，可把接替孩子染黑。若替代位置是黑或 NIL，则沿该分支缺少一个黑结点，需要根据兄弟与其孩子颜色修复。

设缺黑位置 x 是双亲 p 的左孩子，兄弟 w 为 p 的右孩子。w 的左孩子为近侧、右孩子为远侧，均相对 x 而言；NIL 按黑处理。按表中的条件逐步转换，不能把 “红兄弟” 当作已经完成修复。

| 当前条件 | 操作与后续 |
|---|---|
| w 红 | w 染黑、p 染红，对 p 左旋；重新取 x 的兄弟，再处理下列黑兄弟情况 |
| w 黑、两个孩子都黑 | w 染红；若 p 红则 p 染黑并结束，否则将缺黑位置上移到 p 继续，移到根即可结束 |
| w 黑、远侧黑、近侧红 | 近侧染黑、w 染红，对 w 右旋；重新取兄弟，转成下一行 |
| w 黑、远侧红 | w 继承 p 原颜色，p 与远侧染黑，对 p 左旋，结束缺黑修复 |

缺黑在右侧时，左右及旋转方向对称互换。最后一行的最小例子可写成 `10 黑 (缺黑 NIL,20 黑 (NIL,30 红))`：左旋 10 并换色，得到 `20 黑 (10 黑,30 黑)`，两侧向下的黑结点数相同。旋转前后中序仍为 10、20、30；修复既要补黑高，也要保持搜索次序。

这里颜色不表示访问频率，也不影响 BST 比较方向。做性质判断时，先检查中序有序，再检查根与 NIL、红红相邻、各分支黑高；仅数红黑结点总数不能判定一棵红黑树是否合法。

**课程连接·索引层服务外存页访问**

B+ 内部省去整条记录，通常能在同页放更多索引项，降低层数；叶链便于范围扫描。外存优势取决于页大小、缓存和记录存放，不能改写成任何输入均比内存 BST 快。

## 7.8 B 树：一个结点保存多个分隔关键字

本书 m 阶 B 树规定每结点最多 m 个孩子、最多 m−1 个关键字。除根外，每个数据结点至少有 t−1 个关键字，其中 $t = \lceil m/2 \rceil$；有非空孩子的非底层数据结点至少 t 个孩子，非叶根至少 2 个孩子、1 个关键字。最底层数据结点没有非空孩子，并且全部在同一层。若另外画查找失败的空外部叶，它们在下一层且不存关键字；以下高度不计这层空叶。

以下容量与高度公式采用 $m \geq 3$，高度为数据结点层数；“叶” 若按教材专指最下层的空失败结点，就不把它混入该高度。阶数 $m$、最小孩子数 $t = \lceil m/2 \rceil$ 和关键字数必须区分，不能把别处 “最小度数 t” 的算法原样按 m 替换。

一个结点有 k 个升序关键字，就有 k+1 个孩子区间。先在结点内找到目标或确定区间，再读相应孩子。多路分支降低外存层数；结点内可以顺序或折半比较，读盘次数不直接等于关键字比较次数。

### 7.8.1 插入从叶开始，分裂向上传播

以下是 **5 阶 B 树的教学机制例**，用紧凑关键字显示分裂与合并，不对应后面原题的阶数。5 阶结点最多 4 项，非根至少 2 项；非底层非根因此有 3 至 5 个孩子，非叶根仍可只有 2 个孩子。

初态根为 `[5,10,15,20]`，从左到右的 5 个叶分别为 `[1,3]`、`[6,8]`、`[11,13]`、`[16,18]`、`[21,23,25,27]`，全树共 16 个关键字。沿搜索路径插入 29 后，最右叶临时有 5 项。提升第 3 项 25，叶分成 `[21,23]` 与 `[27,29]`，提升项从叶中移走。根因而临时成为 `[5,10,15,20,25]`，也有 5 项，不能到此结束。

再将根的第 3 项 15 提升为新根，左右内部结点分别为 `[5,10]`、`[20,25]`。原来的 6 个孩子按区间各接 3 个：左侧接 `[1,3]`、`[6,8]`、`[11,13]`，右侧接 `[16,18]`、`[21,23]`、`[27,29]`。两次分裂均为 “5 项 → 左 2 项、提升 1 项、右 2 项”，内部结点分裂时还须同步分配孩子。根分裂使数据层数由 2 增至 3；局部叶分裂本身不增加全树高度。

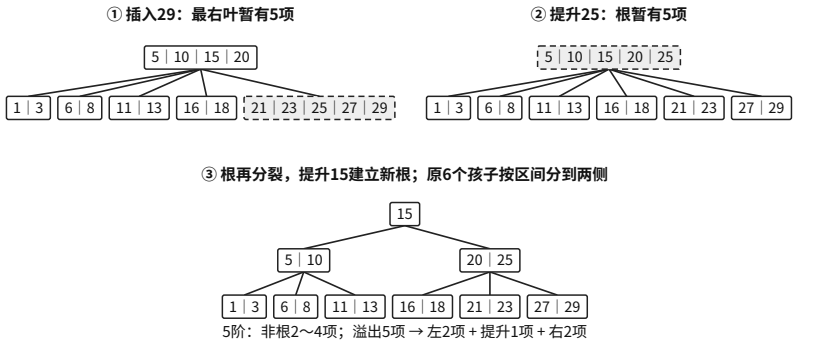



Figure 7.3: 5 阶 B 树插入 29，先分裂叶，再处理根溢出；虚框是待修复的 5 项结点

### 7.8.2 删除后的借位、合并与向上传播

**错项排除·两个取模参数别混用**

散列函数可能用小于表长的素数 p，继续探查却按表长 m 循环。若把探查也模 p，后面合法槽位永远访问不到；二次增量也应每次从原始 H0 计算。

删除内部关键字时，先用相邻前驱或后继关键字替换，再到最底层删除对应项。删除后若该结点的关键字少于最小数，先看相邻兄弟能否借：父分隔关键字下移，兄弟中最靠近当前结点的一项上移到父；若在内部层借位，右兄弟的最左孩子或左兄弟的最右孩子也要转接，保持孩子数等于关键字数加 1。兄弟也只有最小数时，合并当前结点、父分隔关键字与相邻兄弟，父可能继续下溢；空根仅剩一个孩子时，由孩子接替，高度减 1。

先看 5 阶借位小例：根 `[10]`，两个叶 `[2,5]` 与 `[12,15,18]`。删除 2 后左叶只剩 `[5]`，低于 2 项下限；右兄弟有 3 项，能够出借。将父 10 下移到左叶、右兄弟最小项 12 上移到父，得到根 `[12]`、叶 `[5,10]` 与 `[15,18]`。不能把 12 直接塞给左叶而让父仍为 10，否则分隔关系就被破坏。

再从插入图的最终树删除 1：左叶剩 `[3]`，相邻叶 `[6,8]` 已只有 2 项，不能借；合并它们与父分隔项 5，成为 `[3,5,6,8]`。其双亲原为 `[5,10]`，此时只剩 `[10]` 和 2 个孩子，也低于非根下限。右内部兄弟 `[20,25]` 同样不能借，故将左侧 `[10]`、根 15、右侧 `[20,25]` 合为 `[10,15,20,25]`，按顺序接 5 个原孩子。旧根为空，删除旧根后数据层数从 3 降为 2。

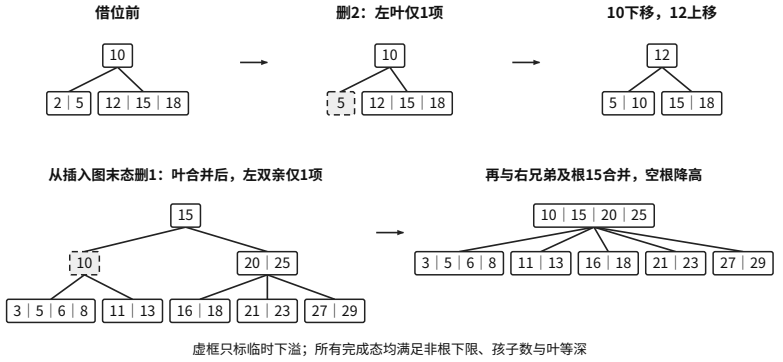



Figure 7.4: 上排为经父结点借位；下排为叶合并使双亲下溢，再合并并降低树高

5 阶 B 树的合并要特别注意时机：**两个各有 2 项的合法兄弟加父分隔项会有 5 项，超过上限**；本例是在删除造成一侧仅 1 项后，按 $1+1+2=4$ 项合并。不能把其他阶数或最小度参数下的提前合并规则直接套来。每个完成态都要复查结点内有序、孩子区间、关键字上下限、孩子数和叶等深；虚框中的临时下溢或溢出不算一个已完成的合法 B 树状态。

若高度 h 按数据结点层数计，非空 m 阶 B 树关键字总数满足 $2t^{h-1}-1 \le n \le m^h-1$，其中 $t=\lceil m/2 \rceil$。左端让根取 1 项、其余结点取最少分支，右端让各结点取满；求最大高度用左端，求最小高度用右端。

### 7.8.3 例 7-5　关键字总数怎样限制高度

**例 7.8.1** (配套案例)

〖7.4-009〗一棵 3 阶 B 树共有 2047 个关键字，最大高度为（　），最小高度为（　）。

A. 11　　B. 10　　C. 8　　D. 7

**解析**。高度按数据结点层数计。要使树尽量高，各层尽量稀疏：根 1 项、2 个孩子；其他结点至少 1 项、至少 2 个孩子。高度 h 时最少关键字 $1+2+\cdots+2^{h-1}=2^h-1$。因此 $2^h-1\le 2047=2^{11}-1$，最大 11，第一空 **A**。

要使树尽量矮，各结点取满，每层关键字数分别 2、6、18……，高度 h 时最多 $3^h-1$。$3^6-1=728<2047\le 3^7-1=2186$，所以最小 7，第二空 **D**。这两端分别使用 “最少容量” 和 “最大容量”，不能将不等式方向颠倒。

### 7.8.4 例 7-6　小规模 B 树的形态怎样完整计数

**例 7.8.2** (配套案例)

【配套例题】给定 7 个不同关键字，能构造的不同 4 阶 B 树的个数最多为（　）。

A. 7　B. 8　C. 9　D. 10

**解析**。每结点最多 3 项，非根至少 1 项；关键字的升序分配由形态与各结点项数确定，不再任意排列。

高度 1 最多 3 项，不够。高度 2 时，设根有 r 项，则有 r+1 个叶，叶项数和为 7−r，各叶 1 至 3 项：r=1 时只有 `(3,3)`，1 种；r=2 时是正整数三元组和 5，各项至多 3，包括 `(3,1,1)` 的 3 个排列与 `(2,2,1)` 的 3 个排列，共 6 种；r=3 时只有 `(1,1,1,1)`，1 种。因此高度 2 共 8 种。

高度 3 时，最少项数为 $1+2+4=7$，只能每结点 1 项且每个内部结点 2 个孩子，1 种。高度 4 最少 15 项，已不可能。合计 $8+1=9$，答案为 **C**。只考虑根的取值而不枚举叶的分配，会漏掉不同的结点分组。

## 7.9 B+ 树：索引层与叶层分工

B+ 树把记录或记录指针集中在叶结点，内部结点保存索引关键字及孩子指针；叶结点按关键字有序，并用链接便于连续访问。一次点查找沿索引降到叶；范围查找先定位起点叶，再沿叶链扫描，避免对每个值重新从根下降。

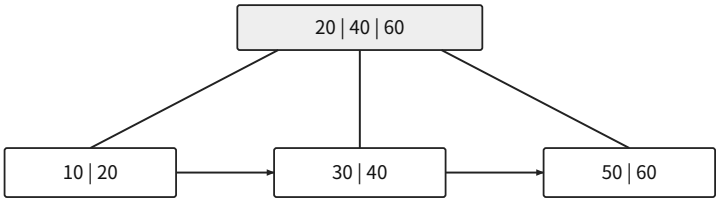



Figure 7.5: 内部层只导航，叶层保存全部关键字并按序相连

内部关键字保存的是子树最大值还是区间分隔值，m 阶结点的键数与指针数怎样对应，不同教材的具体约定并不相同。王道书常用 “内部结点关键字对应子树最大关键字” 的表示；本图按该约定，索引 `[20|40|60]` 指向最大值分别为 20、40、60 的三片子树。做题应先按给定图或定义确定，不把 B 树的 “k 键必有 k+1 孩子” 无条件套给 B+ 树。

**不变量·墓碑保住后续探查路径**

已删除槽说明这里曾发生占用，后面可能仍有沿同一序列插入的元素。查找只能在从未用空槽或探查耗尽处失败；插入复用墓碑前还要排除后方已有同关键字。

B 树的查找可以在内部结点命中时结束；B+ 树的记录查询通常仍要走到叶。叶链让 B+ 适合区间访问，并不意味着每次点查都顺着整条叶链查找。内部不携带整条记录，通常能提高同样页大小下的分支数，减少外存层数；优势来自页组织与访问方式，不能据此声称任何规模都一定比内存 BST 更快。

**错项排除·两个取模参数别混用**

散列函数可能用小于表长的素数 p，继续探查却按表长 m 循环。若把探查也模 p，后面合法槽位永远访问不到；二次增量也应每次从原始 H0 计算。

## 7.10 散列表：冲突、删除与 ASL

散列函数 $H(key)$ 把关键字映射到表地址；不同关键字有相同函数值称为同义词，发生地址冲突。函数应对同一关键字给出确定地址，并尽量均匀分散。随机改变同一关键字的地址会使后续查找无法复现位置，不能直接作为稳定散列函数。

### 7.10.1 散列函数的构造条件

直接定址可用 $H(key)=a\,key+b$，适合关键字分布较连续且范围可控；它避免了所选模型内的碰撞，但地址范围大时空间可能浪费。除留余数法用 $H(key)=key \bmod p$，通常选不大于表长且有利于分布的素数 $p$；取模常数与开放定址继续探查的表长 $m$ 是两个参数。

数字分析法利用已有关键字分布选择较均匀的数位；平方取中法先平方再取中间若干位，希望让多位信息影响结果。没有一种构造能对所有未知输入自动保证无冲突或均匀，应依据关键字类型、分布和地址范围选择；同一关键字必须能稳定复现相同初始地址。

**开放定址**在表内继续探查空位置。线性探查地址为 $(H_0+k) \bmod m$；二次探查可能规定为 $(H_0+k^2) \bmod m$，也可能按正负平方交替，须按题目公式执行。探查时的模数取表长 m；散列函数内部的除数 p 可能小于 m，两者不能混用。线性探查能够在一整轮内访问全表，二次探查不一定遍历所有位置，即使表未满也可能找不到可达空位。

**链地址法**把同义词放在一个桶的链表中，成功时沿链比较，失败时须比较到链尾。开放定址的聚集不仅来自同义词，也可能由不同初始地址的探查链交错引起。装填因子 $\alpha=n/m$ 越大，一般可用位置越少；散列并不保证任何输入下都严格常数时间。

### 7.10.2 例 7-7　删除标记为什么继续计入失败路径

**例 7.10.1** (配套案例)

【配套例题】长度为 5、初始为空的散列表 HT，$H(k)=(k+4) \bmod 5$，线性探查处理冲突。依次插入 2022、12、25，再删除 25，查找失败的平均查找长度为（　）。

A. 1　　B. 1.6　　C. 1.8　　D. 2.2

**解析**。2022 与 12 初始地址都为 1，分别存 1 与 2；25 存 4。删除 4 处关键字后应留 “已删除” 标记，而不是把它当作从未使用的空位置。得到：

| 地址 | 槽状态 | 从此地址起失败探查次数 |
|---|---|---|
| 0 | 从未用空 | 1 |
| 1 | 2022 | 3 |
| 2 | 12 | 2 |
| 3 | 从未用空 | 1 |
| 4 | 已删除 | 2 |

失败查找遇到从未使用的空位才结束；越过删除标记还要继续。按 5 个初始地址等概率，ASL 为 $(1+3+2+1+2)/5=1.8$，答案为 **C**。探查到空单元也计一次表访问，这是本题失败 ASL 的计数口径；删除标记处没有有效关键字可比，却不能在这里终止。

若把一个探查链的中间位置直接清空，后面的元素虽然还在，查找却会在这里提前失败。插入时删除位置可以复用；若不允许重复关键字，要继续查到能排除已有同关键字的位置，再决定使用哪个已删除槽。

**不变量·墓碑保住后续探查路径**

已删除槽说明这里曾发生占用，后面可能仍有沿同一序列插入的元素。查找只能在从未用空槽或探查耗尽处失败；插入复用墓碑前还要排除后方已有同关键字。

### 7.10.3 例 7-8　二次探查必须回到原始地址计算

**例 7.10.2** (配套案例)

【配套例题】将 20、3、11、18、9、14、7 依次插入初始为空、长度 11 的 HT。$H(key)=(key\times 3)\bmod 11$，初始地址为 $H_0$，冲突时 $H_k=(H_0+k^2)\bmod 11$ $(k=1,2,\ldots)$。

（1）画出 HT 并计算装填因子。（2）给出查找 14 时的关键字比较序列。（3）查找 8，确认失败时的散列地址是多少?

**解析**。20、3、11、18 直接存于 5、9、0、10。9 从 5 冲突后到 6；14 从 9 依次探查 10、2，存 2；7 从 10 依次探查 0、3，存 3。

| 地址 | 关键字 |
|---|---|
| 0 | 11 |
| 1 | 空 |
| 2 | 14 |
| 3 | 7 |
| 4 | 空 |
| 5 | 20 |
| 6 | 9 |
| 7 | 空 |
| 8 | 空 |
| 9 | 3 |
| 10 | 18 |

装填因子为 7/11。查 14 的探查地址为 9、10、2，比较关键字为 3、18、14。查 8 时，$H_0=24\bmod 11=2$，探查地址依次为 2、3、6、0、7，第 7 号为空，所以在**地址 7** 确认失败。每个新地址从同一个 $H_0$ 加平方增量计算，不能在上一次地址上再加 $k^2$。

### 7.10.4 开放定址的三种槽状态

```
hashFind(key):
    按约定的探查序列访问地址
    若槽为有效且关键字相同：成功
    若槽从未使用：失败
    若槽已删除或关键字不同：继续探查
    若可探查位置已耗尽：失败
```

删除先按同一路径找到关键字，再把槽改为 “已删除”。插入可记住第一个已删除槽，但不能立即停在那里：若后面已有同关键字且不允许重复，提前写入会制造重复。探查总步数须有界，表满或二次探查可达集合已走完时不能无限循环。

链地址法的插入在相应桶链完成，删除是真实断链并释放目标结点；它不需要开放定址的墓碑标记。链长才是比较量来源，不能拿开放定址中 “遇从未用空槽终止” 的规则解释链地址失败。

**课程连接·索引层服务外存页访问**

B+ 内部省去整条记录，通常能在同页放更多索引项，降低层数；叶链便于范围扫描。外存优势取决于页大小、缓存和记录存放，不能改写成任何输入均比内存 BST 快。

## 7.11 怎样按操作约束选择方法

下表把主要成立条件与查找代价分开。记录数为 n，散列表长为 m，树高记为 h；B/B+ 树的 h 按数据层数计，假定每个结点占一个外存页，表中的读盘上界指索引结点路径上的页访问，还未扣除已缓存页；若叶中只存记录指针，随后取叶外记录的 I/O 另计。

| 方法或结构 | 成立条件 | 查找代价 | 更新、范围或外存特点 |
|---|---|---|---|
| 顺序查找 | 可无序；数组或链表 | 最坏 $O(n)$ | 不需维护有序；范围筛选仍须扫描 |
| 折半查找 | 有序且能随机定位的顺序表 | 最坏 $O(\log n)$ | 数组插删最坏移动 $O(n)$；可定位区间端点 |
| 普通 BST | 满足全部子树的区间关系 | $O(h)$，最坏 $O(n)$ | 支持前驱后继；无平衡维护时可退化成链 |
| AVL | BST 且任一结点两侧高差至多 1 | 最坏 $O(\log n)$ | 动态插删需维护高度与旋转 |
| 红黑树 | BST 及红黑性质均成立 | 最坏 $O(\log n)$ | 动态插删维护颜色、黑高与旋转 |
| B 树 | 多关键字有序、容量约束、叶等深 | 至多 h 次结点读盘 | 内部命中可停；读入结点后的键比较另计 |
| B+ 树 | 按给定索引约定，叶层存全体关键字及记录或记录指针 | 点查沿根到叶，至多 h 次索引页读盘 | 定位起点后沿叶链作范围访问 |
| 散列表 | 散列函数、装填因子和冲突策略匹配 | 合适散列与受控负载下平均 $O(1)$；最坏链扫描 $O(n)$，开放定址探查 $O(m)$ | 扩容有维护成本；原生不保序，不直接支持有序范围定位 |

一个表可能有多种索引。不能只为某一次查找更快，就忽略频繁更新带来的维护成本；也不能用散列的平均等值查找优势替代区间检索所需的有序性。题目给出 “有序、随机访问、频繁插删、外存、范围” 时，分别把它们转成结构成立条件与访问成本，再作选择。

## 7.12 主动回忆自测

以下为自拟补充，检查维护过程与失败口径。

1. BST 中每个结点都满足 “左孩子小于自身、右孩子大于自身”，是否足够？给反例。
2. 4 阶 B 树一个非根结点最多、最少有多少关键字？根为空时能否直接认定整树空？
3. 一条散列探查路径经过 “有效关键字—删除标记—有效关键字—从未用空位”，目标均不等，失败探查几次？
4. 红黑树根 10 黑、孩子 5 红和 15 红，插入 1 后，首先应旋转还是变色？
5. AVL 删除后只修复最深一处失衡是否总够？与插入有何不同？
6. B 树一个内部结点有 3 个关键字，按本章约定有几个孩子区间？B+ 树能无条件沿用吗？

7. 在有序链表上执行折半式比较，为何不能直接承诺总时间 $O(\log n)$？
8. 为什么散列表失败 ASL 的分母未必是有效记录数？若表长 5、只存 2 项，按初始地址等概率应除以几？

**答案**。1. 不够；根 10，左孩子 5，5 的右孩子 12，所有直接比较都满足，但 12 在 10 左子树却大于 10。2. 最多 3 项、最少 1 项；根空但还有唯一孩子时应降层，由孩子接替，树不空。3. 4 次，删除标记不能截断路径。4. 先变色，叔 15 红，把 5、15 变黑、10 暂红，最后根恢复黑，无需该处旋转。

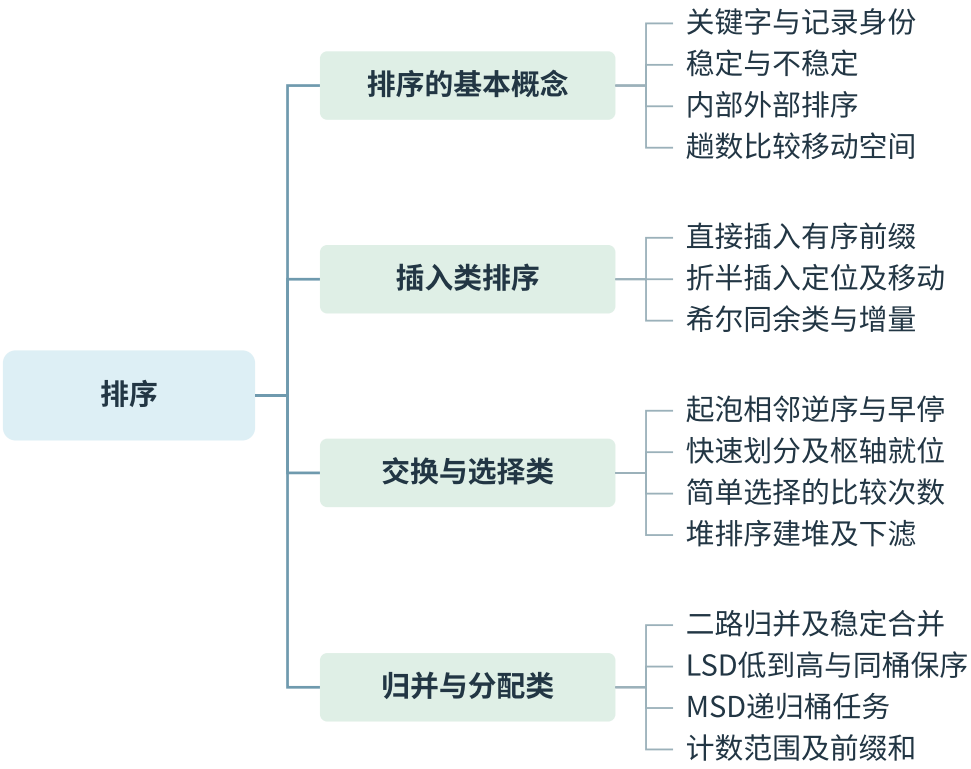
5. 不总够；删除可能让修复后子树高度继续减 1，导致更高祖先失衡。插入修复最小失衡子树后通常恢复原高度，可阻断继续传播。
6. 有 4 个孩子区间；B+ 索引约定各异，若每项表示一棵子树最大关键字，键数和孩子数可以相等，需按给定定义。
7. 比较次数可能为对数，但链表不能常数时间随机定位中点，节点访问成本没有消失。
8. 失败事件分布可以是初始散列地址或失败区间，而非已有记录；该例应按 5 个初始地址求平均，除以 5。

**不变量·B 树合法性有五项**

每次操作完成后同时检查结点内有序、孩子区间、键数上下限、孩子数、叶层深度。只画出平衡外形会漏掉关键字重复或孩子接错；临时溢出框不是最终合法状态。

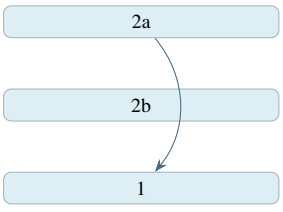
# 排序 8

**知识导图**



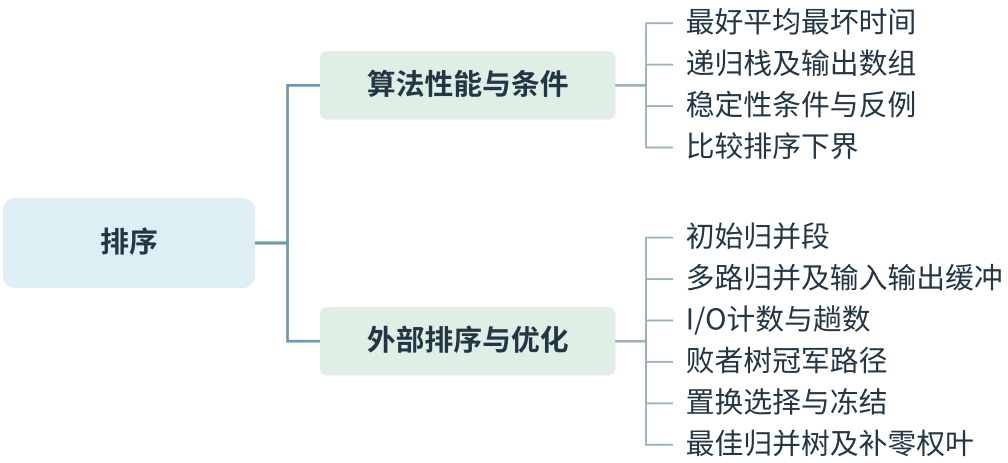

**最小反例·相同数字隐藏记录身份**

最小反例给相等关键字附身份：$2_a, 2_b, 1$ 经简单选择首趟成为 $1, 2_b, 2_a$。只写数字 2、2 会看不出顺序改变；稳定性讨论的是整条记录的身份。




交换首项与 1 跨过 2b，结果为 1、2b、2a。

**知识导图（续）**




**不变量·相等时继续向右定位**

折半插入寻找首个大于 x 的位置，旧相等项都留在左边。定位快只减少比较，数组腾位置仍要移动；看到折半二字就写整体对数时间，漏掉了主要移动量。

排序把记录按关键字排列，同时可能改变相等关键字记录原有的相对次序。做过程题时，先确定这一趟处理的是一个元素、一个增量分组，还是所有当前子区间；做选择算法的题时，再比较规模、初始状态、稳定性、空间和外存访问。算法名字相同、划分约定不同，产生的中间序列也可能不同。

错项排 除·最好线性依赖早停标

起泡有序输入一趟无交换即可停，所以线性；若程序固定执行全部趟数，即使每次都不交换，比较仍是二次。复杂度表需要与实际程序条件配

## 8.1 导图主干与阅读路径

- **排序的基本概念**：关键字与记录身份；稳定与不稳定；内部外部排序；趟数比较移动空间。
- **插入类排序**：直接插入有序前缀；折半插入定位及移动；希尔同余类与增量。
- **交换与选择类**：起泡相邻逆序与早停；快速划分及枢轴就位；简单选择的比较次数；堆排序建堆及下滤。
- **归并与分配类**：二路归并及稳定合并；LSD 低到高与同桶保序；MSD 递归桶任务；计数范围及前缀和。
- **算法性能与条件**：最好平均最坏时间；递归栈及输出数组；稳定性条件与反例；比较排序下界。
- **外部排序与优化**：初始归并段；多路归并及输入输出缓冲；I/O 计数与趟数；败者树冠军路径；置换选择与冻结；最佳归并树及补零权叶。

先按导图说清对象与条件，再沿相应正文执行算法或推导。章末先遮住答案回忆各主枝；不能只凭图中文字是否熟悉判断掌握。

## 8.2 稳定性、内外排序与比较口径

**定义 8.2.1** (排序稳定性)

若两条记录关键字相等，排序后它们的相对次序与原来一致，称该排序算法**稳定**。给记录加身份下标，如 $2_a, 2_b$，就能检验相等值是否被交换；只看两个相同数字无法观察稳定性。稳定性是算法对所有合法输入的保证，某个不稳定算法在一份特定数据上也可能恰好没有打乱相等记录。

内部排序指排序时待处理记录能全部放入内存；外部排序需要在内外存间分批交换。二者的区别在于处理期间是否需要外存参与，不能只按一个固定的数据量阈值划分。外部排序通常优先减少块 I/O，内部排序则常比较关键字比较次数、移动次数和辅助空间。

“一趟” 指什么，须按具体算法的说明确定。直接插入一趟放好一个新记录；归并一趟合并所有相邻段；本章快速排序的过程题，把同一深度上尚未确定最终位置的所有子区间各划分一次，算作一趟。若程序逐子区间调用一次划分就计一趟，应按那份程序重新解释。

## 8.3 直接插入与折半插入

直接插入维护左侧已排好序的前缀。取下一个记录 x，从前缀末尾向左比较，把大于 x 的记录右移，直到找到插入位置。每次循环结束，前缀扩大 1 且仍有序。

```
void insertionSort(int A[], int n) {
    for (int i=1; i<n; ++i) {        int x=A[i], j=i-1;
        while (j>=0 && A[j]>x) {
            A[j+1]=A[j]; --j;
        }
        A[j+1]=x;
    }
}
```

**不变量·d 间隔有序不是相邻有序**

一趟希尔处理全部同余类，得到每个间隔 d 子序列有序。不同余数类之间仍未比较；只有最后 d=1，才把全数组约束为同一有序序列。

相等时不再左移，把新记录放在已有相等记录后面，因此稳定。若条件改为 `A[j]>=x`，会把新记录放到旧相等记录之前，稳定性随之改变。初始有序时每趟只比较一次就停止，时间 $\Theta(n)$；逆序时要移动 $1+2+\cdots+(n-1)$ 个记录，最坏 $\Theta(n^2)$；仅用一个暂存记录和几个下标，空间 $O(1)$。

折半插入在有序前缀中用折半查找决定插入点，减少了关键字比较；但数组中腾出位置仍要移动元素，平均和最坏移动量仍为 $\Theta(n^2)$，因此平均和最坏总时间也为 $\Theta(n^2)$。要保持稳定，应寻找“第一个大于 x”的位置，把 x 插到所有旧相等项之后。

本章比较表采用**每趟都完整执行折半定位**的实现，不先用前缀末项判断是否已经就位。因此即使初始有序，比较总量仍为 $\Theta(\sum_{i=1}^{n-1}\log(i+1))=\Theta(n\log n)$，最好总时间也是 $\Theta(n\log n)$；最好时只是移动少，比较代价并不随之减少。若另加“已不小于前缀末项则直接跳过折半”的判断，有序输入可变成线性时间，那是增加了提前判断的实现。

### 8.3.1 折半定位怎样保持稳定

```
binaryInsertion(A,n):
    for i=1..n-1:
        x=A[i]; low=0; high=i-1
        while low<=high:
            mid=low+(high-low)//2
            若A[mid]<=x: low=mid+1
            否则high=mid-1
        将A[low..i-1]向右移动1位
        A[low]=x
```

相等时向右找，最终 `low` 指向有序前缀中第一个大于 x 的位置。寻找位置的比较减少，腾出位置的移动仍可能为二次；“折半”描述定位方式，不是整体复杂度结论。

## 8.4 起泡：一次相邻交换消除一个逆序

从左向右比较相邻元素，前者大于后者就交换，一趟后最大记录到达最右端；下趟只处理未确定的左侧范围。若某一趟没有发生交换，整段已无相邻逆序，可提前结束。没有这项标志的实现即使输入有序，也仍会执行全部固定比较。

例如 `4,2,3,1` 第一趟依次变为 `2,4,3,1→2,3,4,1→2,3,1,4`，只保证末尾 4 就位，不保证其余已序。每次交换仅在严格大于时进行，不会跨过相等记录，故稳定。带提前终止时最好 $\Theta(n)$，平均、最坏 $\Theta(n^2)$，空间 $O(1)$。

### 8.4.1 带提前停止的起泡代码

```
void bubbleSort(int A[], int n) {
    for (int end=n-1; end>0; --end) {
        bool changed=false;
        for (int j=0; j<end; ++j)
            if (A[j]>A[j+1]) {
                int x=A[j]; A[j]=A[j+1]; A[j+1]=x;
                changed=true;
            }
        if (!changed) break;
    }
}
```

**错项排除·首枢轴遇到已有序可能最坏**

首记录作枢轴时，升序输入每次都产生 0 与 n−1 的分割，时间二次、栈线性。插入排序有序最快的经验不能迁移到这份快排；随机枢轴需另用期望模型分析。

此段使用 C++ 的 `bool`；改用 C 时应包含 `stdbool.h` 或使用整型标志。每趟后尾部已确定的记录不再参与下一趟，未交换则剩余前缀也已序。

## 8.5 简单选择：比较与移动的代价不同

简单选择每趟从未排序区找到最小记录，与该区首记录交换，使前部有序区扩大。无论初始是否有序，找最小值都要扫描当前未排序区，关键字比较总数固定为 $(n-1)+(n-2)+\cdots+1=n(n-1)/2$。至多交换 n−1 次，比较次数却始终为 $\Theta(n^2)$。

它通常不稳定。例如 $2_a, 2_b, 1$，第一趟把 1 与 $2_a$ 交换，得到 $1, 2_b, 2_a$，相等关键字次序逆转。这不是比较时相等就交换造成的，而是与远处最小记录交换时跨越了 $2_b$；仅把找最小值的条件改成严格小于，不能保证稳定。

```
selectionSort(A,n):
    for i=0..n-2:
        pick=i
        for j=i+1..n-1:
            若A[j]<A[pick]: pick=j
        若pick!=i: swap(A[i],A[pick])
```

省掉 `pick==i` 时的自交换可以减少移动，却不减少寻找最小值的比较。这说明“比较固定”不等于“每份输入交换数也固定”。

## 8.6 希尔：一趟只保证各增量子序列有序

希尔排序选择递减增量 d，对下标同余的记录分别作直接插入排序，称为 d 间隔有序。较大增量使远处记录较快移动，最后必须有 d=1 这一趟，才能保证整体有序。某个 d 间隔有序，不代表相邻记录有序；改用更小的增量后，也仍需再排。

### 8.6.1 例 8-1　两趟希尔排序怎样按下标分组

**例 8.6.1** (配套案例)

〖8.2-012〗输入序列为 `13,24,7,1,8,9,11,56,34,51,2,77,5`，增量依次为 5、3、1。两趟排序后为（　）。

A. 1,7,8,9,13,24,11,34,51,2,5,56,77

B. 1,7,5,2,8,9,24,11,34,51,13,77,56

C. 2,11,5,1,8,9,24,7,34,51,13,77,56

D. 2,5,11,1,8,9,7,24,34,13,51,77,56

**解析**。 d=5 按下标余数分组，例如下标 0、5、10 的 `13,9,2` 排为 `2,9,13`；下标 2、7、12 的 `7,56,5` 排为 `5,7,56`。第一趟整体为 `2,11,5,1,8,9,24,7,34,51,13,77,56`。

d=3 重新分组：下标 0、3、6、9、12 的 `2,1,24,51,56` 排为 `1,2,24,51,56`；下标 1、4、7、10 的 `11,8,7,13` 排为 `7,8,11,13`；下标 2、5、8、11 的 `5,9,34,77` 已有序。填回原下标得 `1,7,5,2,8,9,24,11,34,51,13,77,56`，答案为 **B**。

“一趟” 包含全部余数分组，不是只排第一组。最后 d=1 这一趟才对全序列作插入排序。希尔排序不稳定，复杂度与增量序列有关：不附增量条件，便给不出通用的精确平均阶；空间通常为 $O(1)$。

**不变量·尾部已序，前部只是堆**

升序堆排序每轮把当前最大值固定到尾部，下一轮只调整缩小后的前部堆。前部兄弟不保证有序，不能把一次建堆的数组误判为一趟完整升序输出。

### 8.6.2 希尔排序的同余类循环

```
shellSort(A,n,gaps):
    对每个递减正增量d（最后必须为1）：
        for i=d..n-1:
            x=A[i]; j=i-d
            while j>=0且A[j]>x:
                A[j+d]=A[j]; j-=d
            A[j+d]=x
```

内层以 d 为步长在同一同余类中移动，因此一次 i 循环只处理一个新记录；完整一趟要遍历全部 i。所有增量必须为正，否则无法保证进展；末增量 1 将各分组约束最终汇成整体有序。

## 8.7 快速排序：先划分，再递归

一趟划分选择枢轴，使左区记录不大于枢轴、右区记录不小于枢轴，并确定一个枢轴的最终位置；两侧内部仍可能无序。以下采用首记录作枢轴、双向扫描的 “挖坑” 方式，便于明确指针停留与填回位置。

```
int partition(int A[], int low, int high) {
    int pivot=A[low];
    while (low<high) {
        while (low<high && A[high]>=pivot) --high;
        A[low]=A[high];
        while (low<high && A[low]<=pivot) ++low;
        A[high]=A[low];
    }
    A[low]=pivot;
    return low;
}
```

```
quickSort(A,l,r):
    if l>=r: return
    p=partition(A,l,r)
    quickSort(A,l,p-1)
    quickSort(A,p+1,r)
```

每次划分扫过当前区间，代价与区间长度同阶。两侧规模较均衡时，每层合计 $O(n)$、约 $\log n$ 层，时间 $O(n\log n)$；若首枢轴总是最小或最大，规模为 n−1 与 0，累积为 $\Theta(n^2)$。原序列已经有序时，这个首枢轴实现反而落到最坏情况，不能把插入排序 “有序输入最快” 的结论照搬过来。

递归栈平均 $O(\log n)$、最坏 $O(n)$；数组内划分的常数临时变量不能抵消调用栈。远距离移动会改变相等记录顺序，通常不稳定。等值很多时，具体左右扫描规则会影响划分平衡；三路划分是另一种处理方式，不能把它的状态与上述程序的状态混写。

**课程连接·优先队列与选择小部分记录**

只找最小 10 项时可维护 10 项大根堆，把更小新值替换堆顶；不必完整排序。流式读入适合这种状态维护，若可原地随机访问，还可比较选择算法的期望线性方案。

### 8.7.1 例 8-2　第二趟结束后应有几个枢轴就位

**例 8.7.1** (配套案例)

【配套例题】对尚未确定最终位置的所有元素进行一遍处理称为一趟。下列序列中，不可能是快速排序第二趟结果的是（　）。

A. 5,2,16,12,28,60,32,72

B. 2,16,5,28,12,60,32,72

C. 2,12,16,5,28,32,72,60

D. 5,2,12,28,16,32,72,60

**解析**。先确定完全有序结果为 `2,5,12,16,28,32,60,72`。一个就位枢轴还必须满足左边全不大于它、右边全不小于它。若第一枢轴在端点，第二趟只划分一个非空子区间，可能只有两个枢轴就位；若第一枢轴不在端点，两侧均非空，第二趟后至少有三个就位位置（单元素区间自身已就位）。

选项 D 中只有 12 与 32 这两个位置满足分隔关系，而且两者都不是端点。无论谁作为第一枢轴，第二趟都应在它左右再各形成一个就位位置，这与只有两个就位位置相矛盾，所以答案为 **D**。

选项 A 可先把最大值 72 定在末尾，再在左区把 28 就位；选项 B 可先把最小值 2 定在首位，再把 72 就位；选项 C 可先把 28 就位，再让左区的 2、右区的 32 就位。判断依据不是 “看起来有几个数字已按升序排好”，而是每个枢轴对整个区间的分隔作用。

## 8.8 堆排序：堆范围与有序区分开

升序堆排序先建大根堆，根即未排序区的最大值；把根与当前末元素交换，这个最大值就留在末尾，堆范围减 1，再对新根下滤。重复到堆范围只剩一个元素。此时数组尾部有序，前部只是一个堆，整体并未递增。

建堆 $O(n)$，每次删除最大值并调整至多 $O(\log n)$，完整排序有 $O(n\log n)$ 时间上界；常规迭代下滤空间 $O(1)$。教材常按关键字互异的模型，把各种情况都列为对数线性阶，但不能据此断言任意重复输入都要做同样多的比较：若关键字全相等且下滤在相等时立即停止，具体实现可以只做线性数量次比较。末尾与根的远距离交换使堆排序通常不稳定。小根堆反复交换到末尾得到的是降序数组，不能只看 “每次取最小” 就认定数组最后为升序。

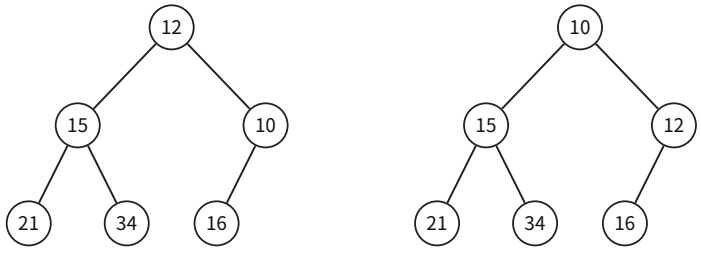
### 8.8.1 例 8-3　删除堆顶时怎样数关键字比较

**例 8.8.1** (配套案例)

【配套例题】小根堆为 8,15,10,21,34,16,12，删除关键字 8 之后重建堆，此过程中关键字之间的比较次数是（　）。

A. 1　B. 2　C. 3　D. 4

**解析**。用最后的 12 填根，堆范围由 7 缩为 6。先比较两个孩子 15 与 10，选 10（第 1 次）；再比较候选 12 与 10（第 2 次），10 上移、12 下到原 10 位置。该位置只有一个孩子 16，比较 12 与 16（第 3 次）后停止。结果为 10,15,12,21,34,16，答案为 **C**，过程如下图所示。




比较：15与10；12与10；12与16，共3次

Figure 8.1: 删除堆顶后沿较优孩子路径下滤；只比较真实存在的孩子

无需再把 16 与不存在的右孩子比较，循环边界检查也不是关键字比较。若题目给了另一个下滤程序，例如始终下沉至叶再回升，应按那个程序计数，不能把本过程的 3 次当作所有实现的共同常数。

### 8.8.2 完整堆排序的两段工作

```
heapSort(A,n):
    for i=n//2 downto 1: siftDown(A,i,n)
    for end=n downto 2:
        swap(A[1],A[end])
        siftDown(A,1,end-1)
```

此处用 1 起始数组及大根堆，`siftDown` 每轮先选较大孩子，再判断暂存根值能否停止；其核心代码见第 5 章，也可按下面的短伪码独立实现。

```
siftDown(A,r,end):
    x=A[r]
    while 2*r<=end:
        c=2*r; 若c<end且A[c]<A[c+1]: c++
        若x>=A[c]: break
        A[r]=A[c]; r=c
    A[r]=x
```

第一段自底向上建堆，第二段才把最大值逐个固定到末尾。不能把建堆 $O(n)$ 当成完整堆排序的最坏时间。

**错项排除·比较上界不等于具体次数**

归并 a、b 项最坏比较 a+b−1 次，但一段提前耗尽就直接复制另一段。计算给定值的过程题应逐次比较段首，不能每次都机械代入最坏上界。

## 8.9 二路归并：有序子段怎样变成长有序段

**不变量·LSD 依赖每轮的稳定性**

更高位相同的记录仍要靠已经排好的低位决定先后，故同桶必须保留上轮次序。改成插桶头或逆序收集可能破坏这个不变式，即使每轮单独看都按本位有序。

归并两个有序段时，只需比较当前未输出首记录，较小者先输出；一段耗尽后，另一段直接复制。相等时先取左段能保持稳定。归并长度分别为 a、b 的段，最坏作 a+b−1 次关键字比较：到只剩最后一个记录时，它属于哪个段已经确定，直接复制，不再作跨段比较。

自底向上先把每个记录视为长度 1 的段，一趟两两合并，段长最长变为 2，再到 4、8……；自顶向下先递归处理两半，再归并。每层总处理 $O(n)$ 个记录，层数 $O(\log n)$，故普通二路归并时间为 $O(n\log n)$，不因原序列已序就自动跳过整套过程。数组实现的归并辅助区通常为 $O(n)$，递归栈的 $O(\log n)$ 不改变总空间阶。

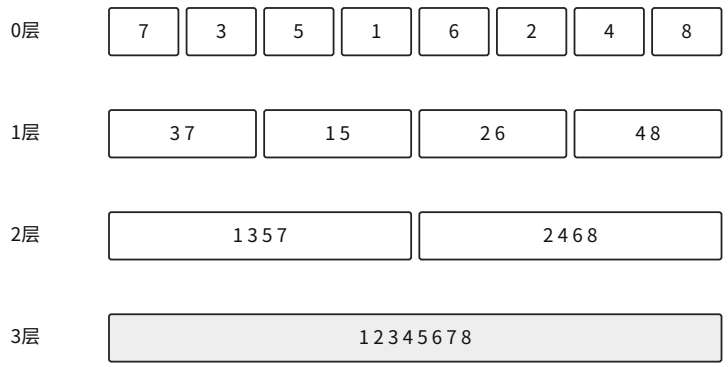



Figure 8.2: 每层归并读取上一层的有序段，最后一步仍需比较两个段首

### 8.9.1 例 8-4　比较次数不等于移动次数

**例 8.9.1** (配套案例)

【配套例题】三个有序序列 (3,5)、(7,9)、(6)，按从左到右次序选择序列进行二路归并，关键字总比较次数为（　）。

A. 3　B. 4　C. 5　D. 6

**解析**。先归并前两段，比较 3 与 7、5 与 7，两次后左段耗尽，直接接 7、9，得到 3,5,7,9。再与单元素 6 归并，比较 3 与 6、5 与 6、7 与 6，共 3 次，之后接 7、9。总数 $2+3=5$，答案为 **C**。最坏上界 a+b−1 只有在具体过程能一直交错到仅剩一个元素时才取到，本题第一次只比较 2 次，不能机械按 4−1 计 3 次。

### 8.9.2 两段归并的稳定合并核心

```
merge(A,l,mid,r,B):
    i=l; j=mid+1; k=l
    while i<=mid且j<=r:
        若A[i]<=A[j]: B[k++]=A[i++]
        否则: B[k++]=A[j++]
    复制左段剩余项至B
    复制右段剩余项至B
    将B[l..r]写回A[l..r]
```

**相等时先取左段为何保序**

假设两个输入子段内部已经稳定有序。来自同一段的记录依次输出，相对次序不变；来自不同段的相等记录，左段原本在前，比较相等时仍先输出左段。因此合并保留所有相等记录的原相对次序，递归或分层重复后全序列仍稳定。

□

**课程连接·外排瓶颈联系操作系统 I/O**

外排优先按块读写，输出缓冲攒满再写可减少细粒度 I/O。多路需要更多输入缓冲，路数受实际内存约束；双缓冲与异步 I/O 还会改变可用块数和耗时模型。

## 8.10 基数排序：低位排好后怎样保留次序

基数排序按关键字的组成部分分配、收集。LSD 从最低有效位开始，逐位稳定排序；每轮收集必须保持同一桶内原顺序，才能保留先前已经排好的低位关系。处理更高位后，低位只在高位相同时才决定顺序，这也正是从低到高稳定处理的依据。

### 8.10.1 例 8-5　第一趟后为什么尚未整体递增

**例 8.10.1** (配套案例)

〖8.5-012〗对 `05,46,13,55,94,17,42` 进行基数排序，一趟后的结果为（　）。

A. 05,46,13,55,94,17,42

B. 05,13,17,42,46,55,94

C. 42,13,94,05,55,46,17

D. 05,13,46,55,17,42,94

**解析**。本题采用通常的十进制 LSD，第一趟按个位分配：桶 2 放 42，桶 3 放 13，桶 4 放 94，桶 5 依次放 05、55，桶 6 放 46，桶 7 放 17。按桶号收集得 `42,13,94,05,55,46,17`，答案为 **C**。桶 5 不能把 55 放到 05 之前。第二趟按十位稳定处理，才得到 `05,13,17,42,46,55,94`。

若共 d 位、基数 r，每轮分配 n 项并收集 r 个桶，时间为 $O(d(n+r))$。空间要按实现：已有链式记录、链接域计入原记录时，额外的桶头、桶尾指针为 $O(r)$；数组分桶另开输出数组时为 $O(n+r)$。固定十进制位数与 r 时可近似线性，但不能把位数随规模增长的情形也无条件写为 $O(n)$。

MSD 从最高位开始，把记录分成桶后递归处理各桶；它的任务分解与 LSD 不同。负整数、可变长字符串、空缺位的次序都需要事先定义，不能把非负定长十进制示例原样用于所有关键字。

### 8.10.2 按取值计数：空间与稳定实现的条件

计数排序把关键字的值映射为计数数组的下标。若整数取值从 min 到 max，令 $k=\max-\min+1$，以 `key-min` 为下标：先统计频次，再求前缀和，得到每个值应占据的输出位置范围。例如序列 $2_a,4,3_a,0,2_b,3_b$ 的频次数组为 `[1,0,2,2,1]`，前缀和为 `[1,1,3,5,6]`。从后向前扫描原序列，每次把记录放到该值当前最后一个输出位置，再将位置减 1，得到 $0,2_a,2_b,3_a,3_b,4$，相等记录仍保序。

按这一稳定实现，时间为 $\Theta(n+k)$，输出数组与计数数组合计辅助空间 $\Theta(n+k)$。它适合整数取值范围较小的情形；k 很大时，清零和累计计数本身就可能不合算。若只按次数把数值重新覆写回数组，没有保存相等记录的身份及附加数据，不能据此宣称对记录作了稳定排序。

计数、基数利用关键字结构，不属于仅靠关键字两两比较来确定次序的模型，因而不直接受比较排序的 $\Omega(n \log n)$ 最坏下界约束。

**最小反例·输出段可以比内存更长**

最小直观情形：工作区 1 条，输入 1、2、3。每输出一条再换入更大者，三条属于同一段，内存始终只存 1 条。不能把段长固定等同工作区容量。

### 8.10.3 LSD 和稳定计数的核心次序

```
LSD(records,d,r):
    从最低位到最高位执行d轮：
        清空r个桶
        按当前序列从前向后，把记录追加到对应桶尾
        按桶号递增、桶内先入先出收集
```

```
stableCounting(records,min,max):
    k=max-min+1; count[0..k-1]=0
    扫描记录并按key-min累计count
    对count求前缀和
    从后向前扫描原记录x:
        count[x.key-min]--
        B[count[x.key-min]]=x
    返回B
```

计数输出位置以 0 开始。输出的是整条记录而非只覆写关键字，才有条件讨论相等记录身份的稳定性；若取值范围很大，必须先衡量 $k$ 所需空间与清零代价。

## 8.11 外部排序：归并段、缓冲与 I/O

外部归并先把可装入内存的一批记录排成有序初始归并段，再多路归并。内存可同时放 B 个块、每路至少 1 个输入缓冲块且留 1 个输出缓冲块时，最多支持 B−1 路简单归并；双缓冲或额外选择结构还可能占更多空间。提高路数可以减少趟数，但不能忽略缓冲的实际容量。

若有 r 个初始段，固定 k 路平衡归并需要 $\lceil \log_k r \rceil$ 趟，最后一组可不足 k 路。每一趟把全部 N 个数据块读入并写出，通常为 2N 次块传输；生成初始段也读写全文件，若计入该阶段，总量为 $2N[1+\lceil \log_k r \rceil]$，这里不计寻道、缓存命中或最后输出直送等特殊节省。

例如 4500 条记录，每块 75 条，内存能容 450 条，则文件 60 块、内存 6 块。直接分批内排序可得到 10 个初始段，每段 450 条、6 块；若用 5 路归并，10→2→1，需要 2 趟归并。计初始段生成，总块传输 $2 \times 60 \times 3 = 360$ 次。记录数、块数、段数各起不同作用，不能把 4500 直接当作 I/O 次数。

### 8.11.1 置换—选择为什么能产生长于内存的段

工作区保存当前仍可进入本段的记录。每次输出其中最小值 x，再读一个新记录 y：若 $y \geq x$，它仍可参加当前段；若 $y < x$，就把它冻结，留给下一段。可用记录耗尽时结束本段，把全部冻结记录解冻，重新开始。这里工作区只保持 m 条候选，输出段却可以随着不断换入而增长。

### 8.11.2 例 8-6　冻结记录在什么时候重新参加

**例 8.11.1** (配套案例)

【配套例题】对含 $n>0$ 个记录的文件外部排序，以置换—选择生成初始归并段，工作区能保存 m 个记录。

（1）文件 19 个关键字依次为 `51,94,37,92,14,63,15,99,48,56,23,60,31,17,43,8,90,166,100`，m=4 时生成几个初始段？各是什么？

（2）对任意 m（$n \gg m > 0$），第一个初始段长度最大、最小值分别是多少？

**错项排除·下界没有否定计数排序**

比较排序下界假设只凭两两比较区分互异关键字排列；计数索引利用整数取值范围，成本还含 k。若 k 很大，线性于 n 的口号忽略了计数数组和前缀和代价。

**解析**。先装入 51、94、37、92，输出 37 后读入 14，因为 14<37 而冻结；输出 51 后读入 63，63 仍可参加本段。前 6 次输出和换入如下：

| 输出 | 新读入 | 新记录归属 |
|---|---|---|
| 37 | 14 | 冻结 |
| 51 | 63 | 当前段 |
| 63 | 15 | 冻结 |
| 92 | 99 | 当前段 |
| 94 | 48 | 冻结 |
| 99 | 56 | 冻结 |

此时工作区 14、15、48、56 全被冻结，第一段结束。解冻后继续，同样逐次选最小；读入 17、43、8 和 100 时，它们分别小于当时最后输出的 31、48、56 和 166，留给下一段。最终得到 3 段：

1. `37,51,63,92,94,99`;
2. `14,15,23,31,48,56,60,90,166`;
3. `8,17,43,100`。

每个原记录恰好输出一次，三段长度 $6+9+4=19$，段内均递增。第一个初始段最多可含全部 n 项，输入已经非降时即如此；最少为 m 项，例如输入严格递减，每个新读入都冻结，但最初 m 个仍要全部输出才能结束。答案是**最大 n、最小 m**，不是每段都恰好 m 或 2m。

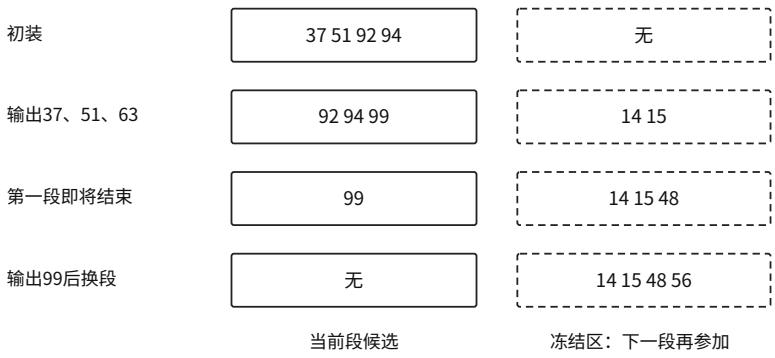



Figure 8.3: 实框为当前段候选，虚框为冻结记录；全部冻结时才换段

### 8.11.3 败者树怎样减少每次输出后的比较

k 路归并每次都要找各段当前首记录中的最小者。败者树把各路作为叶，内部结点记录一次比赛的败者，并额外保存总冠军的路号。冠军输

出后，仅该路换入新首记录，它沿原比赛路径与保存的败者重新比较；每层只需一次关键字比较，更新约 $O(\log k)$。

例如 4 路首记录为 A:3、B:5、C:2、D:7，冠军是 C 的 2，内部保存 5、7 及最终败者 3。C 换入 6 后，先与 7 比较，再与 3 比较，冠军变为 A 的 3。这次更新不必重新比较未变化的 3 与 5。堆的下滤一般先比较两个孩子再与候选比较，败者树每层沿既定对手路径比较，过程不同；堆可以在中途已满足堆序时停止，败者树更新冠军路径则要传到总冠军位置。

**最小反例·输出段可以比内存更长**

最小直观情形：工作区 1 条，输入 1、2、3。每输出一条再换入更大者，三条属于同一段，内存始终只存 1 条。不能把段长固定等同工作区容量。

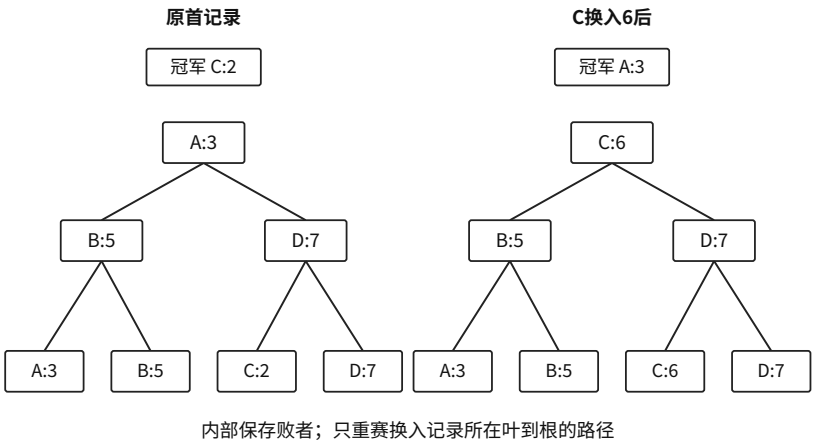



Figure 8.4: 两轮败者树；内部框存败者路号和值，冠军另外保存

### 8.11.4 例 8-7　不等长有序表怎样安排合并

**例 8.11.2** (配套案例)

【配套例题】6 个升序表 A、B、C、D、E、F，分别含 10、35、40、50、60、200 个元素。通过 5 次两两合并成为一个升序表，要求最坏情况下比较总次数最小。

（1）给出完整合并过程及最坏比较总次数。（2）描述 N 个不等长升序表的合并策略并说明理由。

**解析**。一张表每参加一次合并，它的全部元素就再次参与一层归并工作。让长表尽量晚参加、短表尽早参加，等价于最小化以表长为叶权的二叉归并树 WPL，即采用哈夫曼合并策略：

| 合并段长 | 新段长 | 本次最坏比较 |
|---|---|---|
| 10 与 35 | 45 | 44 |
| 40 与 45 | 85 | 84 |
| 50 与 60 | 110 | 109 |
| 85 与 110 | 195 | 194 |
| 195 与 200 | 395 | 394 |

最坏比较总数 $44+84+109+194+394=825$。合并长度之和即 WPL，为 830；每次归并比较至多“合并长度 $-1$”，共有 5 次，所以也可得 830−5=825。推广到 N 段，就是反复选当前两个最短段合并后放回候选集合，直到只剩一段，理由与哈夫曼最优树相同。

### 8.11.5 例 8-8　四路最佳归并为什么补零权段

**例 8.11.3** (配套案例)

〖8.7-004〗12 个初始段长分别为 30、44、8、6、3、20、60、18、9、62、68、85。进行 4 路外部归并，画出最佳归并树并计算 WPL。

**解析**。满 4 叉归并树的叶数应满足 $(r-1) \bmod 3 = 0$。12 不满足，补 1 个零权虚段得到 13 个叶。每次取 4 个最小权值：

`0+3+6+8=17`，`9+17+18+20=64`，`30+44+60+62=196`，最后 `64+68+85+196=413`。

所以 WPL 为 $17 + 64 + 196 + 413 = 690$。虚段不含真实记录，补它是为了让第一次归并允许不足 4 个真实输入，不会平白增加文件数据量。若每次合并段都完整读写，且段长按块计，归并阶段总块 I/O 为 $2 \times 690 = 1380$；这是按该模型计数，不包含此前生成初始段的读写。

### 8.11.6 外部归并和置换—选择的状态

```
mergeKRuns(runs):
    每路读入首缓冲块，另留输出块
    重复选择各非空路当前最小首记录，写入输出缓冲
    选中路的块用尽时读它的下一块
    输出块满时写外存；最后写出不足一块的剩余输出
```

```
replacementSelection(input,m):
    工作区读入至多m条，标记为当前段可用
    while 工作区非空:
        若无可用记录：结束当前段，全部冻结项解冻
        输出可用记录中最小的x
        若还有输入，读入y占用x释放的位置：
            若y<x则冻结，留下一段；否则本段可用
        否则删除该工作区位置
```

每个输入记录恰好换入并输出一次；冻结表示属于下一段，不能在当前段末尾直接按数值插回。工作区大小固定，输出段长度却可以继续增长，二者不是同一个量。

> **外部排序先统一单位再计 I/O**
>
> 依次写记录数、每块记录数、内存块数、初始段数、路数与趟数。若题目包含初始段生成，另计一次全文件读写；若按归并树 WPL 计不等长段合并，则逐内部结点累计读写，不能再乘一个全文件的统一趟数。

## 8.12 怎样比较和选择排序算法

下表比较的是**完整排序的总时间**，同时计入比较、移动和循环工作，不把一次划分、一次下滤或只建堆当作完整排序。辅助空间不计输入记录本身，但计入递归栈和另开输出数组；$\Theta$ 表示同阶，$O$ 只给上界。$r$ 为基数，d 为固定执行的位数，k 为计数排序的取值范围大小。比较排序的平均列按互异关键字的等概率随机排列分析；随机枢轴快排也可对其随机选择取期望，这些都不等于每份输入的保证。

**错项排除·下界没有否定计数排序**

比较排序下界假设只凭两两比较区分互异关键字排列；计数索引利用整数取值范围，成本还含 k。若 k 很大，线性于 n 的口号忽略了计数数组和前缀和代价。

| 算法 | 最好时间 | 平均时间 | 最坏时间 |
|---|---|---|---|
| 直接插入 | $\Theta(n)$ | $\Theta(n^2)$ | $\Theta(n^2)$ |
| 折半插入 | $\Theta(n \log n)$ | $\Theta(n^2)$ | $\Theta(n^2)$ |
| 起泡 | $\Theta(n)$ | $\Theta(n^2)$ | $\Theta(n^2)$ |
| 简单选择 | $\Theta(n^2)$ | $\Theta(n^2)$ | $\Theta(n^2)$ |
| 希尔 | 依增量序列 | 依增量序列 | 依增量序列 |
| 快速 | $\Theta(n \log n)$ | $\Theta(n \log n)$ | $\Theta(n^2)$ |
| 堆 | $O(n \log n)$，见下文 | $\Theta(n \log n)$ | $\Theta(n \log n)$ |
| 二路归并 | $\Theta(n \log n)$ | $\Theta(n \log n)$ | $\Theta(n \log n)$ |
| LSD 基数 | $\Theta(d(n+r))$ | $\Theta(d(n+r))$ | $\Theta(d(n+r))$ |
| 计数 | $\Theta(n+k)$ | $\Theta(n+k)$ | $\Theta(n+k)$ |

| 算法 | 辅助空间与稳定性 | 重要实现条件 |
|---|---|---|
| 直接插入 | $O(1)$；稳定 | 只移动严格大于新项的记录 |
| 折半插入 | $O(1)$；稳定 | 每趟都折半定位；插在旧相等项之后 |
| 起泡 | $O(1)$；稳定 | 有无交换标志可提前停；仅逆序相邻项交换 |
| 简单选择 | $O(1)$；不稳定 | 每趟选最小后与首位交换；比较固定 |
| 希尔 | $O(1)$；不稳定 | 逐组插入，末增量为 1；未给增量不填统一时间阶 |
| 快速 | 栈平均 $O(\log n)$，最坏 $O(n)$；不稳定 | 本章单枢轴划分与双支递归；平均需随机模型 |
| 堆 | $O(1)$；不稳定 | 自底向上建堆、迭代下滤；最好紧界有条件 |
| 二路归并 | $\Theta(n)$；稳定 | 数组常规归并，不跳过已有序段；相等先取左段 |
| LSD 基数 | 链式额外 $O(r)$；数组 $O(n+r)$；稳定 | 固定执行 d 轮；同桶保序；链式原记录已有链接域 |
| 计数 | $\Theta(n+k)$；稳定 | 计数、前缀和、逆向放入输出数组的记录实现 |

堆排序的教材汇总表在关键字互异的常规模型下，把最好、平均、最坏都列为对数线性阶；本表最好一栏保留总时间上界，并提醒具体停机条件：若允许所有关键字相等，下滤比较到相等即停止，建堆和全部输出可只做 $\Theta(n)$ 工作。不能把教材汇总表的最好紧界无条件套到这类重复输入上，也不能因这一特例删去堆排序的 $\Theta(n \log n)$ 最坏时间。起泡若无提前停止标志，则有序输入仍执行固定二次比较；归并若另加跳过有序段的优化，最好情况也应随实现重算。

规模小、接近有序时，直接插入移动少、控制简单；需要可靠的最坏对数线性时间且额外空间有限时，可考虑堆；要求稳定而可用线性辅助区时，可用归并；关键字可分成少量固定位，或整数取值范围较小时，可分别比较基数排序、计数排序的代价。外存排序则优先设计初始段和多路归并，不能只把内部快排换到磁盘上逐元素访问。

移动、比较和趟数不能互相替代。简单选择比较量不受初始次序影响，交换次数却依实现而定，某次交换可以在代码中省去；折半插入比较量变化小，移动量仍依赖初始排列；LSD 按固定 d 位执行时，分配收集的轮数固定；快速排序的划分深度取决于划分结果。并行能力也依实现：快排两子区间可独立，归并同层不同段可独立；LSD 的下一位依赖本位稳定结果，但每一位内部仍可设计并行分配。不能把某个串行教材步骤的依赖，扩大成算法在任何实现下“绝不可能并行”。

**不变量·LSD 依赖每轮的稳定性**

更高位相同的记录仍要靠已经排好的低位决定先后，故同桶必须保留上轮次序。改成插桶头或逆序收集可能破坏这个不变式，即使每轮单独看都按本位有序。

**定理 8.12.1** (比较排序下界的适用模型)

仅通过两两关键字比较区分 $n$ 个互异关键字的全部排列时，最坏比较次数为 $\Omega(n\log n)$。

**决策树至少要容纳全部排列**

互异关键字有 $n!$ 种相对次序，一棵二分比较决策树至少需要 $n!$ 个叶。高度为 $h$ 的二叉树至多 $2^h$ 个叶，所以 $h \geq \lceil \log_2(n!) \rceil = \Omega(n\log n)$。计数与基数利用取值或数位结构，操作不只在这棵两两比较树中分支，故不与其线性特殊模型矛盾。

□

**课程连接·外排瓶颈联系操作系统 I/O**

外排优先按块读写，输出缓冲攒满再写可减少细粒度 I/O。多路需要更多输入缓冲，路数受实际内存约束；双缓冲与异步 I/O 还会改变可用块数和耗时模型。

## 8.13 主动回忆自测

第 2 题为配套例题，其余为自拟补充，检查模型变化。

1. 直接插入把大于条件改为大于等于，时间阶没变，哪项性质可能改变?
2. 【配套例题】n（n>100000）个数保存在一维数组 M 中，需要查找最小的 10 个数。（1）设计算法，要求平均情况下比较次数尽可能少，简述思想，不需要程序实现。（2）说明平均时间复杂度和空间复杂度。
3. 100 个初始段，最多 4 路归并，需要几趟? 若每趟完整读写 500 块，归并阶段多少块传输?
4. 一个不稳定排序在没有相等关键字的输入上，能否观察到稳定性差异?
5. 直接选择用严格小于找最小值，为什么仍可能不稳定? 给最小三记录反例。
6. 折半插入将相等时的分支改为向左找，会影响比较次数的阶还是稳定性?
7. 两个长度分别为 2、3 的有序段，归并一定需要 4 次关键字比较吗?
8. 置换—选择工作区 4 条，输入已递增的 100 条记录，第一段最多多长? 内存是否因此装入 100 条?

**答案**。1. 可能破坏稳定性，新相等记录移到旧记录前。2. 用随机枢轴划分，把第 10 小作为选择目标，只继续处理包含这一秩的子区间；其余一侧无需继续排序。确定分界后，左侧保留 10 个最小数，边界有相等值时按还需个数选取。平均时间 $O(n)$，在原数组上迭代选择可用 $O(1)$ 辅助空间；题目不要求这 10 个数内部有序。也可维护大小 10 的大根堆：只用小于堆顶的数替换并下滤，适合流式读入，代价为 $O(n\log 10)$、辅助 $O(10)$，不能把它误写成必须完整排序。3. $\lceil \log_4 100 \rceil = 4$ 趟，传输 $4 \times 2 \times 500 = 4000$ 块；若包含初始段生成，再加 1000 块。4. 不能。输入中不存在相等记录，但算法的稳定性定义仍按所有合法输入判断。

5. $2_a, 2_b, 1$ 第一趟把 1 与 $2_a$ 远距离交换，得到 $1, 2_b, 2_a$，破坏稳定性；找最小值时没有因相等更新 pick 也无法避免。
6. 比较阶通常不变，但把新相等记录插到旧相等记录之前，稳定性可能破坏。
7. 不一定，4 只是最坏上界；若较短段的所有值都小于另一段，只比较 2 次就直接复制余段。
8. 可长 100 条；输出一条就换入一条，工作区始终最多 4 条，段长不等于同时驻留的记录数。